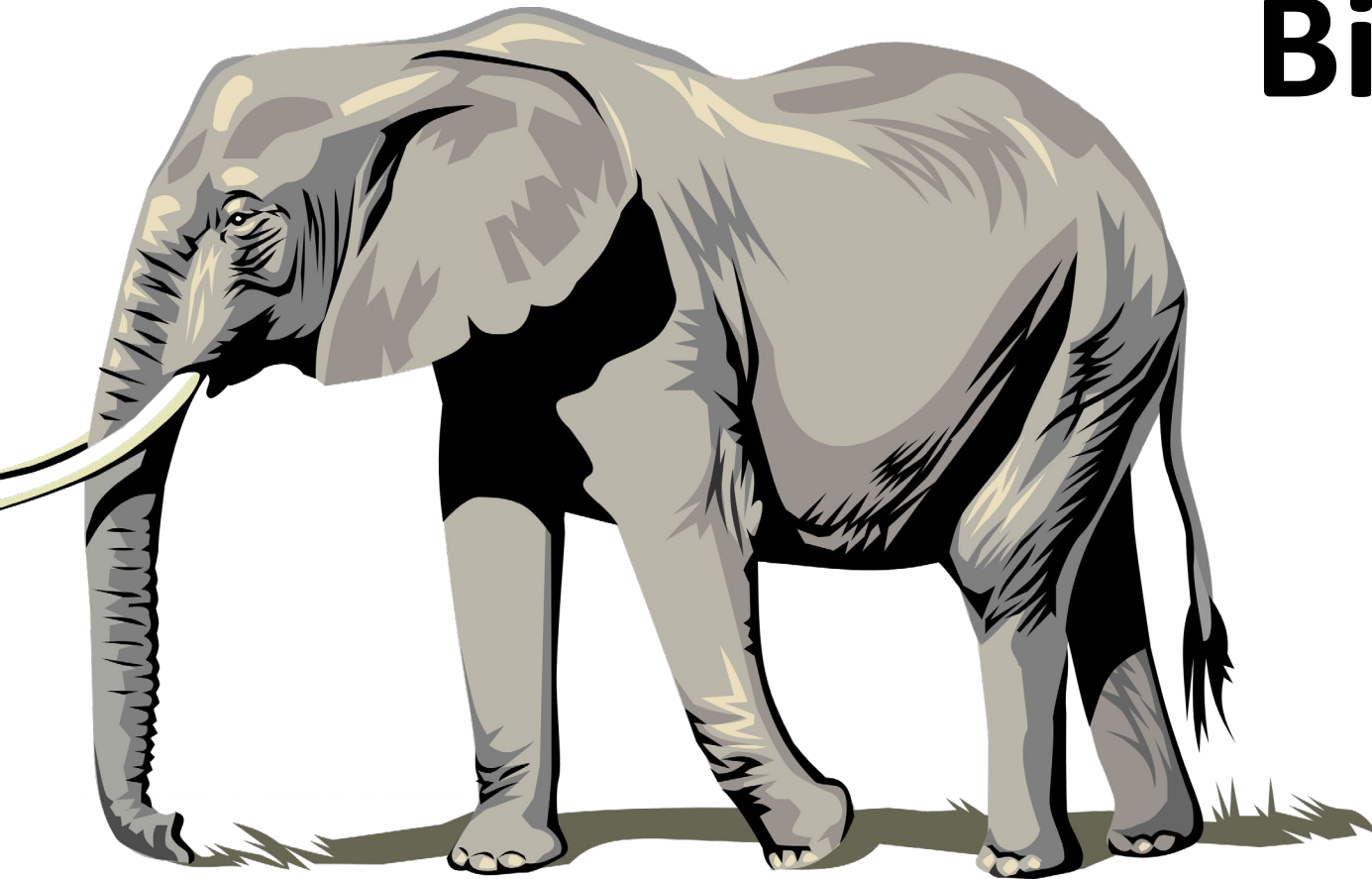


NOVA

IMS

Information
Management
School

data with **purpose**



Big Data Analytics

NoSQL

Lecturer:

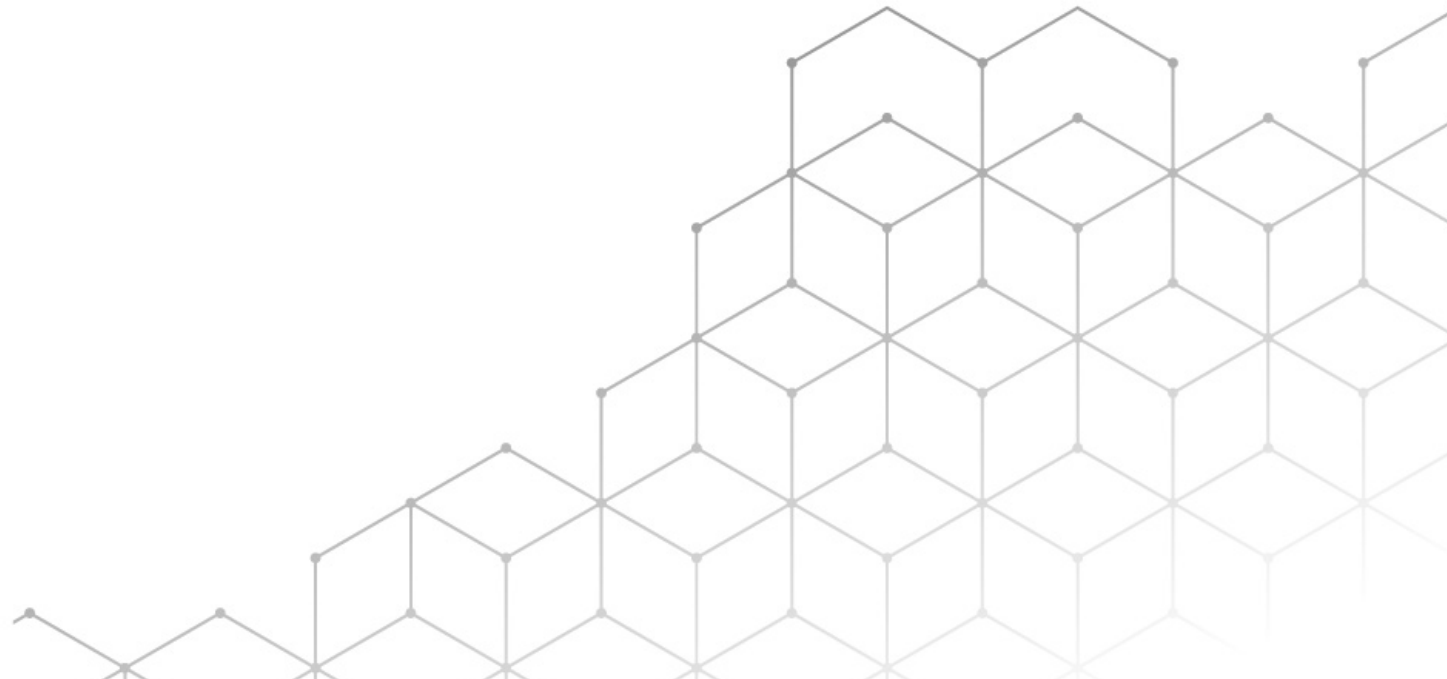
Márcia Lourenço Baptista

m.baptista@novaims.unl.pt

Agenda for Today



- **Introduction**
- History of Databases
- Relational Databases
- Distributed Databases
- NoSQL Databases

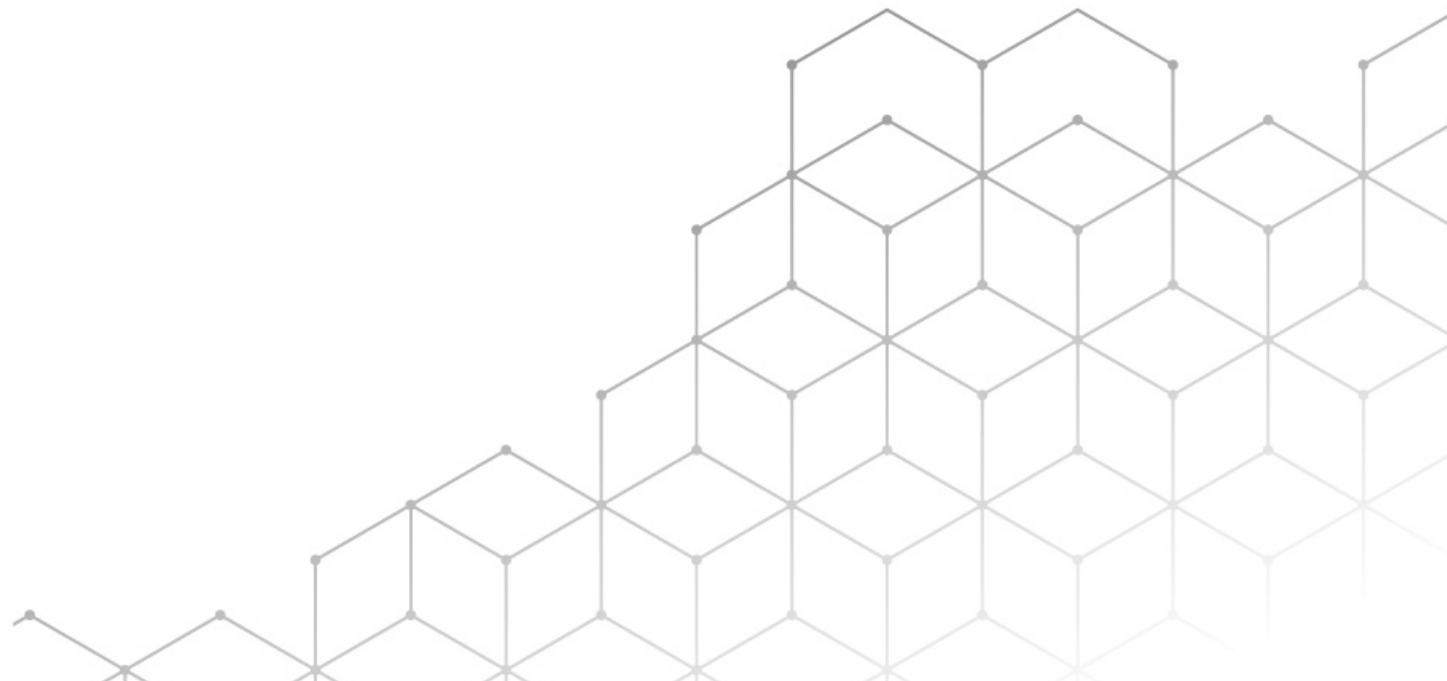


Agenda for Today



- **Introduction:** concepts and history of databases.

- Definition of Data and Database
- Types of Databases
- Types of Databases: Relational Database
- Types of Database: NoSQL Database
- Definition of DBMS
- Properties Ensured by DBMS
- History of Databases
- Relational Databases
- Distributed Databases
- NoSQL Databases



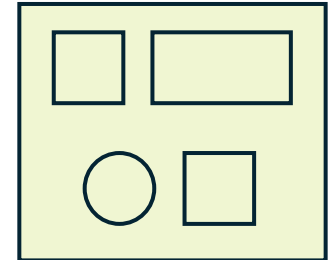
Definition of Data and Database

Definition of Data:

Word 'Data' is originated from the word 'datum' that means 'single piece of information.' It is plural of the word datum.

Data is a collection of a **distinct small** unit of information.

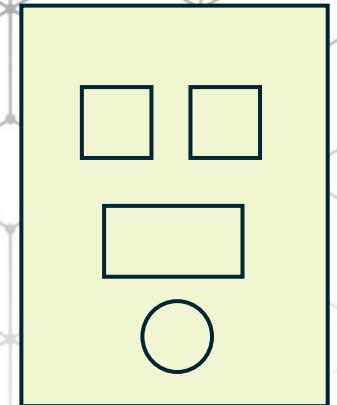
Data



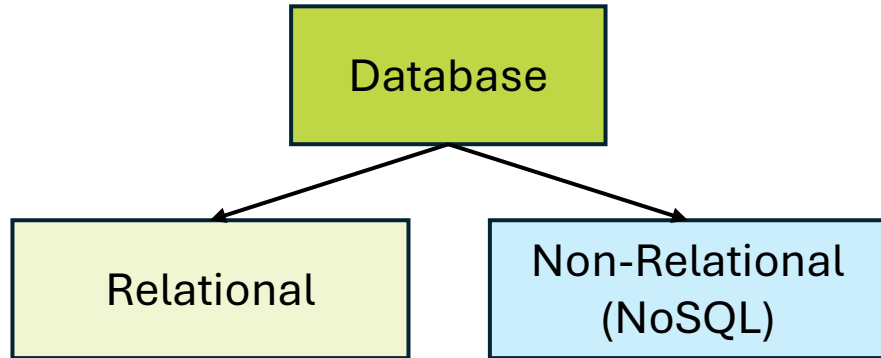
Definition of Database:

Database is an organized collection of data. Stored digitally in one or more computer systems.

Database



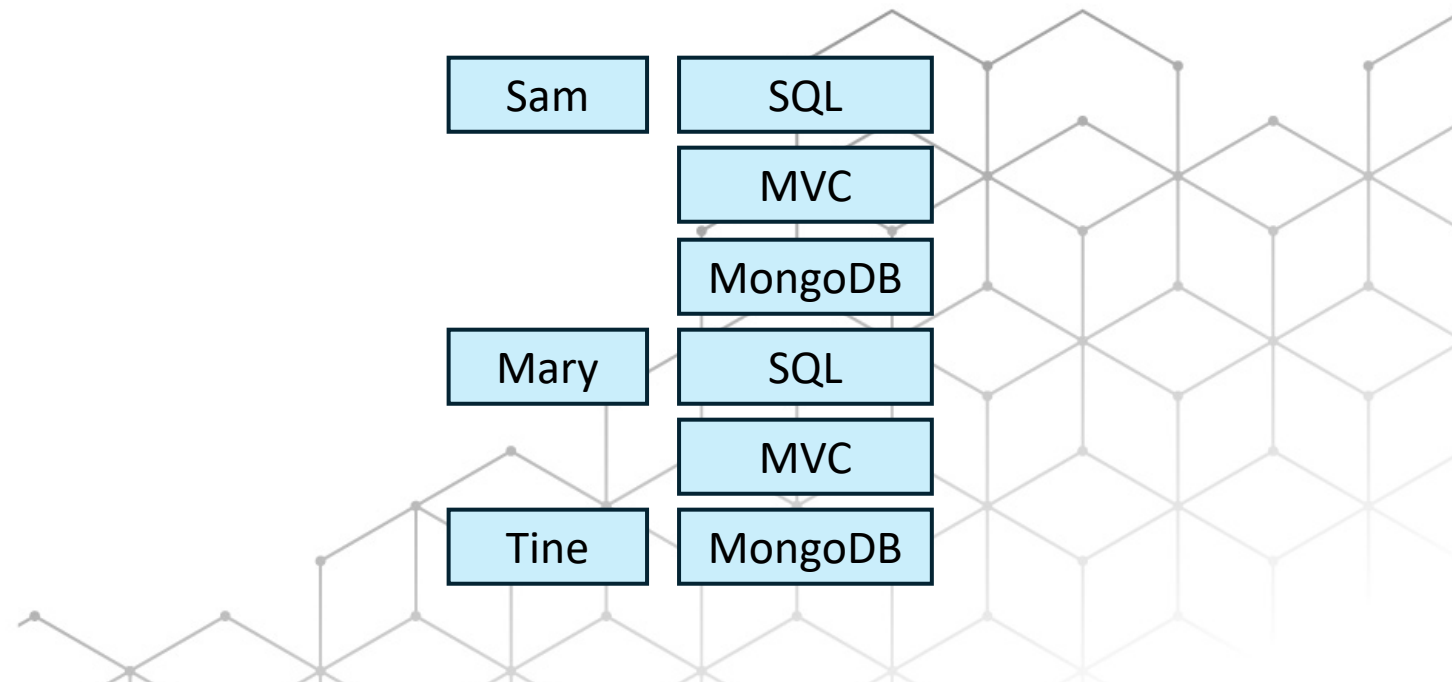
Types of Databases



Students	
Id:	Name:
1	Sam
2	Mary
3	Tine

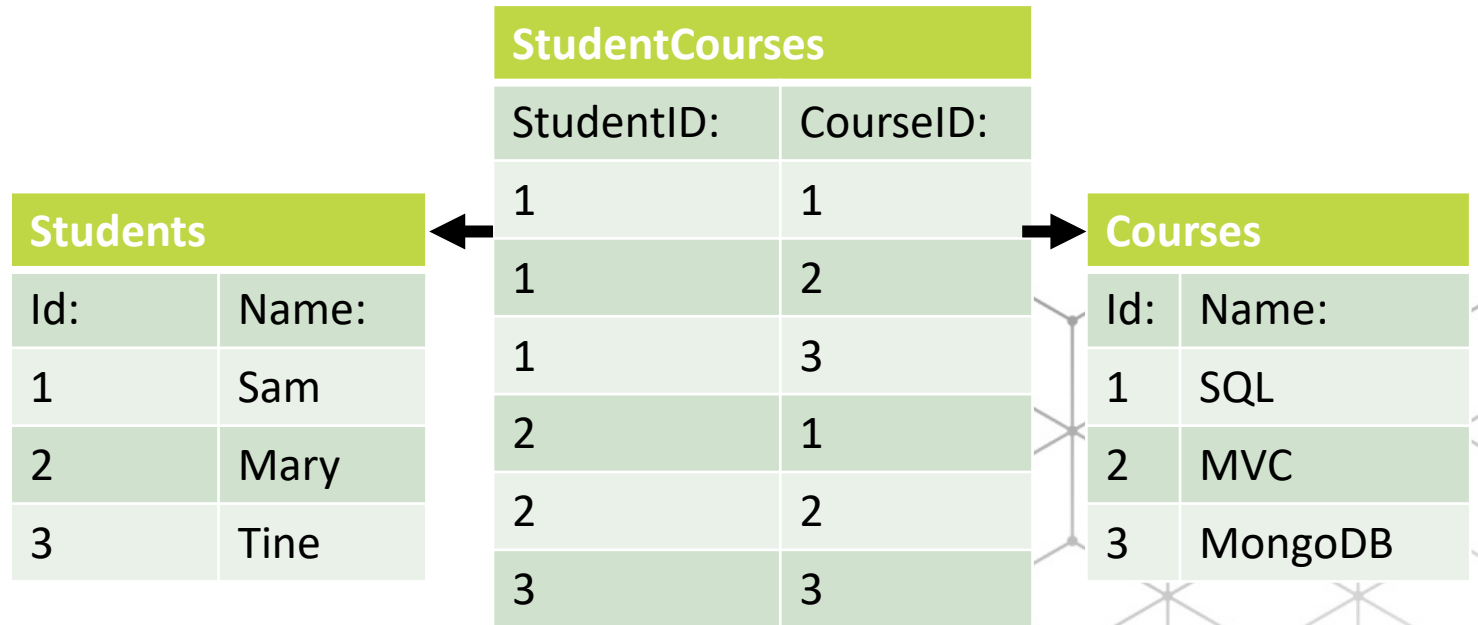
StudentCourses	
StudentID:	CourseID:
1	1
1	2
1	3
2	1
2	2
3	3

Courses	
Id:	Name:
1	SQL
2	MVC
3	MongoDB



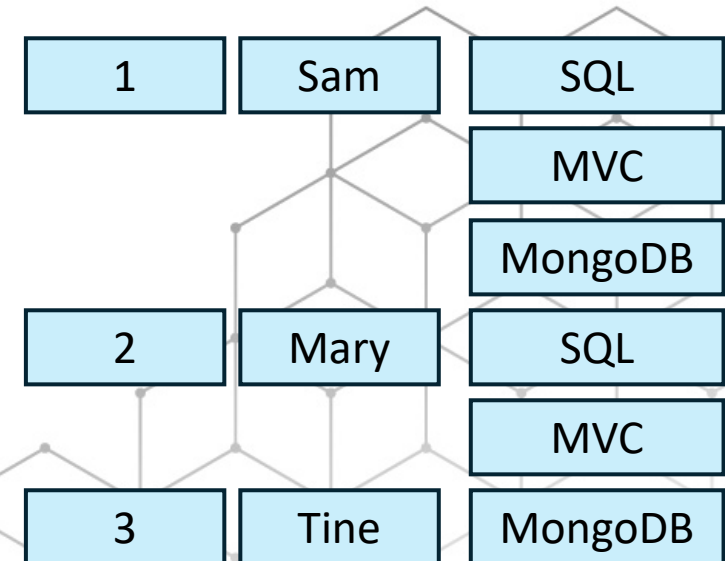
Types of Database: Relational Database

- Relational Database is simply a set of tables with rows and columns.
- This **structured** format allows **easy access**, **modification**, and **organization** of data



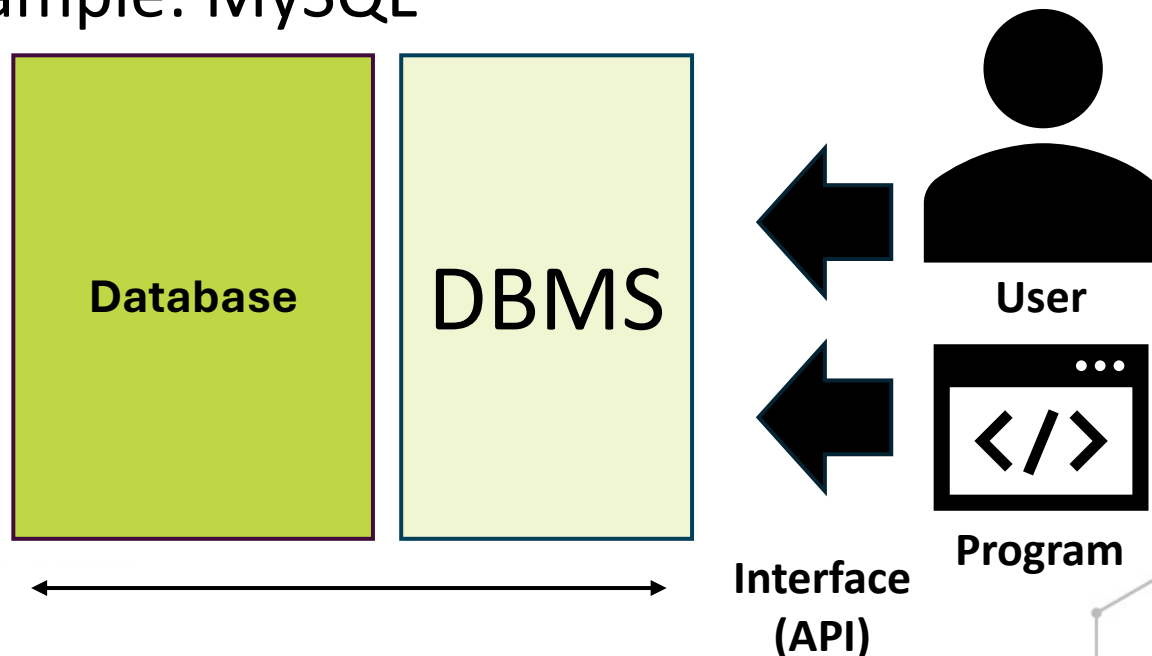
Types of Database: NoSQL Databases

- NoSQL databases are not primarily built on tables
- Do typically do not use SQL to manipulate data
- Less stringent data model and does not have a fixed schema.
- May not stick to the ACID properties



Definition of DBMS

- A database management system is a computer program designed to manage a database storage at one place. We structure a large set of data and run operations on the data requested by users/programs.
- Example: MySQL



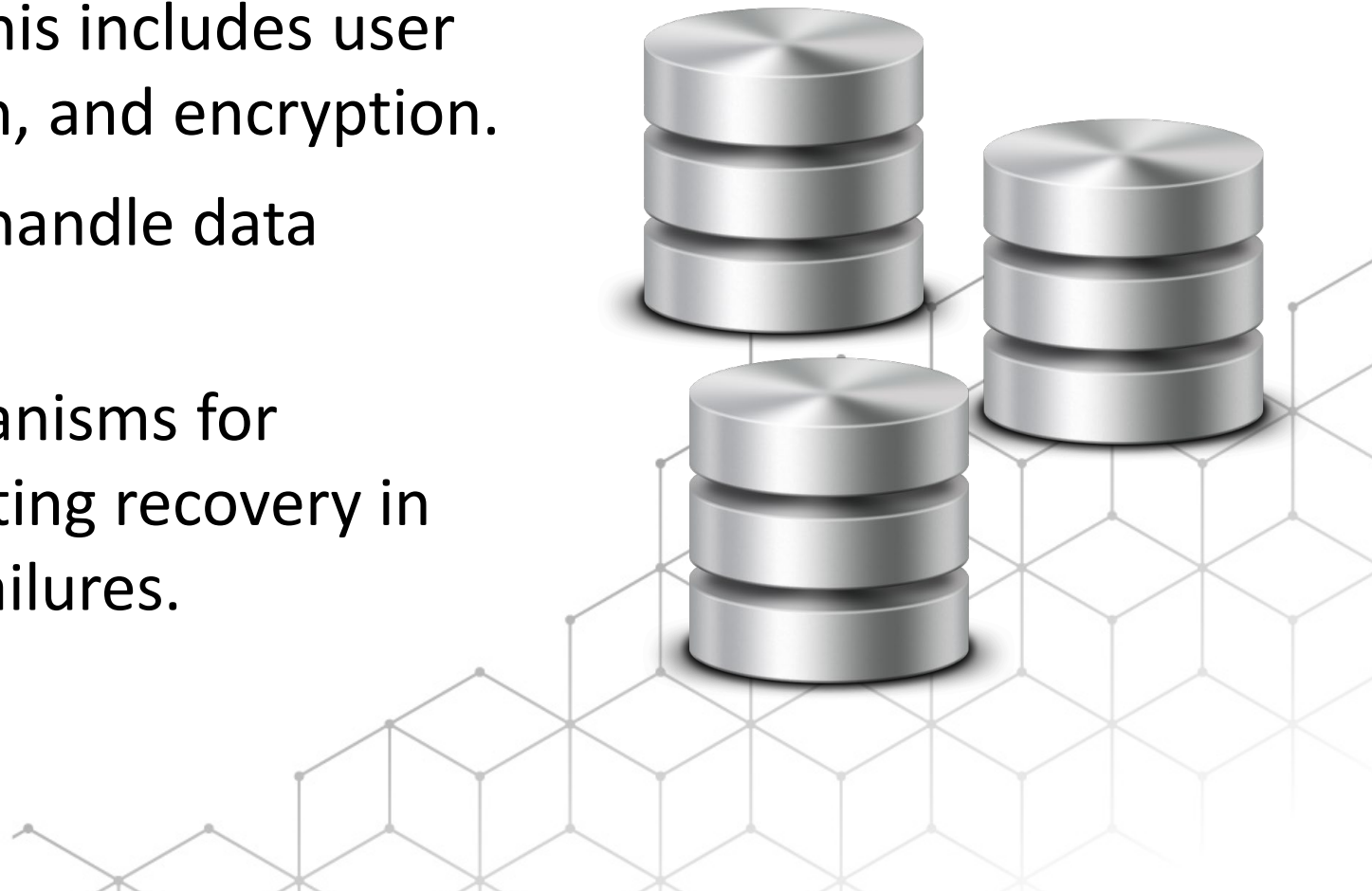
Properties Ensured by DBMS (1)

- Structure: A database organizes data within some structure.
- Data Management: A database can be managed (insert, update, delete, get)
- Data Integrity: A database tries to have a certain level of integrity.
- Concurrency Control: A database often supports multiple users



Properties Ensured by DBMS (2)

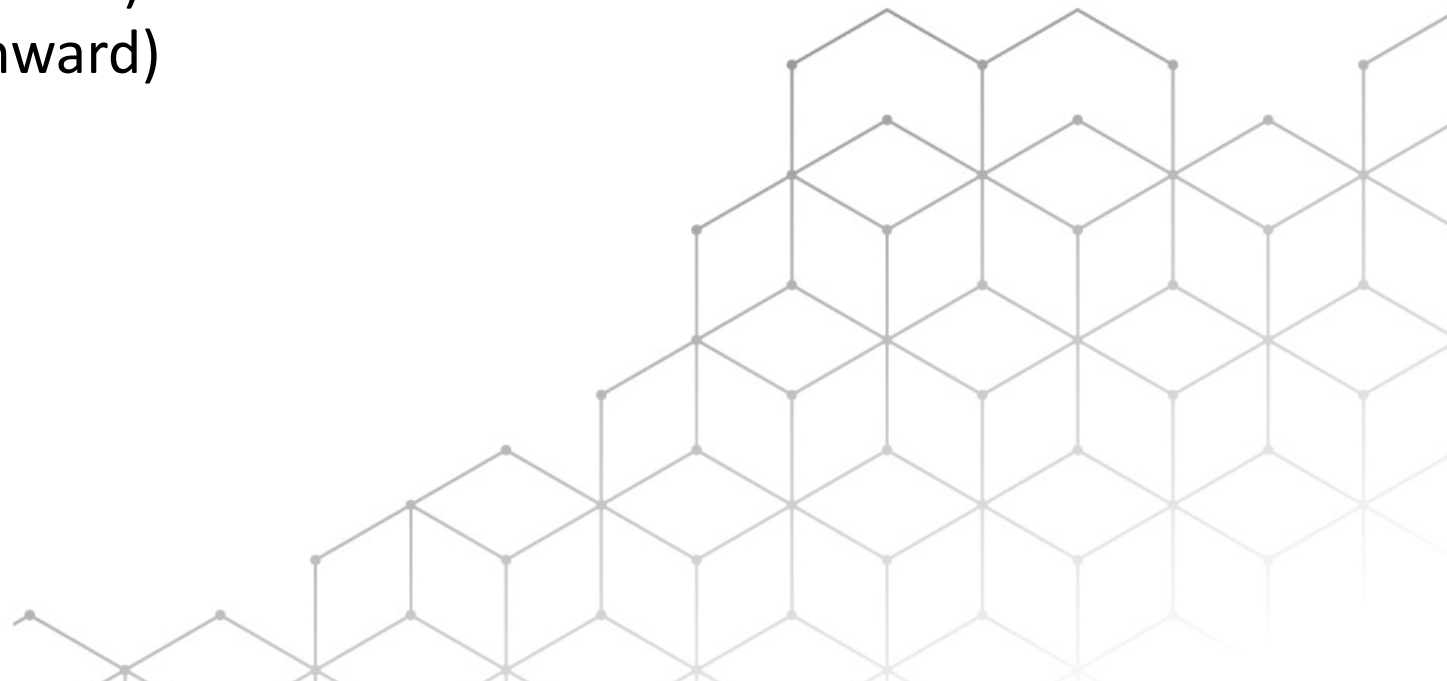
- **Security:** A database has mechanisms to control access to the data. This includes user authentication, authorization, and encryption.
- **Scalability:** Databases often handle data volume and user loads.
- **Backup and Recovery:** mechanisms for creating backups and facilitating recovery in case of data loss or system failures.



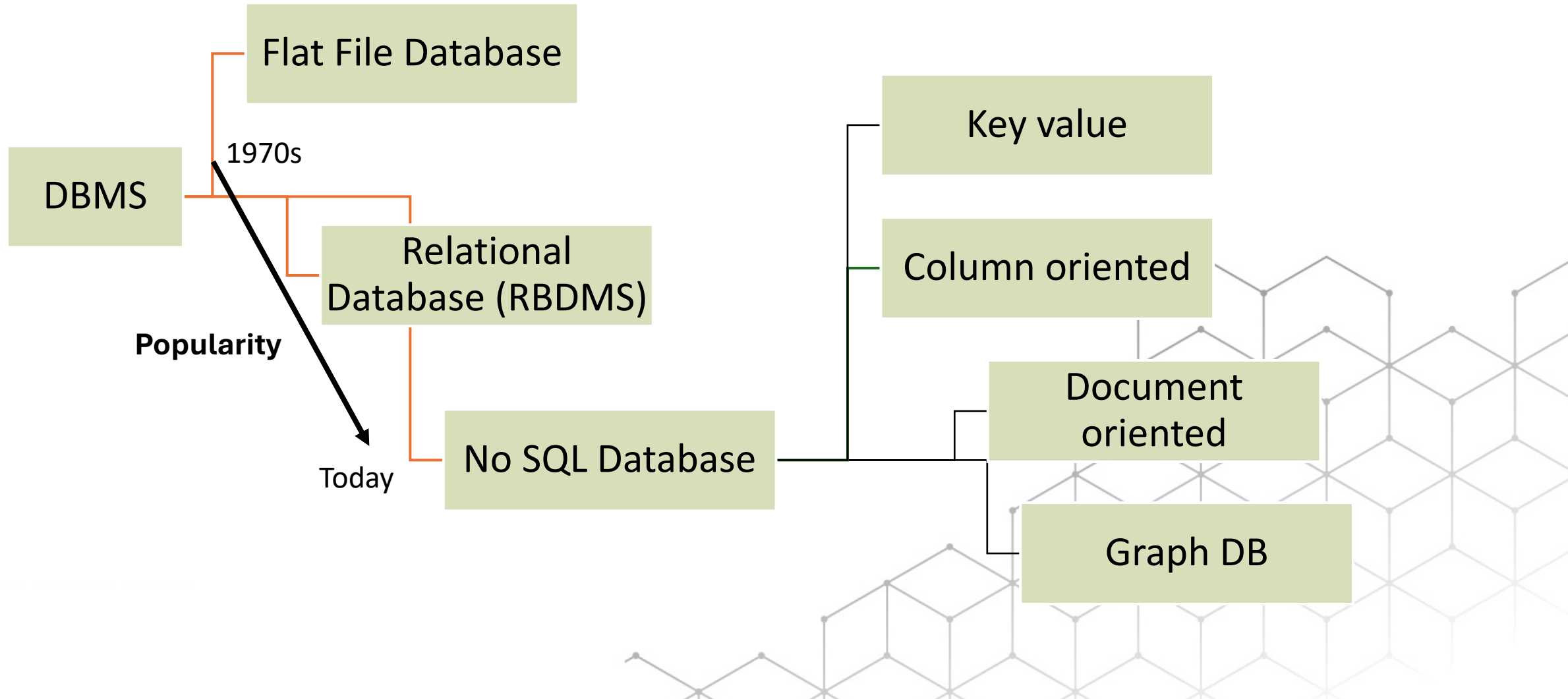
Agenda for Today



- Introduction
- History of Databases: evolution of DBMS.
 - Evolution of DBMS
 - Flat File “Database” (1970s)
 - Hierarchical Database (1970,1980s)
 - Relational Database (1970s onward)
 - NoSQL Database
 - Types of NoSQL
- Relational Databases
- Distributed Databases
- NoSQL Databases



Evolution of DBMS



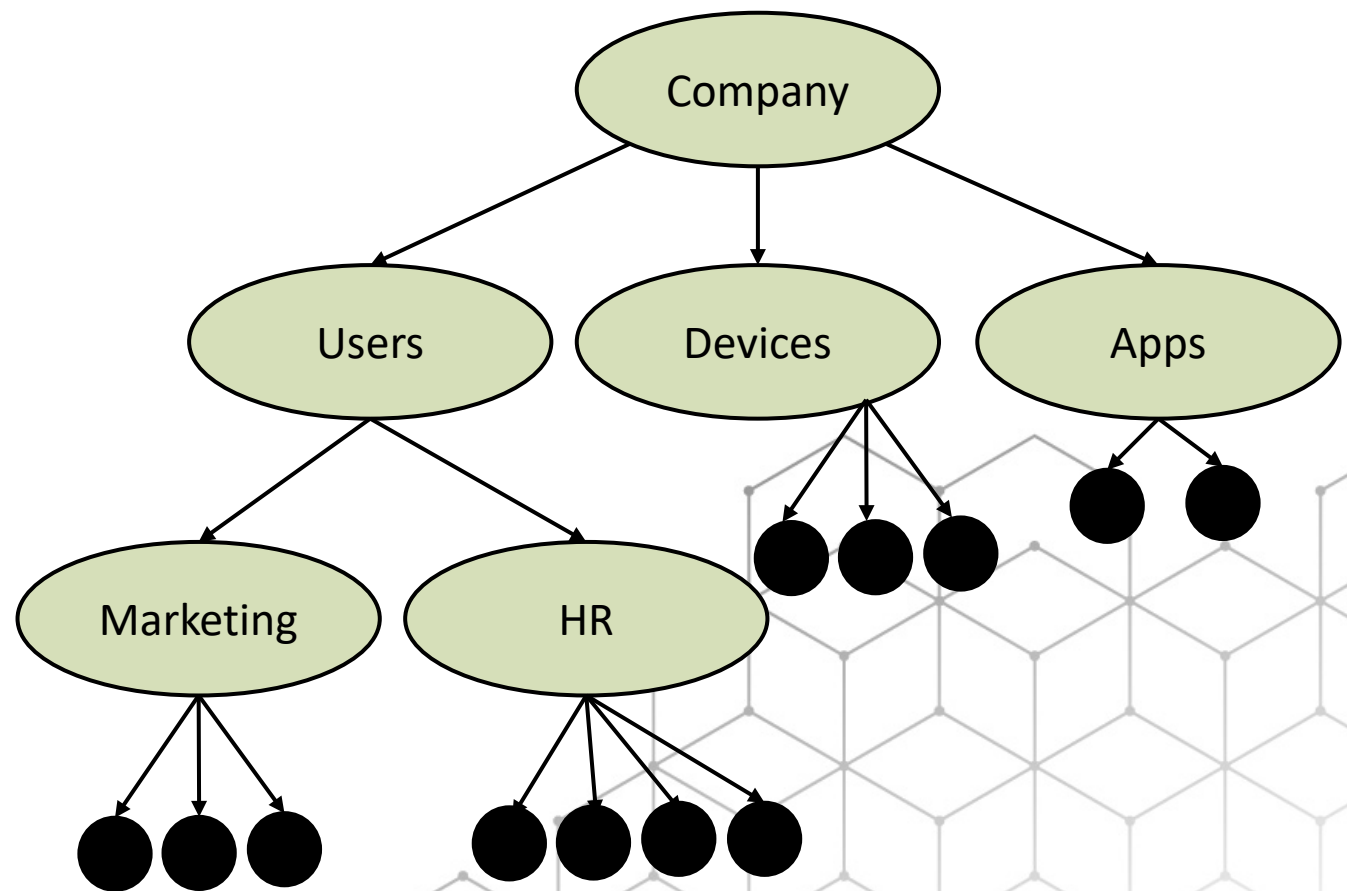
Flat File “Database” (1970s)

- In file-based databases, data is in a flat file.
- There are no structures for indexing or recognizing relationships.
- Can be a plain text (txt) or a binary file.
- Data arranged in a single big table.

```
id,name,team  
1,Amy,Blues  
2,Bob,Reds  
3,Chuck,Blues  
4,Richard,Blues  
5,Ethel,Reds  
6,Fred,Blues  
7,Gilly,Blues  
8,Hank,Reds  
9,Hank,Blues
```

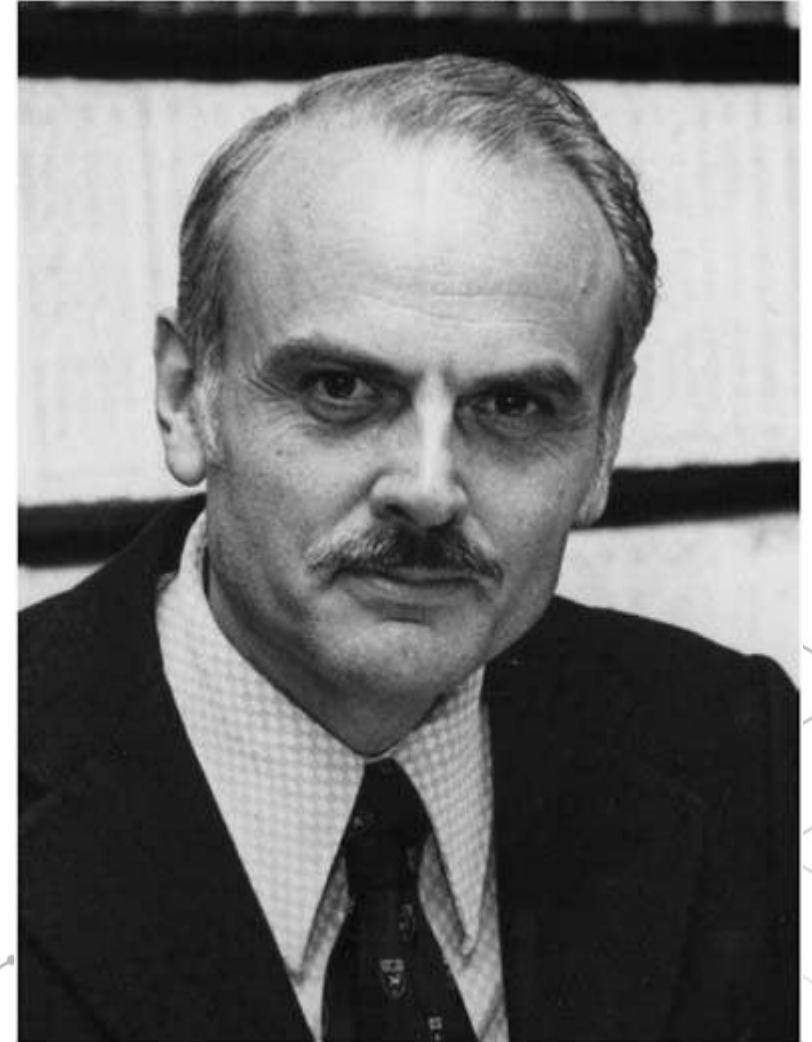
Hierarchical Database (1970s, 1980s)

- 1970s and 1980s was the era of the Hierarchical Database.
- A popular hierarchical database model was IBM's first DBMS. It was called **IMS** (Information Management System).
- In this model, files are related in a parent/child manner.



Relational Database

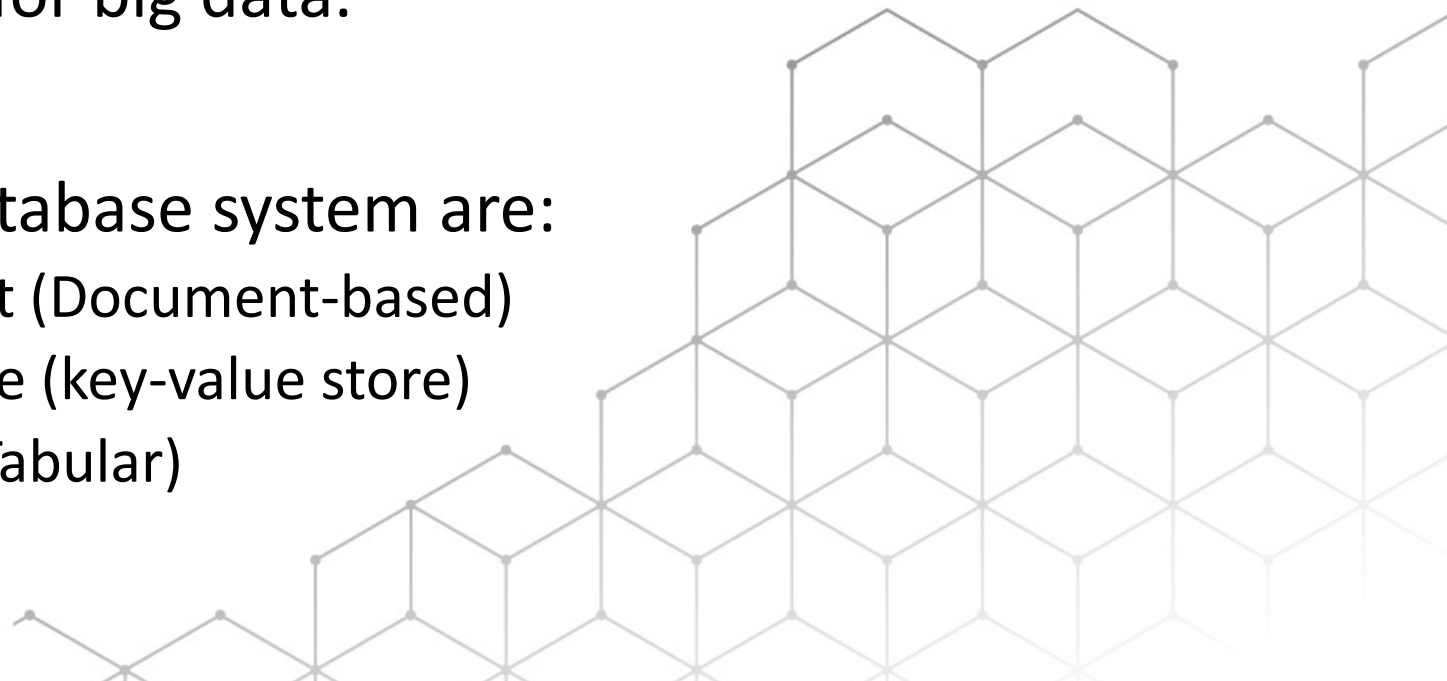
- 1970 - Present: It is the era of Relational Database and Database Management. In 1970, the relational model was proposed by E.F. Codd (IBM).
- Relational database model has two main terminologies called instance and schema.
- The instance is a table with rows or columns
- Schema specifies the structure like name of the relation, type of each column and name.



A handwritten signature in black ink, likely belonging to E.F. Codd, positioned below the portrait. The signature is stylized and cursive.

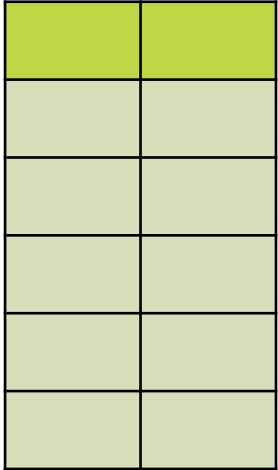
NoSQL Database (1970s onwards)

- NoSQL database stands for "not only SQL."
- It is an alternative to traditional relational databases.
- Accommodate a wide variety of data models
- NoSQL databases are useful for big data.
- Some examples of NoSQL database system are:
 - MongoDB, CouchDB, Cloudant (Document-based)
 - Memcached, Redis, Coherence (key-value store)
 - HBase, Big Table, Accumulo (Tabular)

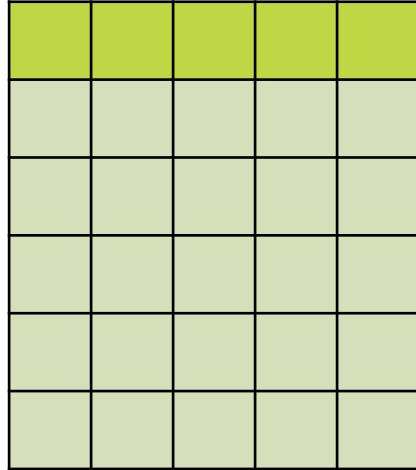


Types of NoSQL

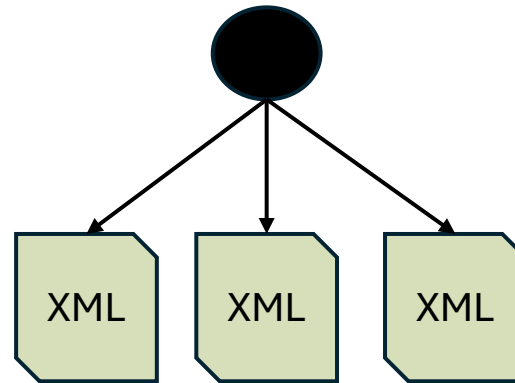
Key-Value



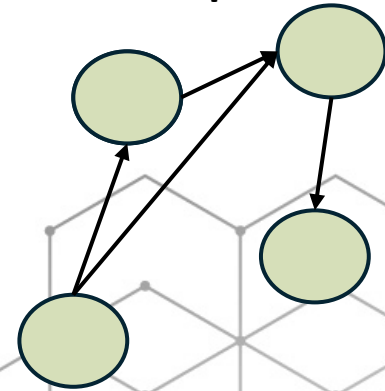
Column



Document



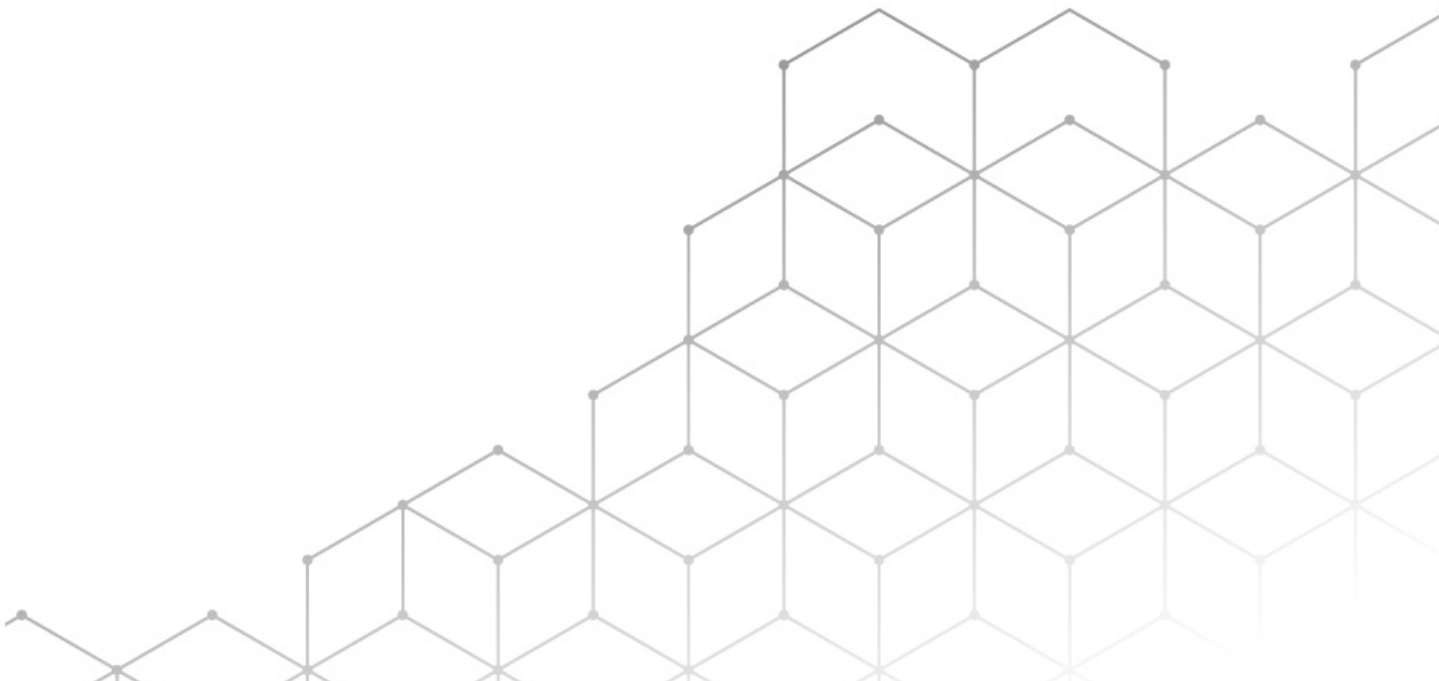
Graph



Agenda for Today



- Introduction
- History of Databases
- Relational Databases: remember basic principles of RDBMS
 - Relational Data Model
 - Codd's 13 Rules
 - Rule Zero
 - Information Rule
 - Guaranteed Access Rule
 - Systematic Treatment of Null Values
 - Dynamic Online Catalog
 - Based on the Relational Model
 - Comprehensive Data Sublanguage Rule
 - View Updating Rule
 - High-Level Insert, Update, and Delete
 - Physical Data Independence
 - Logical Data Independence
 - Integrity Independence
 - Distribution Independence
 - Non-Subversion Rule
- Distributed Databases
- NoSQL Databases



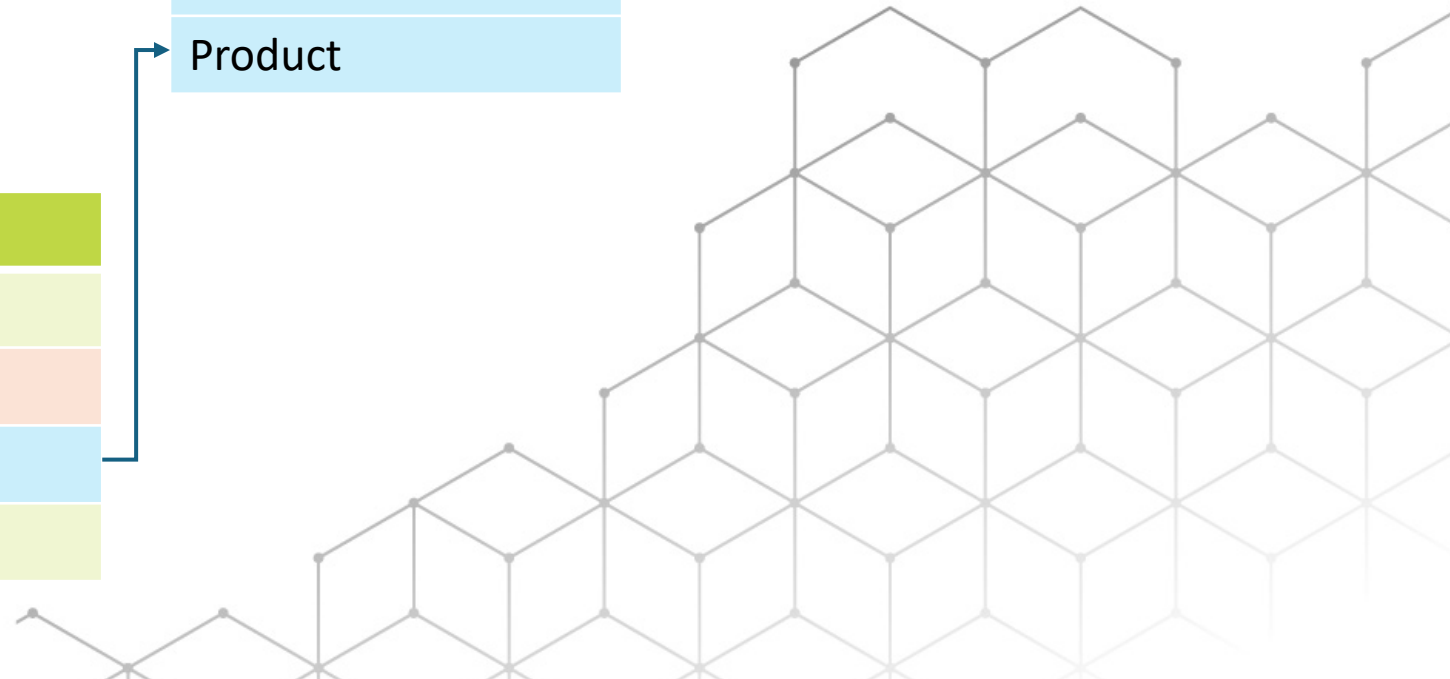
Relational Model

- Relational Data Model: represents data in the form of relations or tables.

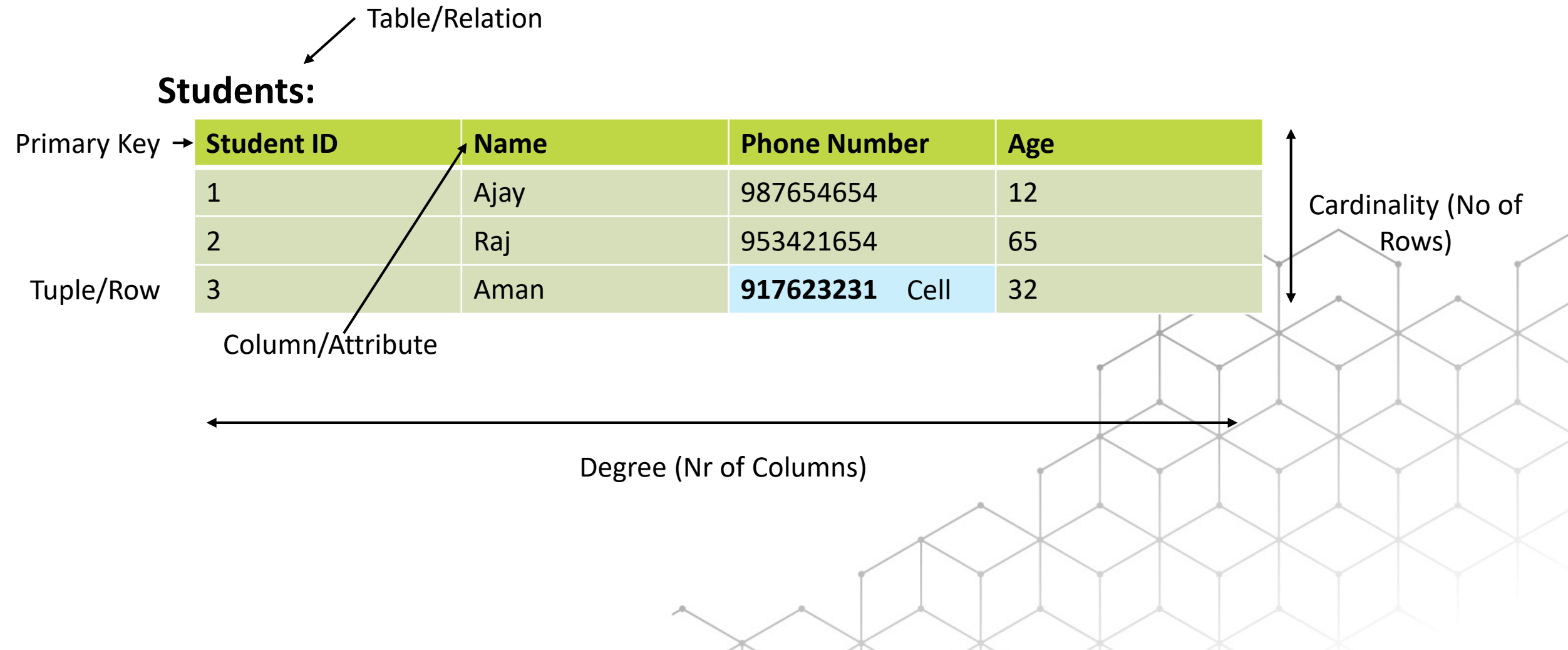
Sales Person
Sales Person ID
Sales Person Name
Age
City
Phone

Product
Product ID
Product

Sales
Date
Sales Person ID
Product ID
Sales



Relational Instance

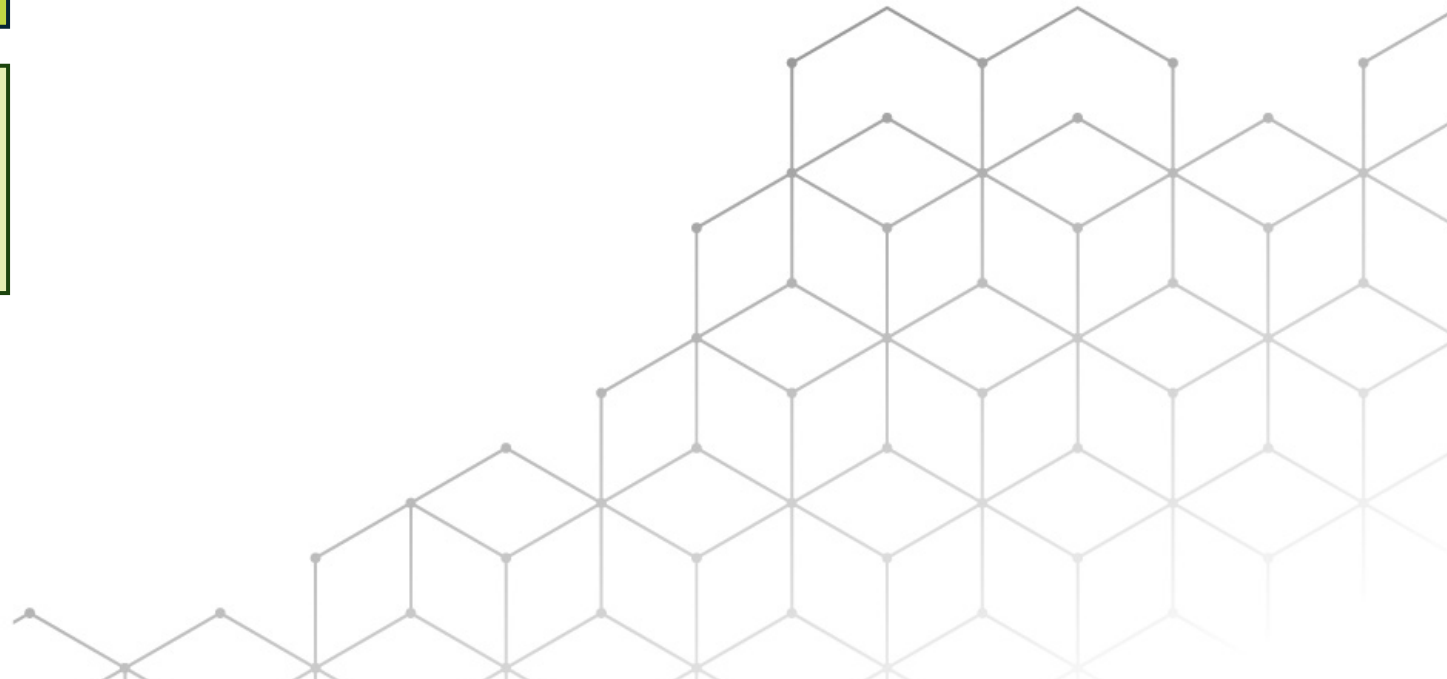
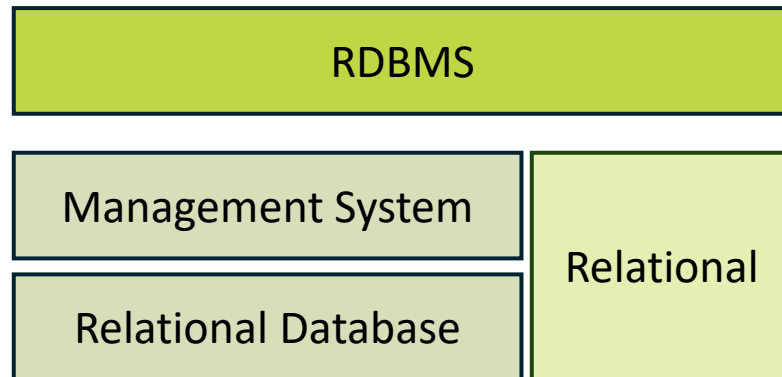


Codd's 13 Rules

- Codd proposed 13 rules and said that if a DBMS meets these, it can be called Relational Database Management System (RDBMS).
- Hardly any commercial product follows all (Codd's 12).
- Rules:
 - Rule Zero
 - Information Rule
 - Guaranteed Access Rule
 - Systematic Treatment of Null Values
 - Dynamic Online Catalog
 - Based on the Relational Model
 - Comprehensive Data Sublanguage Rule
 - View Updating Rule
 - High-Level Insert, Update, and Delete
 - Physical Data Independence
 - Logical Data Independence
 - Integrity Independence
 - Distribution Independence
 - Non-Subversion Rule

Foundation Rule (Rule 0)

For a RDBMS, the system must qualify as relational, as a database, and as a management system.



Information Rule (Rule 1)

- Data stored in Relational model must be the value of a cell in a table.

TABLE:		
	COLUMN	
	CELL	
ROW		

Guaranteed Access Rule (Rule 2)

- All data must be accessible. Every data element (cell) must be accessible by table name, its primary key, and name of the attribute whose value is to be determined.

The diagram illustrates the Guaranteed Access Rule (Rule 2) using a table named 'TABLE: CLIENT'. The table has three columns: ID, NAME, and TELEPHONE. The first column (ID) is highlighted in light green, and the first row (header) is also highlighted in light green. Three numbered arrows point to specific cells in the table:

- Arrow 1 points to the first column (ID).
- Arrow 2 points to the first row (header).
- Arrow 3 points to the cell containing '967609615' in the third column (TELEPHONE) of the first row (header).

TABLE: CLIENT		
ID	NAME	TELEPHONE
1234	Sam	969509615
1235	Matilde	919509615
1236	John	967609615

Treatment of Null Values (Rule 3)

- Null values are supported in relational DBMS
- They represent missing information or inapplicable information, independent of data type.

TABLE: CLIENT		
ID	NAME	TELEPHONE
1234	Sam	969509615
1235	Matilde	919509615
1236	John	NULL

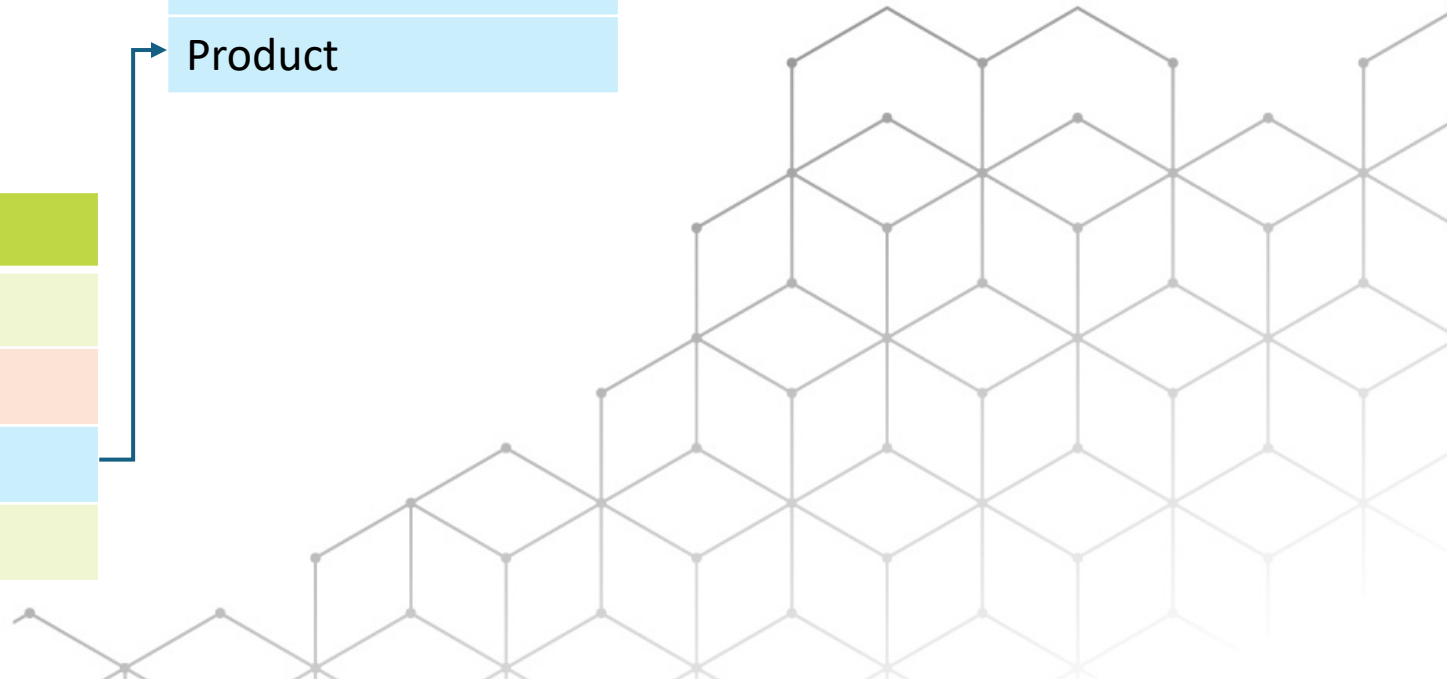
Active Online Catalog (Rule 4)

- That is, users must be able to access the database's structure (catalog).

Sales Person
Sales Person ID
Sales Person Name
Age
City
Phone

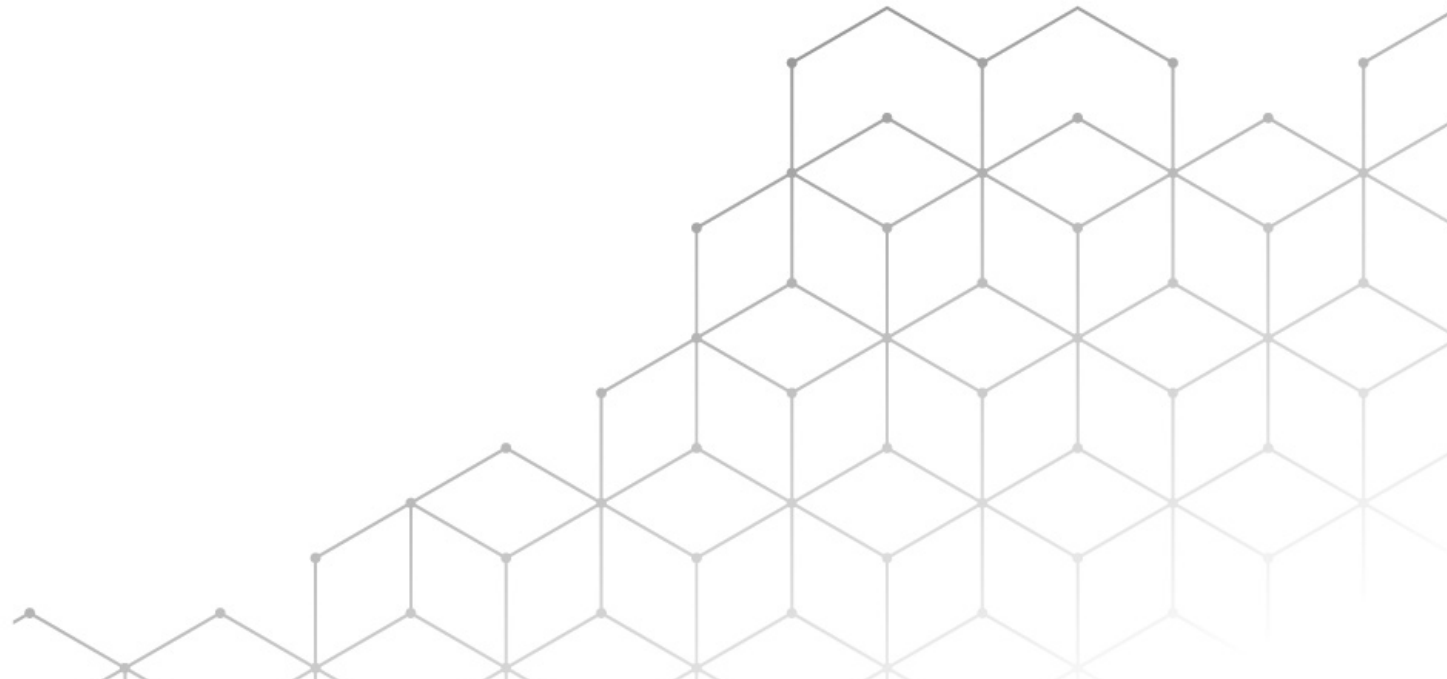
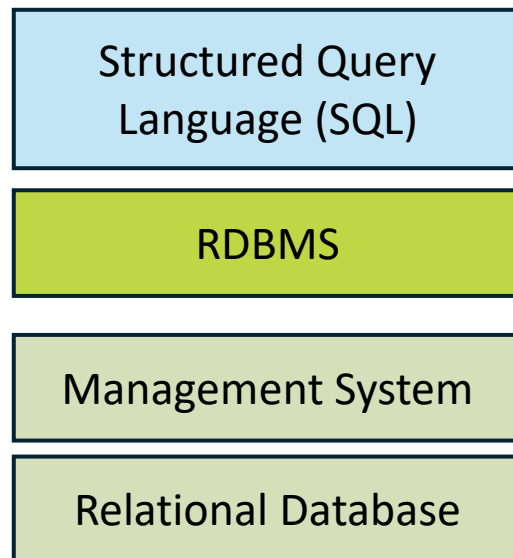
Product
Product ID
Product

Sales
Date
Sales Person ID
Product ID
Sales



Data Sub-Language Rule (Rule 5)

- A relational database should be accessible by a language supported for definition, manipulation and transaction management operation.



View Updating Rule (Rule 6)

If a view is formed as join of 3 tables, changes to view should be reflected in base tables

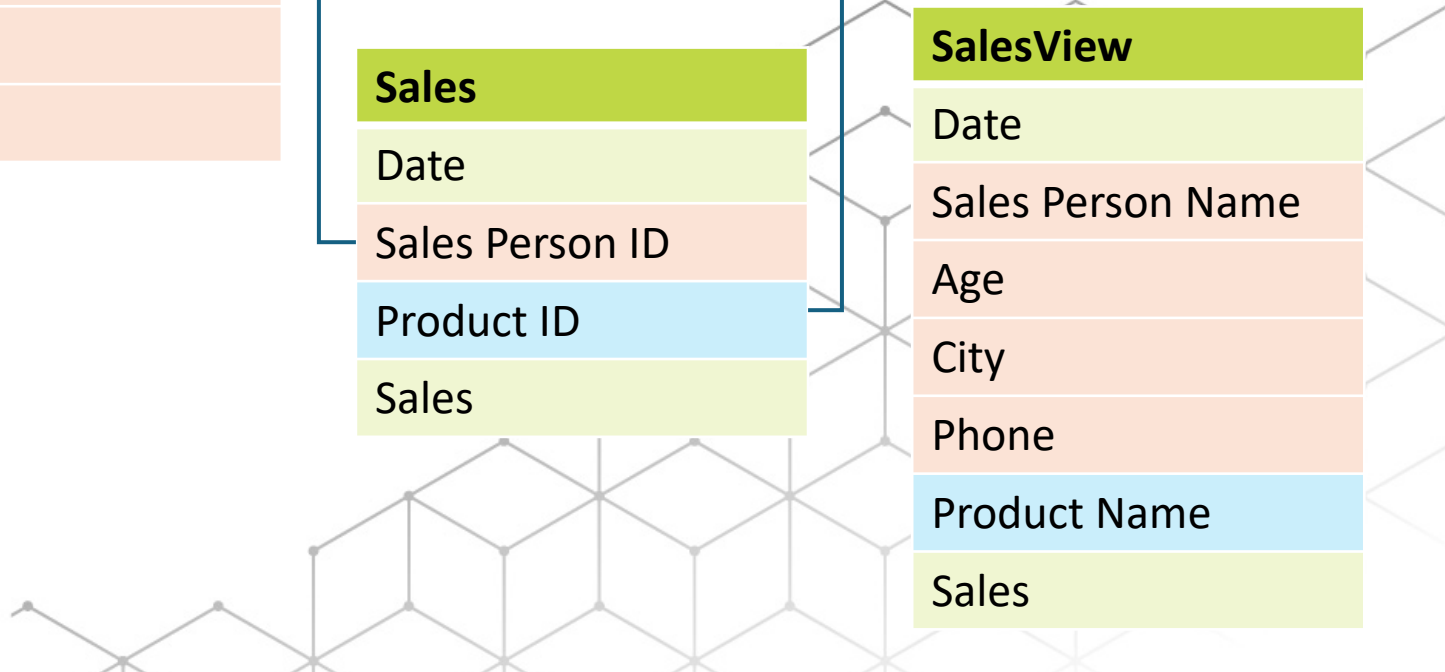
- Rule: any view is automatically updatable by the system.
- View is a **virtual table** used to give a different view of its structure.
- Not all views are updateable. Views can be updateable if they follow certain rules

Sales Person
Sales Person ID
Sales Person Name
Age
City
Phone

Product
Product ID
Product

Sales
Date
Sales Person ID
Product ID
Sales

SalesView
Date
Sales Person Name
Age
City
Phone
Product Name
Sales



High level insert, update, delete (Rule 7)

- Relational Model should support insert, delete, update operations.

- **Insertion:**

Insert into **Students** values ('1111', ...);

- **Deletion:**

Delete from **Students** Where sid = '1111';

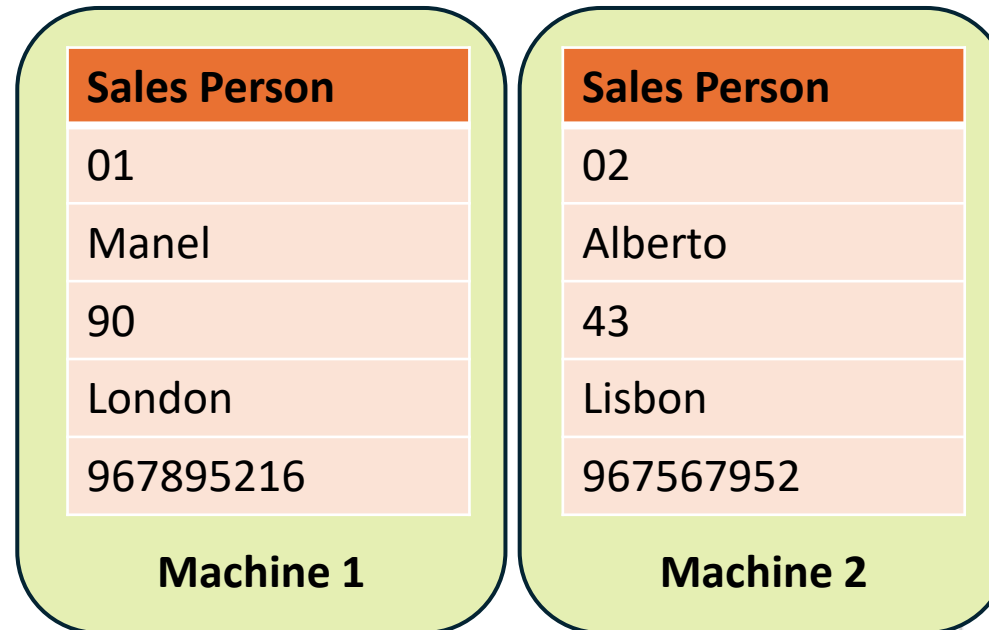
- **Update:**

Update **Students** **Set** **GPA = GPA + 0.4** Where sid = '1111';

Physical Data Independence (Rule 8)

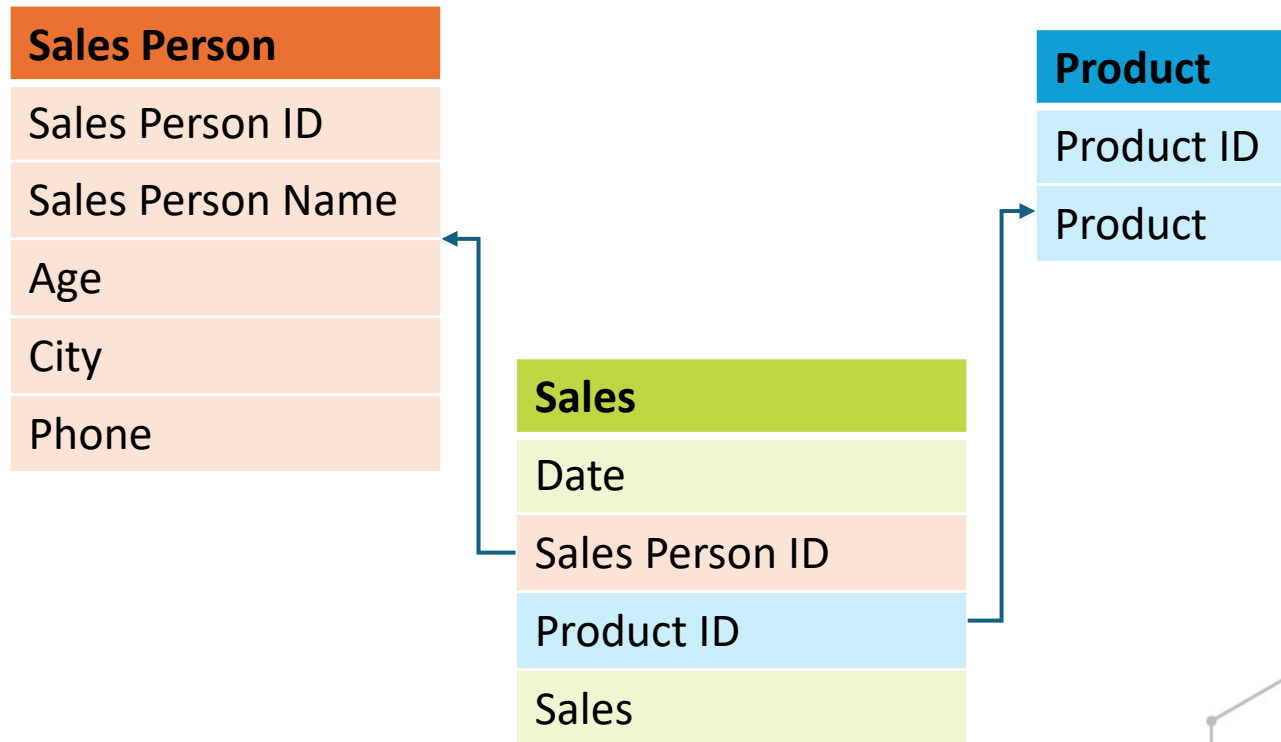
- Any modification in the physical location of a table should not enforce modification at the application level.

Sales Person
Sales Person ID
Sales Person Name
Age
City
Phone



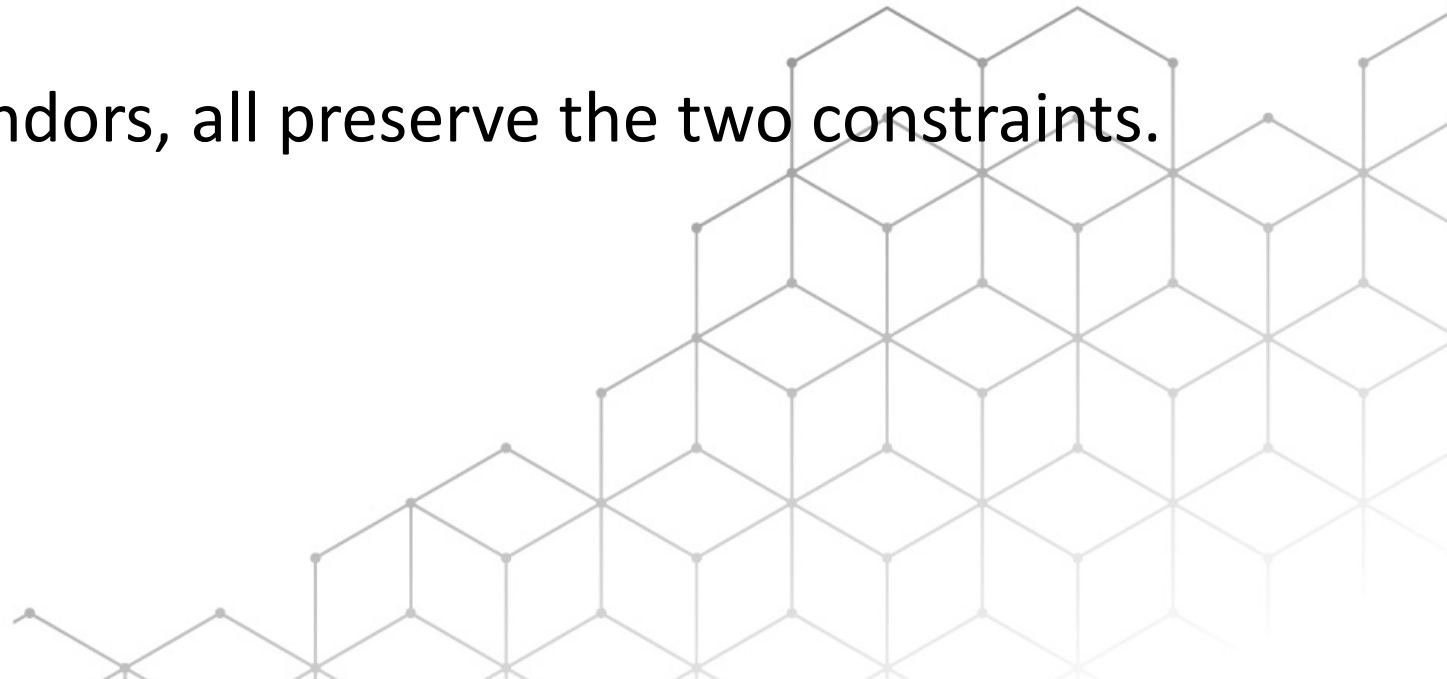
Logical Data Independence (Rule 9)

- Any modification in the schema should not enforce modification at the application level.



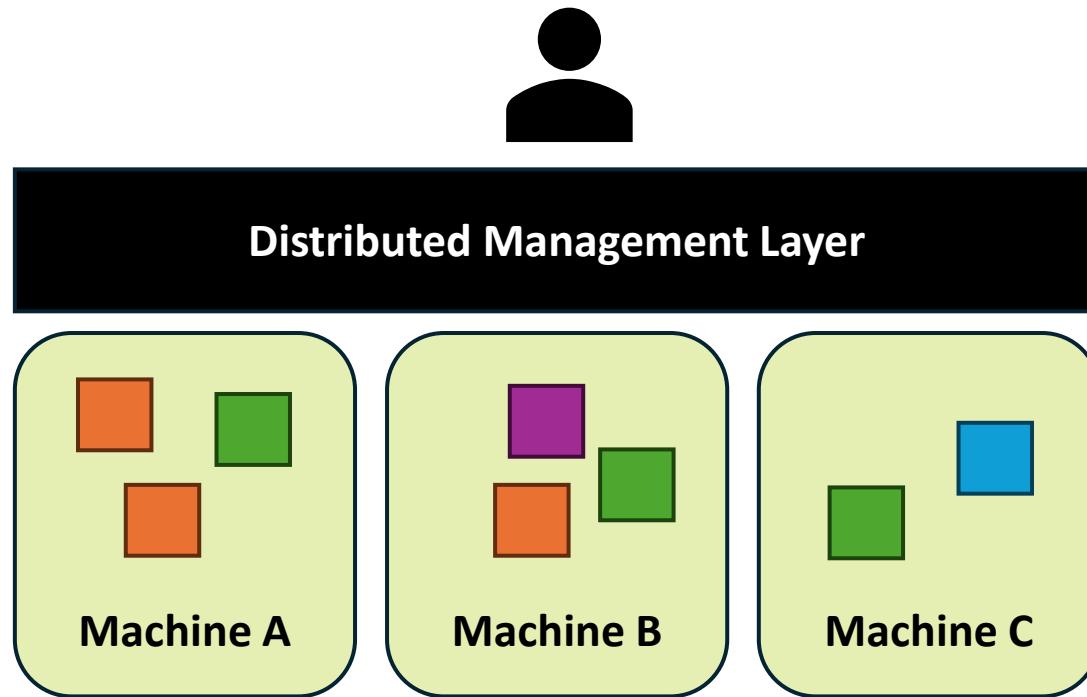
Integrity Independence (Rule 10)

- The sub-language (e.g., SQL) has constraints to maintain integrity.
- Minimum Constraints:
 - A primary key cannot contain a NULL value.
 - A foreign key value must exist as a primary key somewhere (referential integrity).
- Not fully supported by all vendors, all preserve the two constraints.



Distribution Independence (Rule 11)

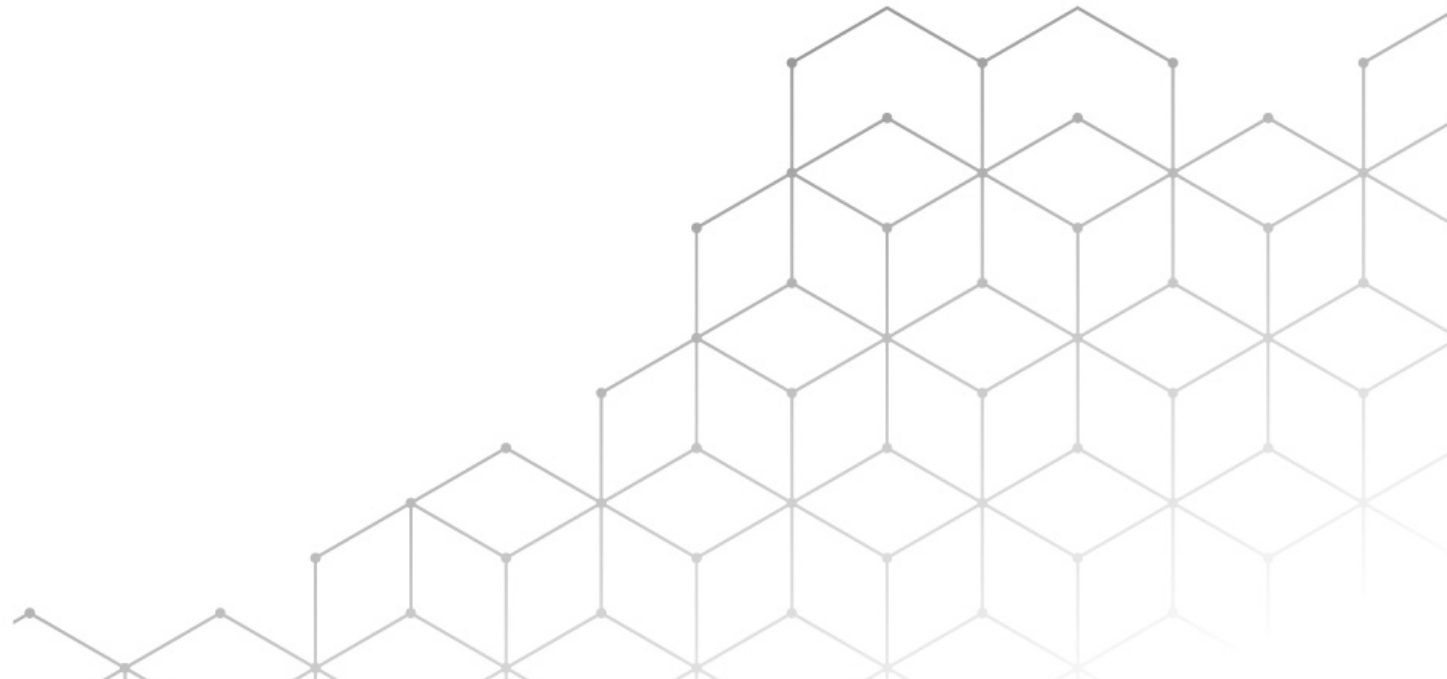
- Distribution of data over various locations should not be visible to end-users.



Non-subversion Rule (Rule 12)

- Low-level access to data should not be able to bypass integrity rules to change data.

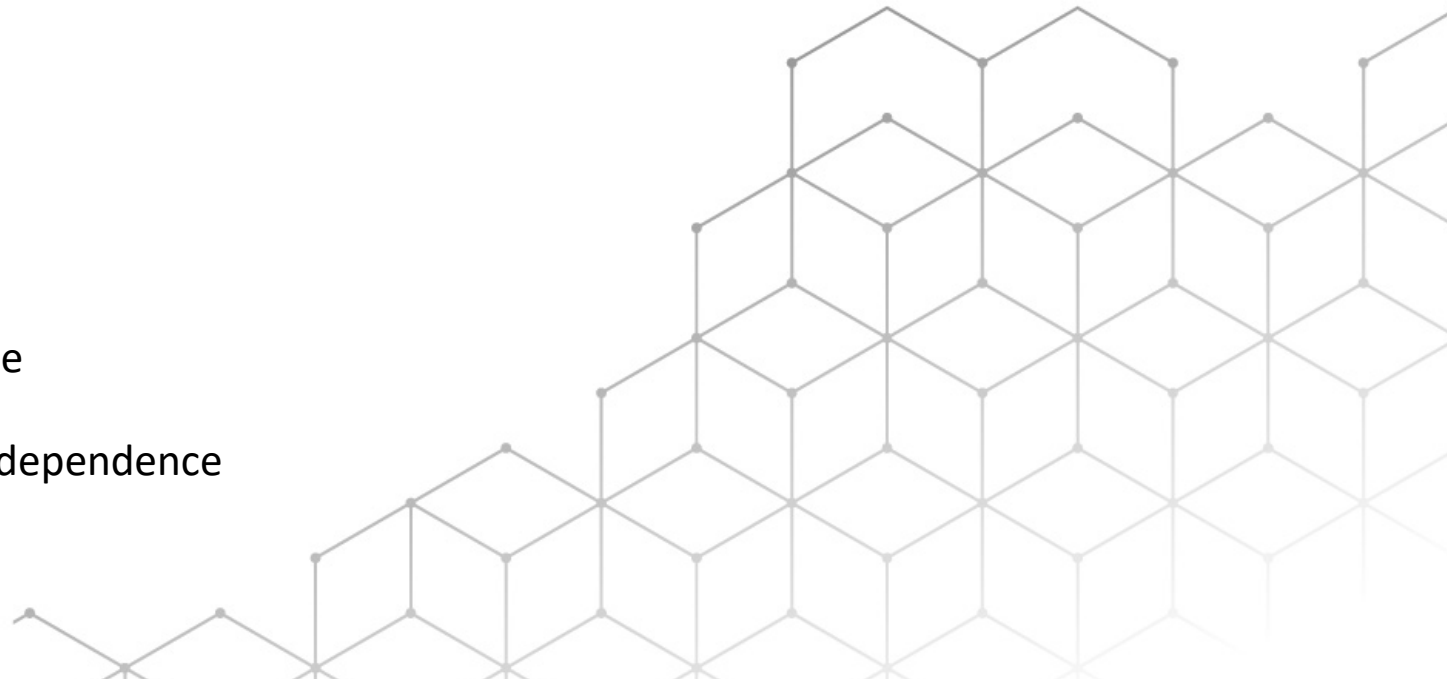
**NO
TRESPASSING**



Agenda for Today

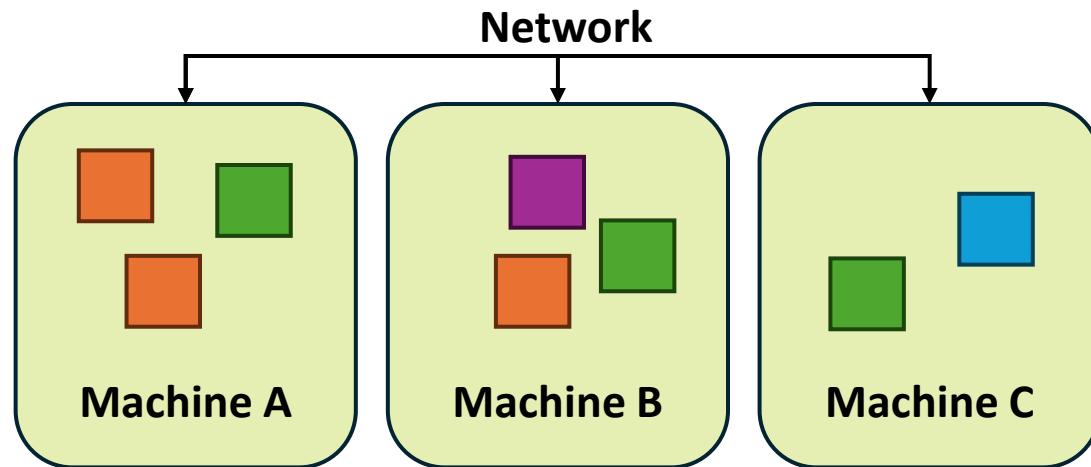


- Introduction
- History of Databases
- Relational Databases
- Distributed Databases: principles of the “perfect” distributed database
 - Date's Twelve Rules for Distributed Database Systems
 - Local Autonomy
 - Central Site Independence
 - Continuous Operation
 - Local Independence
 - Fragmentation Independence
 - Replication Independence
 - Distributed Query Independence
 - Distributed Transaction Independence
 - Distributed Atomocity
 - Hardware, OS, Network, Database Independence
- NoSQL Databases



Distributed Database

- A single logical database that is spread physically across computers in multiple locations that are connected by a data communication link.



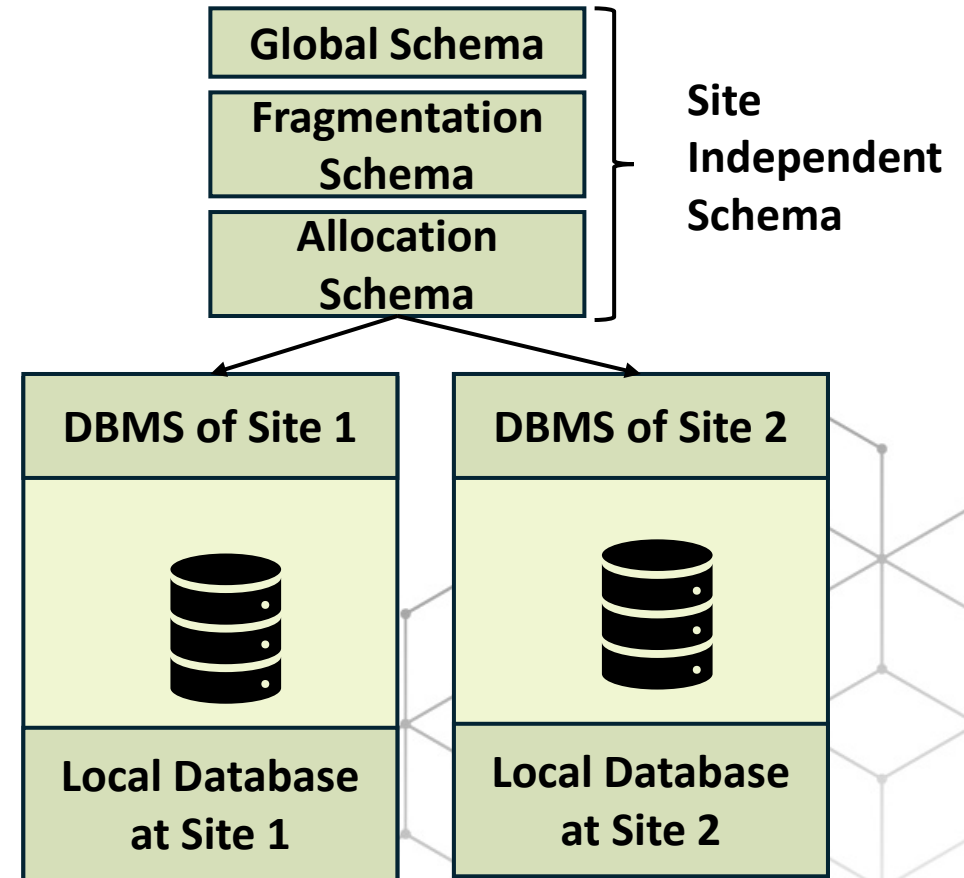
Date's 12 Rules for DDBMS

- C J Date defined a set of twelve objectives to be satisfied by a fully Distributed Database Management System.
- Note: currently no DDBMS conforms to all the twelve rules.

1. Local autonomy
2. No reliance on a central site
3. Continuous operation
4. Location independence (Location Transparency)
5. Fragmentation independence (Fragmentation Transparency)
6. Replication independence (replication transparency)
7. Distributed query processing
8. Distributed transaction management
9. Hardware independence
10. Operating system independence
11. Network independence

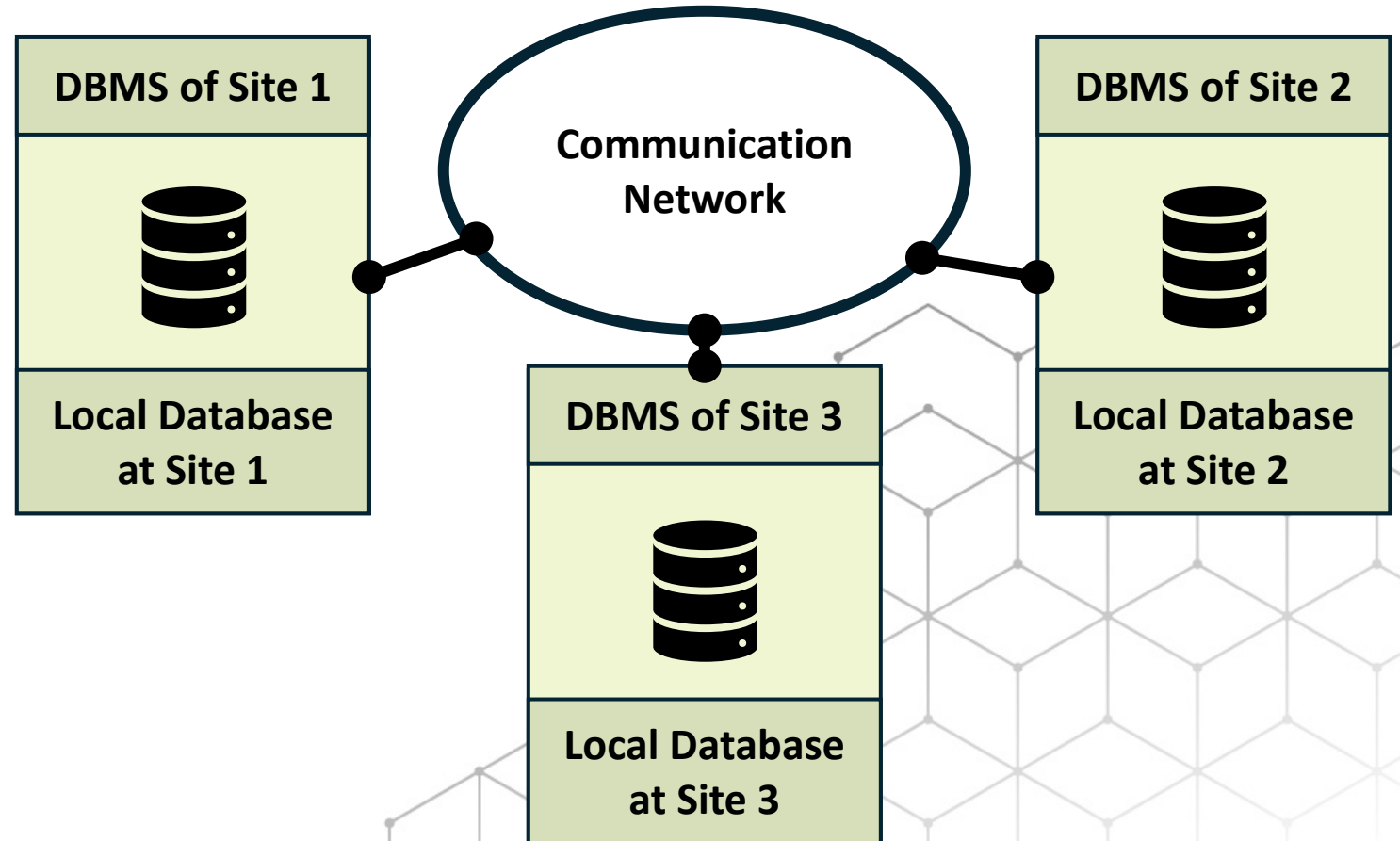
Local Autonomy (Rule 1)

- Each site has its own operations and also act as independent autonomous, centralized DBMS.
- For security, concurrency control, backup and recovery there is responsibility of each site (node)



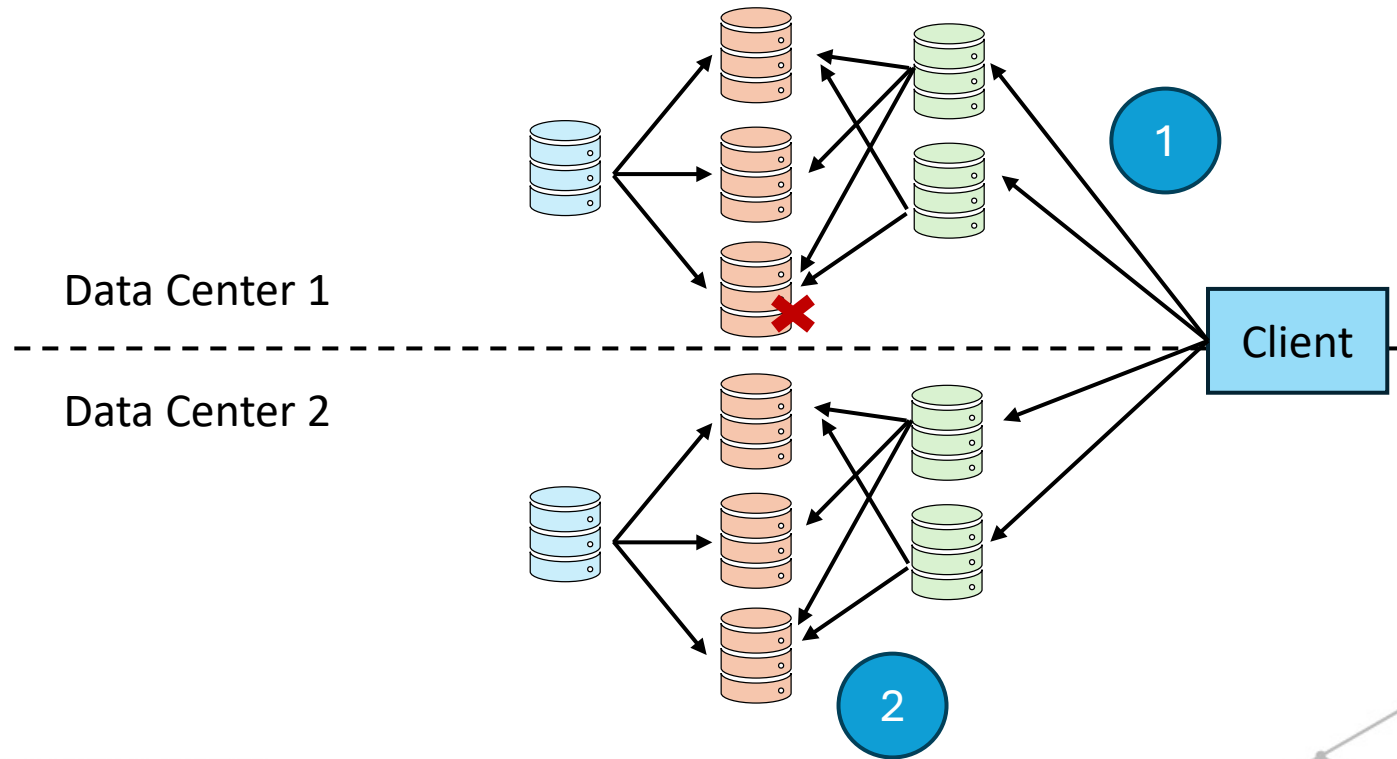
Central Site Independence (Rule 2)

- All sites are equal and independent on central site for any service.
- Central site is bottleneck and source of failure



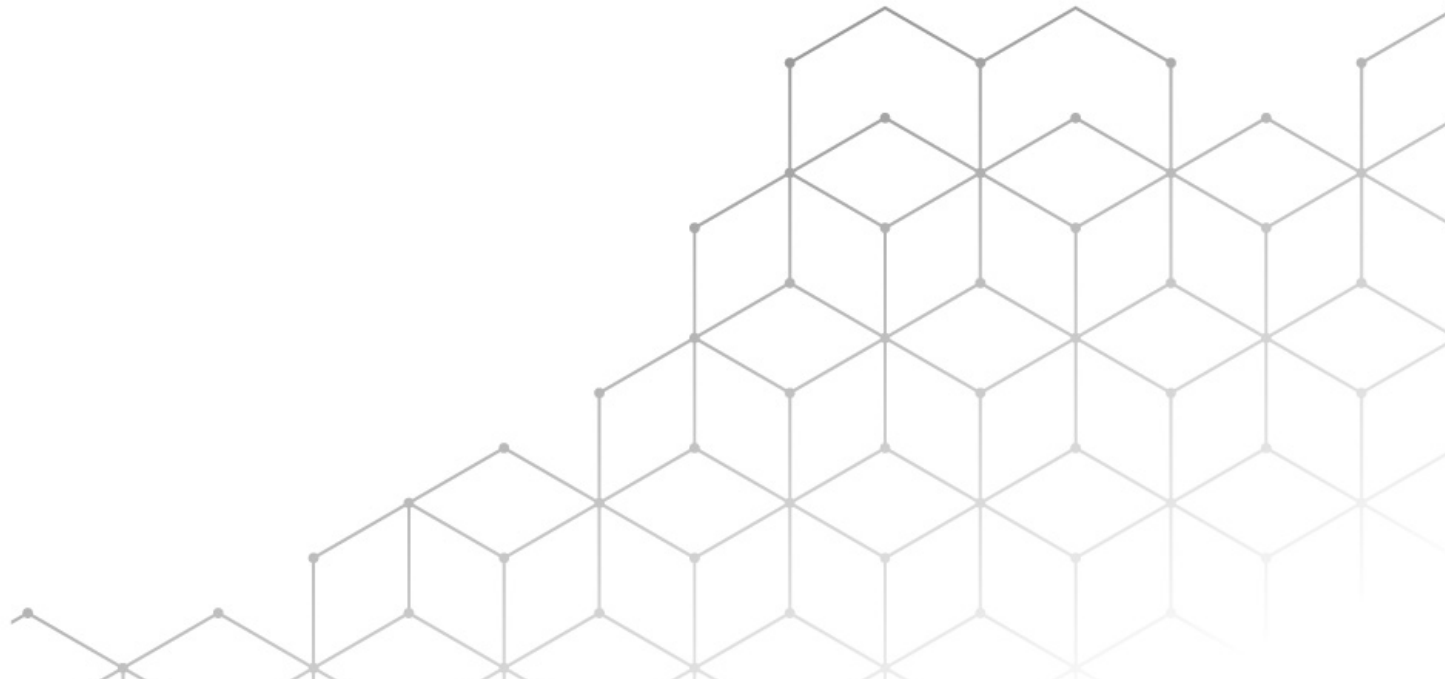
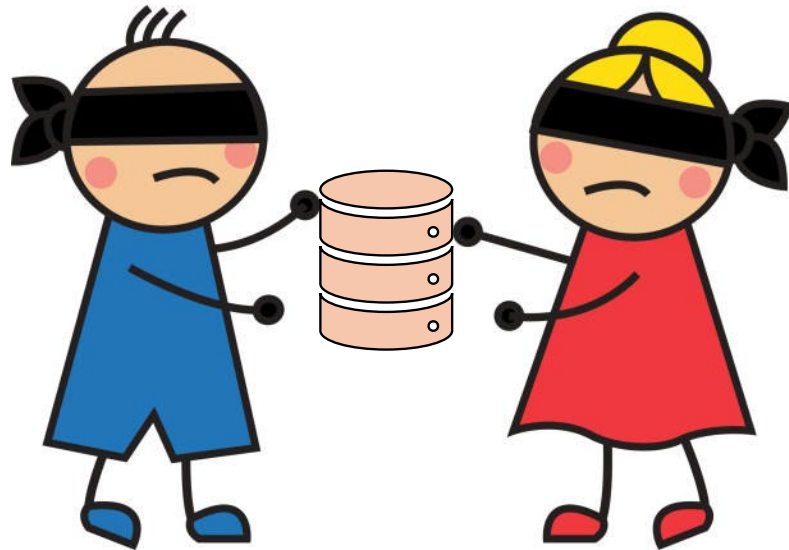
Continuous Operation (Rule 3)

- There is no affect of site failure to system.



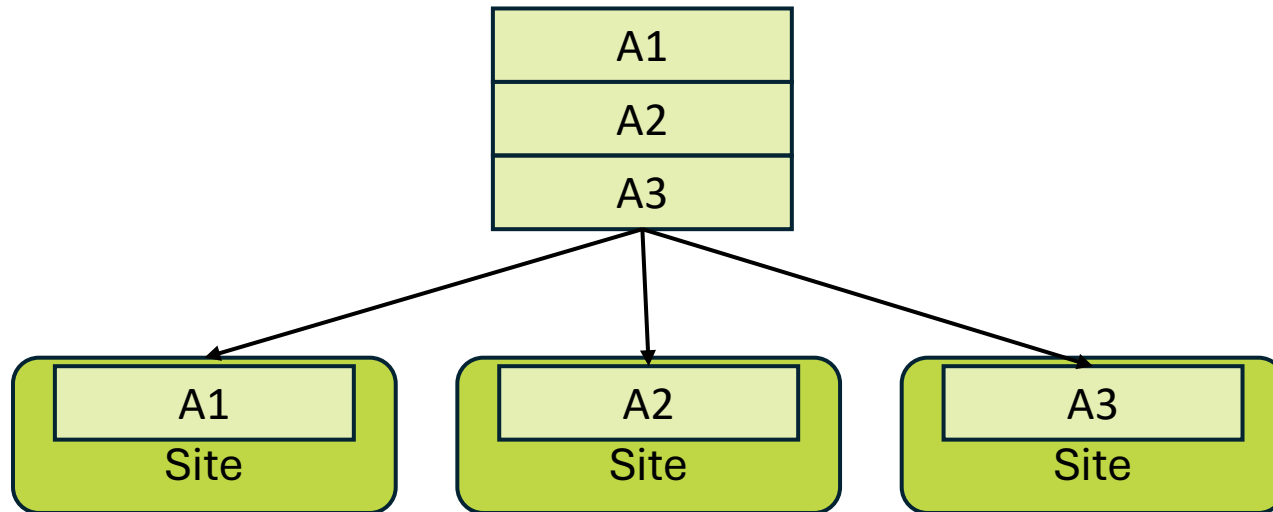
Local Independence (Rule 4)

- To retrieve any data in system, there is need to know about the storage of data i.e where the data is stored in system.



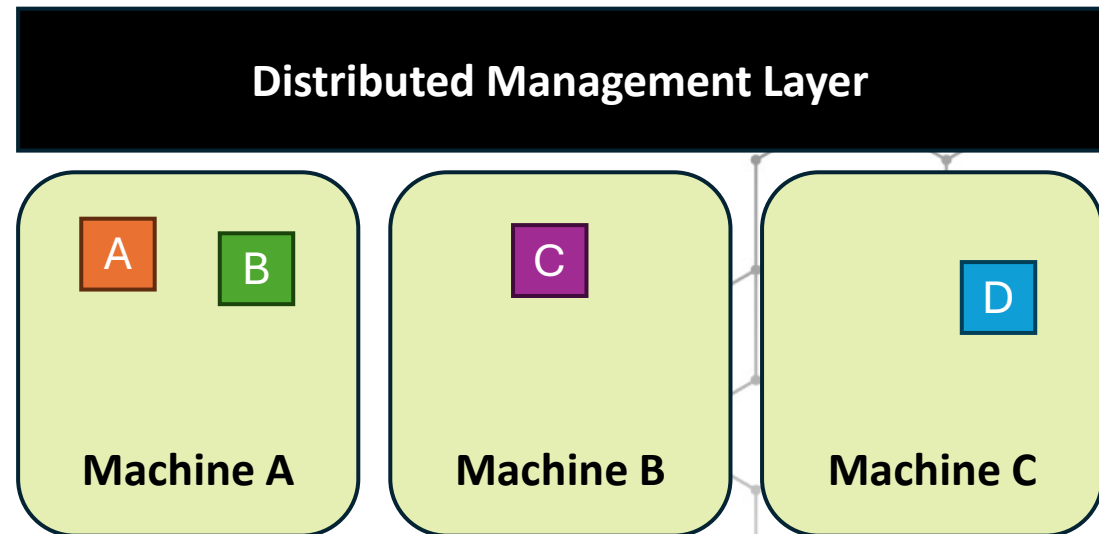
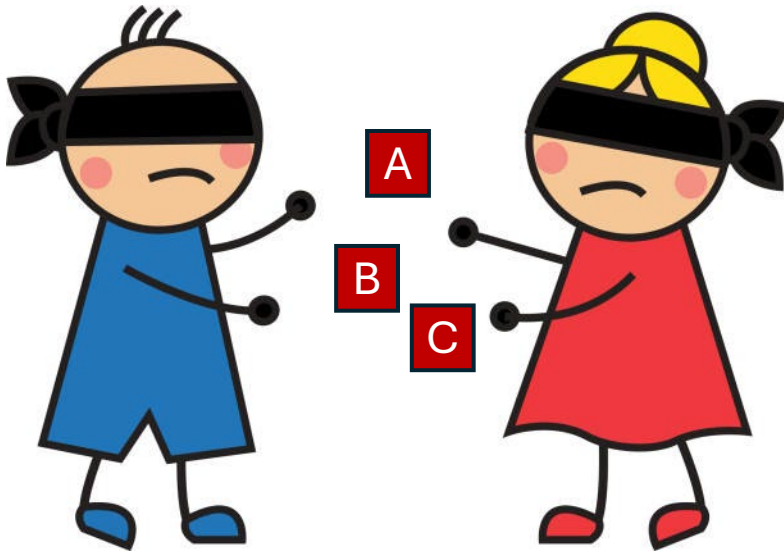
Distributed Storage (Rule 4)

- Data may be partitioned or fragmented across several sites
- Typically at sites where they are most often accessed
- The data can be replicated as well – when the relation is in high demand



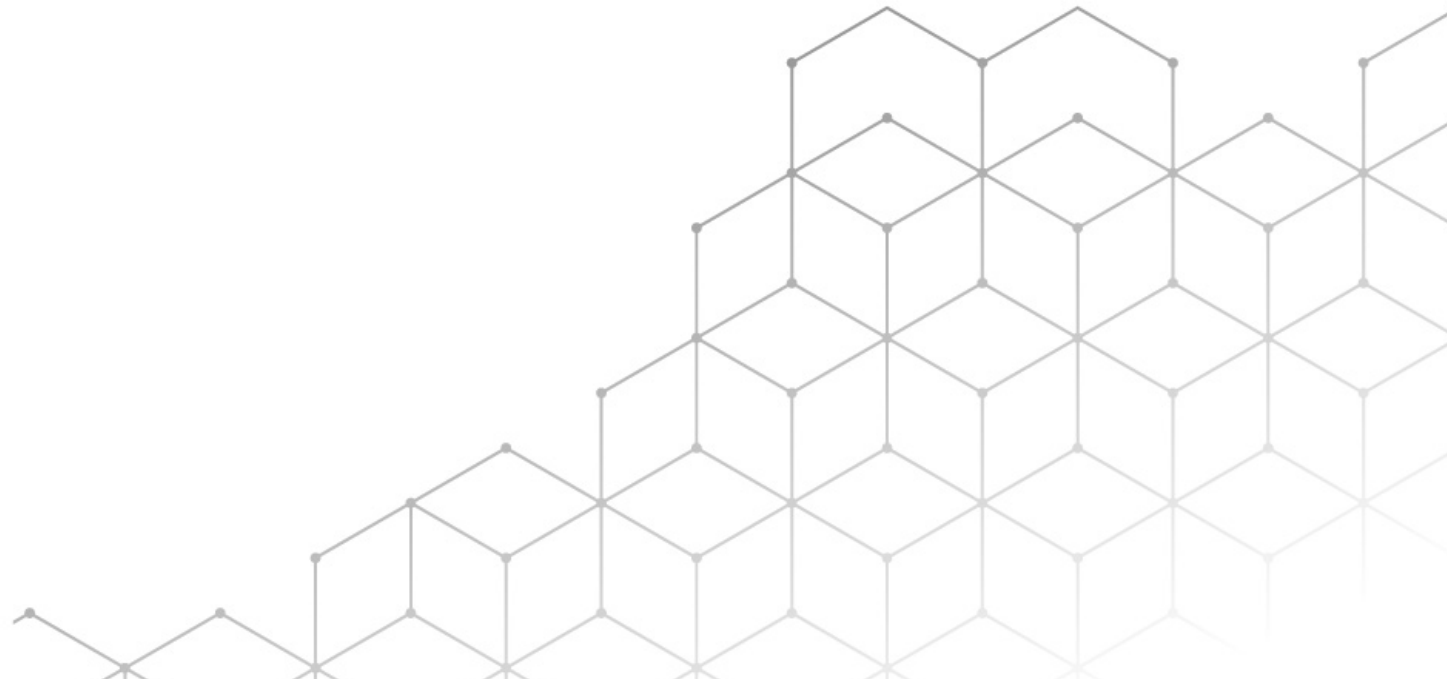
Fragmentation Independence (Rule 5)

- To retrieve any fragments of database, there is no need to know about name of database fragments.



Consistency and Aggregates

- It is a unit for data manipulation.
- Aggregates will be updated with atomic operations.
- Aggregate makes a natural unit for replication and sharding.



Sharding Example

ID	Client Nr	Phone

ID	Client Nr	Phone

Server A

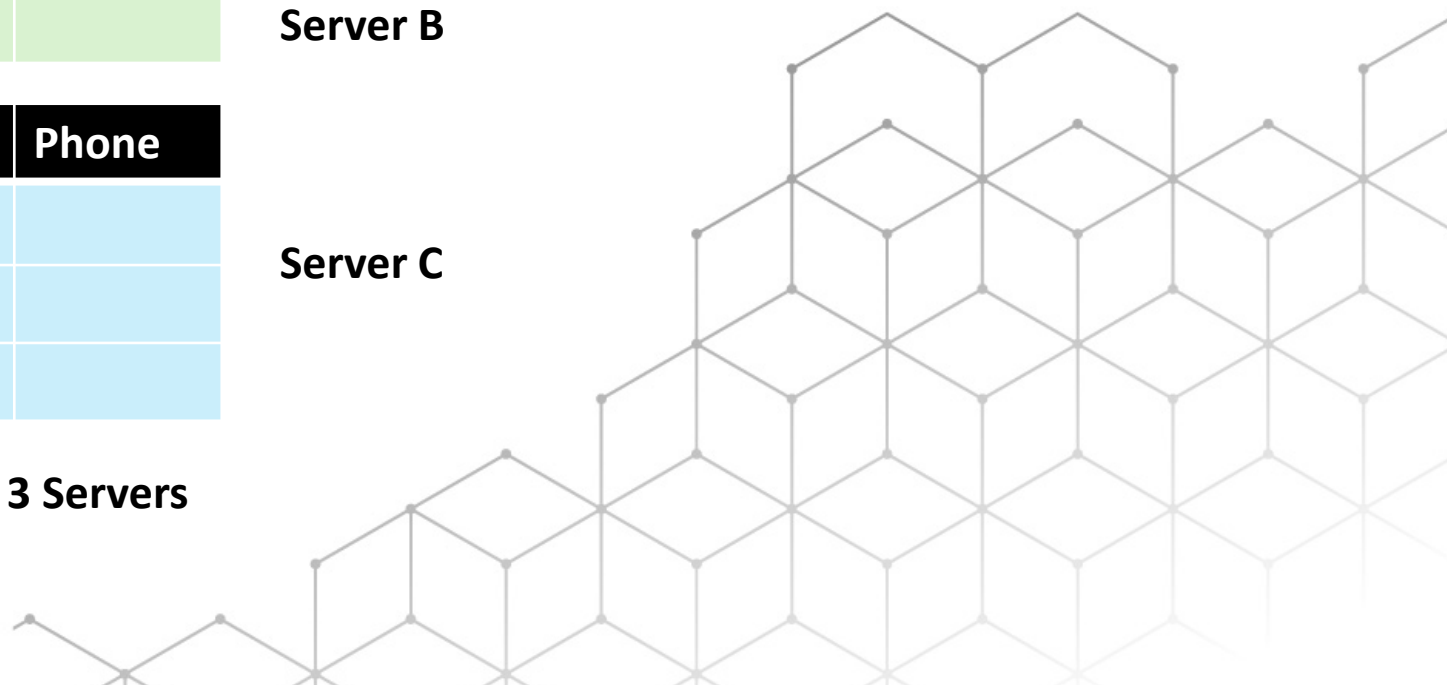
ID	Client Nr	Phone

Server B

ID	Client Nr	Phone

Server C

Shared Table in 3 Servers



Example of Different Aggregate Models

```
//Customer
{
  "id": 99,
  "customerId": 1,
  "orderItems": [
    {
      "productId": 27,
      "price": 34,
      "productName": "Scala in Action"
    }
  ],
  "shippingAddress": [
    { "city": "Bari" }
  ],
  "orderPayment": [
    {
      "ccinfo": "100-432423-545-134",
      "txnId": "afdsfsd",
      "billingAddress": [{ "city": "Bari" }]
    }
  ]
}
```

```
//Orders
{
  "id": 99,
  "customerId": 1,
  "orderItems": [
    {
      "productId": 27,
      "price": 34,
      "productName": "Scala in Action"
    }
  ],
  "shippingAddress": [
    { "city": "Bari" }
  ],
  "orderPayment": [
    {
      "ccinfo": "100-432423-545-134",
      "txnId": "afdsfsd",
      "billingAddress": [{ "city": "Bari" }]
    }
  ]
}
```

//Customer with Orders

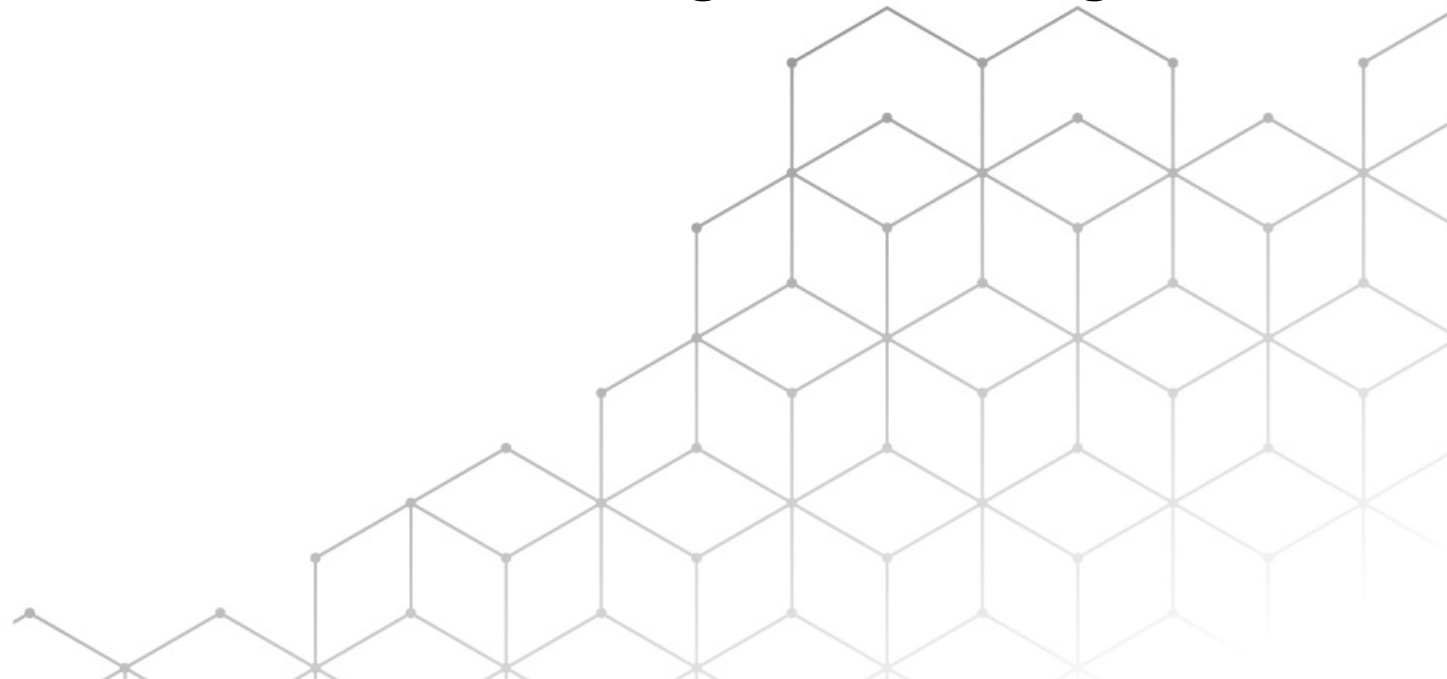
```
{
  "customer": {
    "id": 1,
    "name": "Martin",
    "billingAddress": [{ "city": "Chicago" }],
    "orders": [
      {
        "id": 99,
        "customerId": 1,
        "orderItems": [
          {
            "productId": 27,
            "price": 32.45,
            "productName": "NoSQL Distilled"
          }
        ],
        "shippingAddress": [{ "city": "Chicago" }],
        "orderPayment": [
          {
            "ccinfo": "1000-1000-1000-1000",
            "txnId": "abelif879rft",
            "billingAddress": { "city": "Chicago" }
          }
        ]
      }
    ]
  }
}
```

How to Define the Aggregate Model?

- There's no universal answer for how to draw your aggregate boundaries.
- It depends entirely on how you tend to manipulate your data.
- If you tend to access a customer together with all of that customer's orders at once, then you would prefer a single aggregate.
- However, if you tend to focus on accessing a single order at a time, then you should prefer having separate aggregates for each order.
- Naturally, this is very context-specific; some applications will prefer one or the other, even within a single system, which is exactly why many people prefer **aggregate ignorance**.

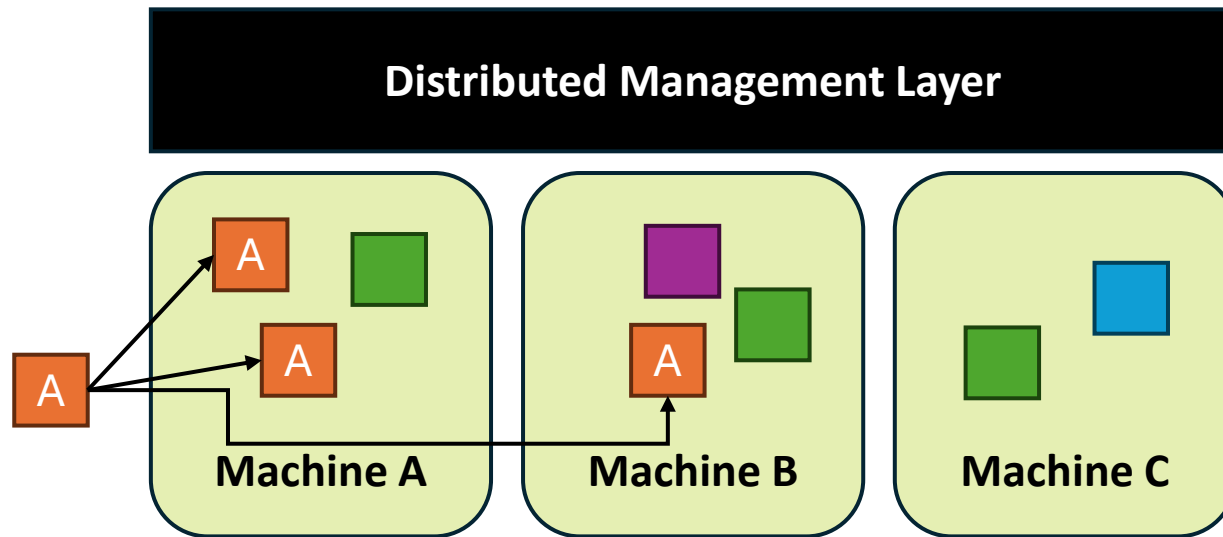
Summary of Aggregates

- An aggregate is a collection of data that we interact with as a unit.
- Key-value, document, and column-family databases can all be seen as forms of aggregate-oriented databases.
- Aggregates make it easier for the database to manage data storage over clusters.



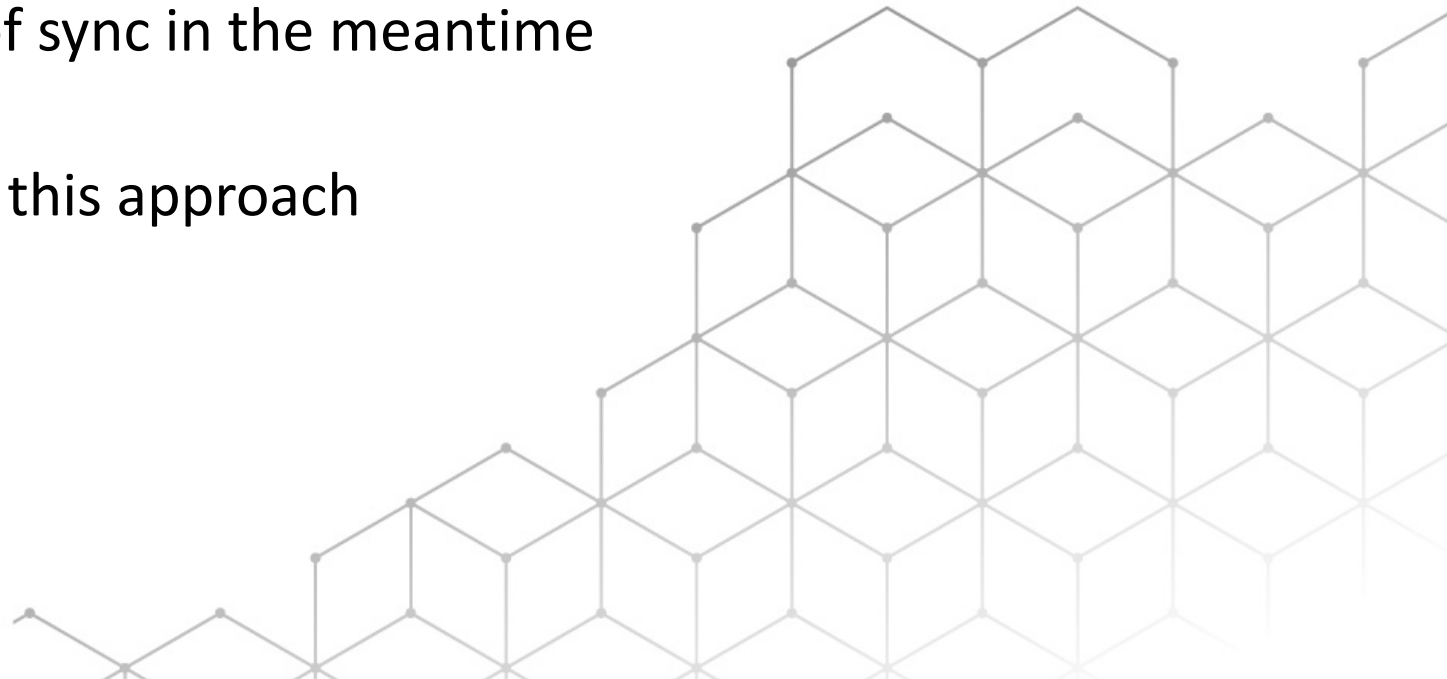
Replication Independence (Rule 6)

- Data can be replicated and stored in different sites.
- The DDBMS manages all fragments transparently the user.



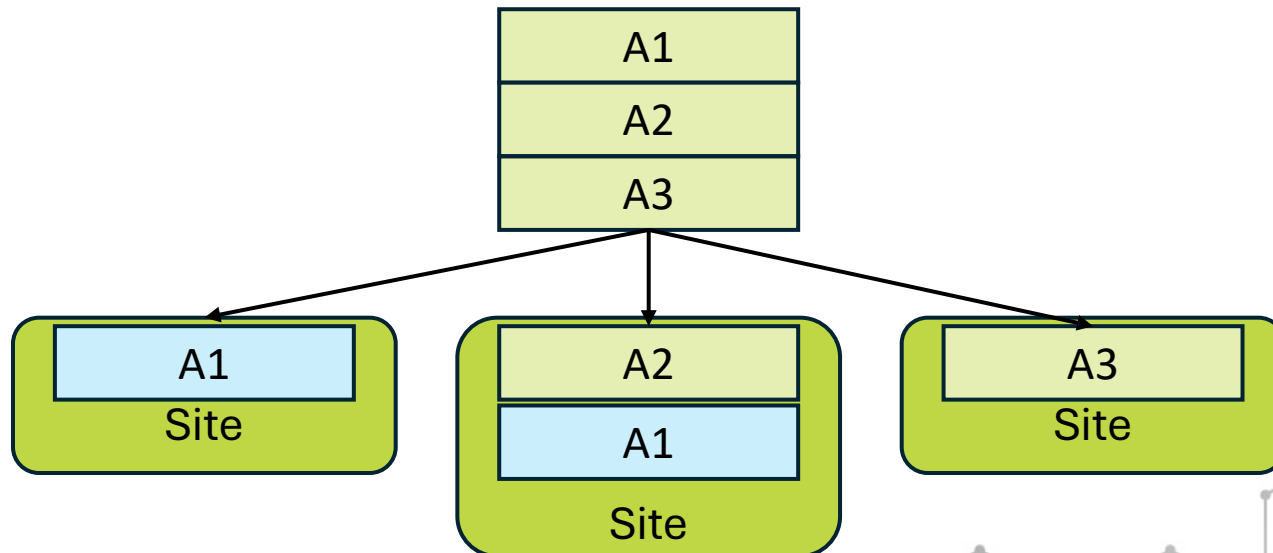
Synchronous v. Asynchronous Replication

- Synchronous Replication:
 - all copies must be updated before the modifying transaction commits
- Asynchronous Replication:
 - copies of a modified relation are only periodically updated;
 - different copies may get out of sync in the meantime
 - more efficient
 - many current products follow this approach



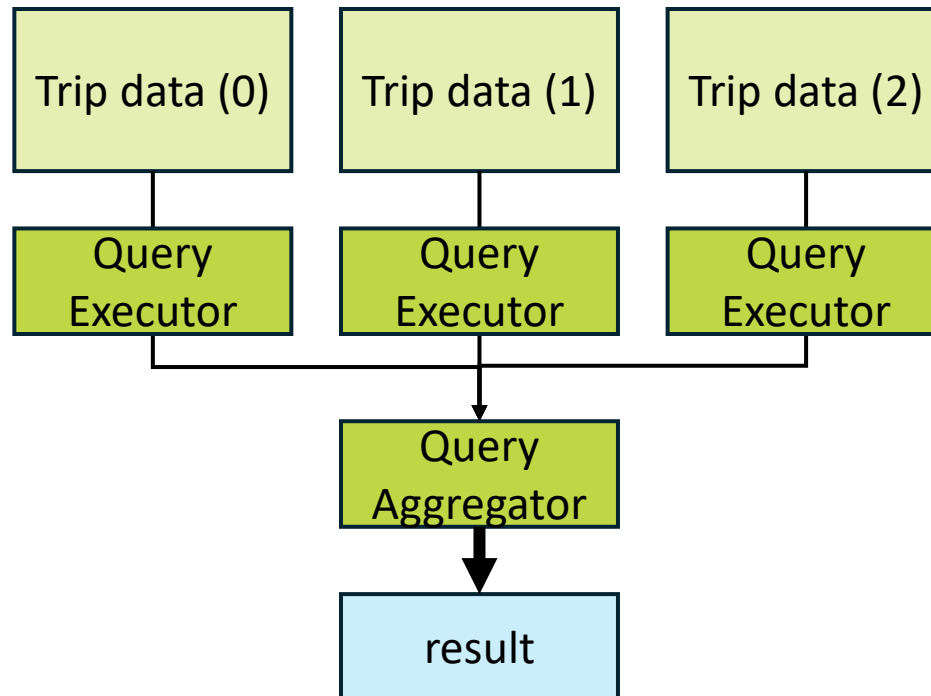
Replication (Rule 6)

- When we store several copies of a relation or relation fragments – can be replicated at one or more sites
- Advantages
 - Gives increased availability e.g. when a site or communication link goes down



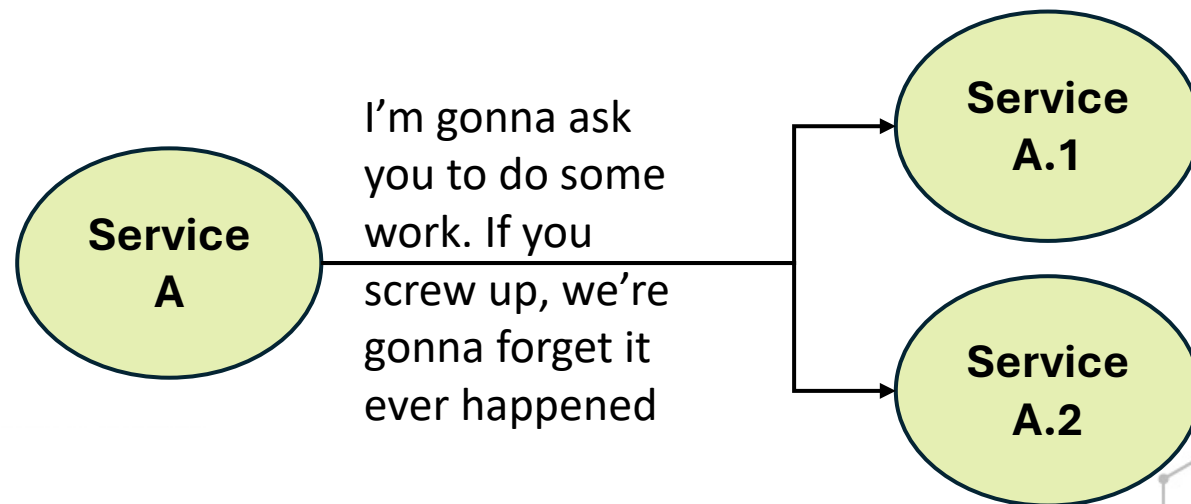
Distributed Query Independence (Rule 7)

- Queries spanning multiple sites should be optimized in a cost-based manner



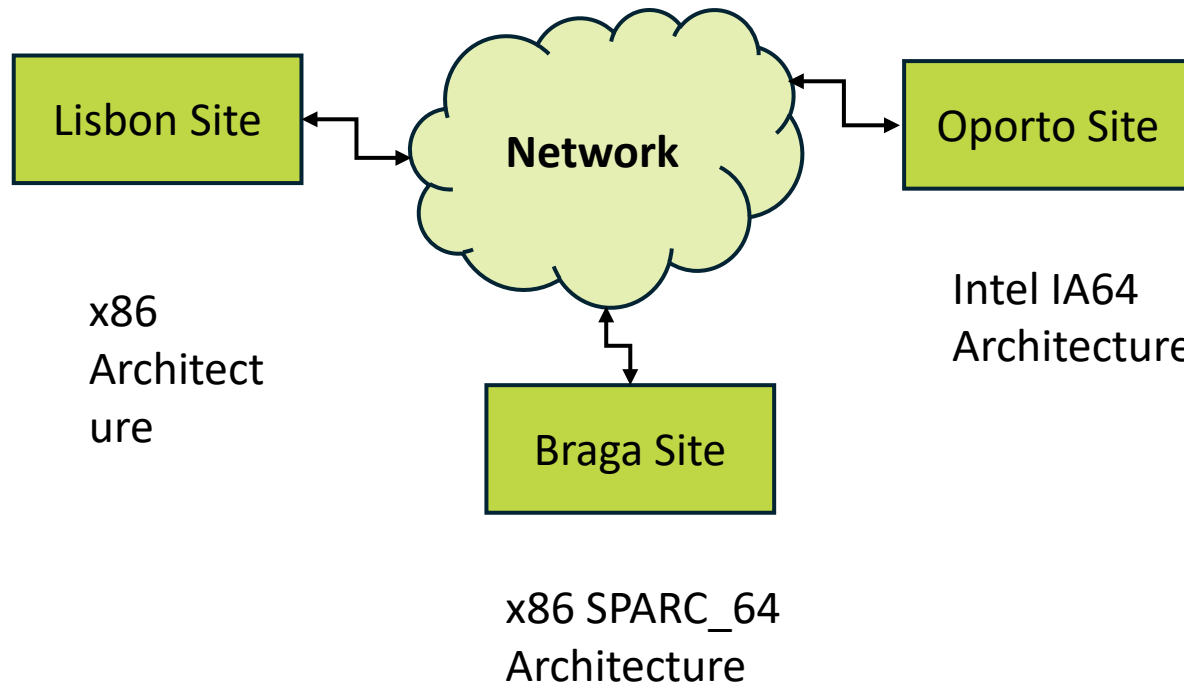
Distributed Transaction Rule (Rule 8)

- Users should be able to write transactions accessing multiple sites just like local transactions
- 2. The effects of a transaction across sites should be atomic
 - – all changes persist if transaction **commits**
 - – none persist if transaction **aborts**



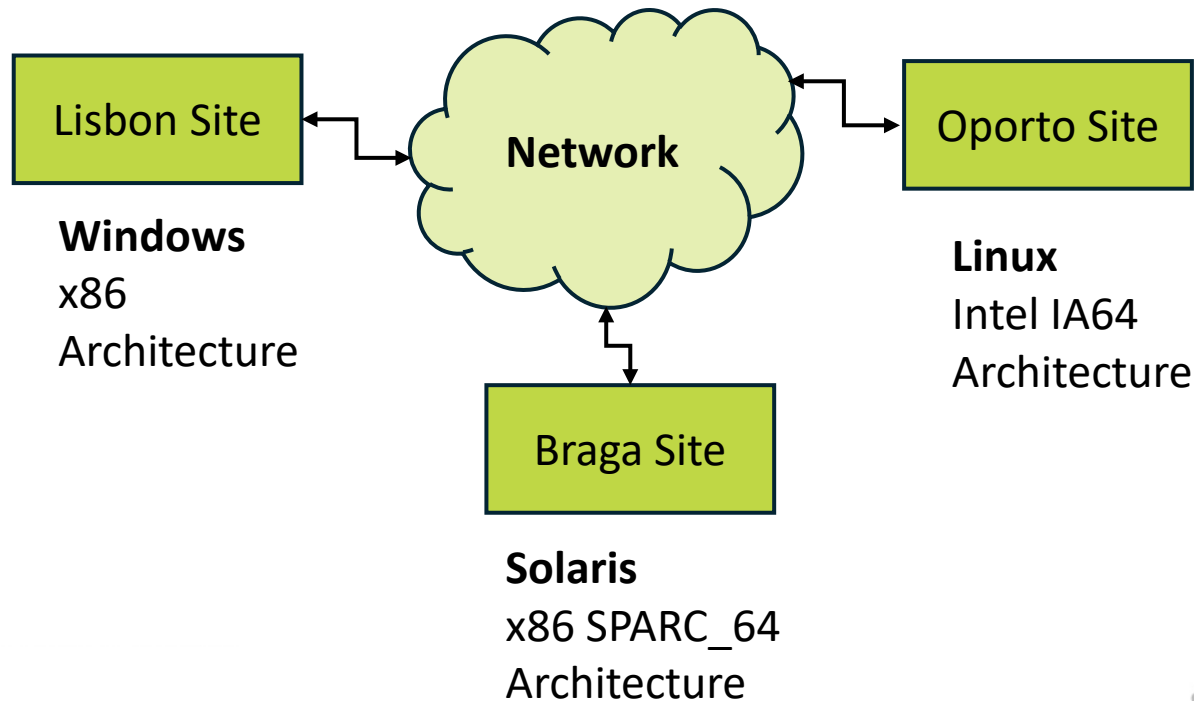
Hardware Independence (Rule 9)

- It should be possible for DDBMS to run on different hardware platforms.



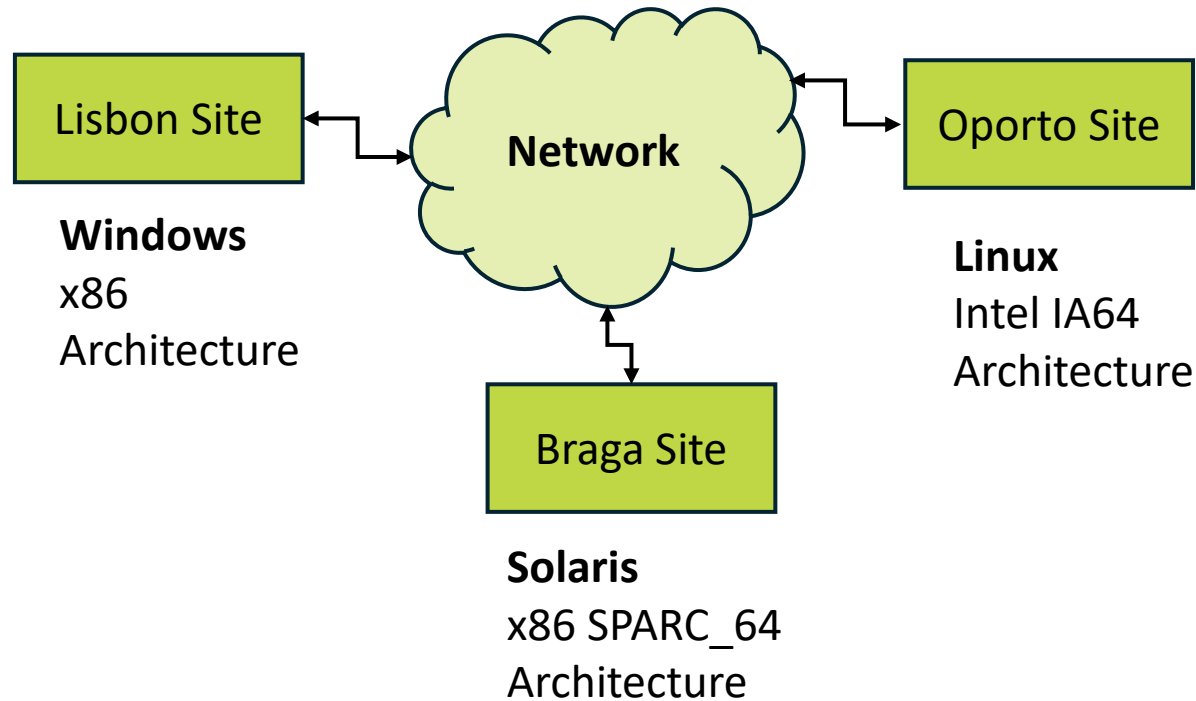
OS Independence (Rule 10)

- It should be possible for DDBMS to run on different OS platforms.



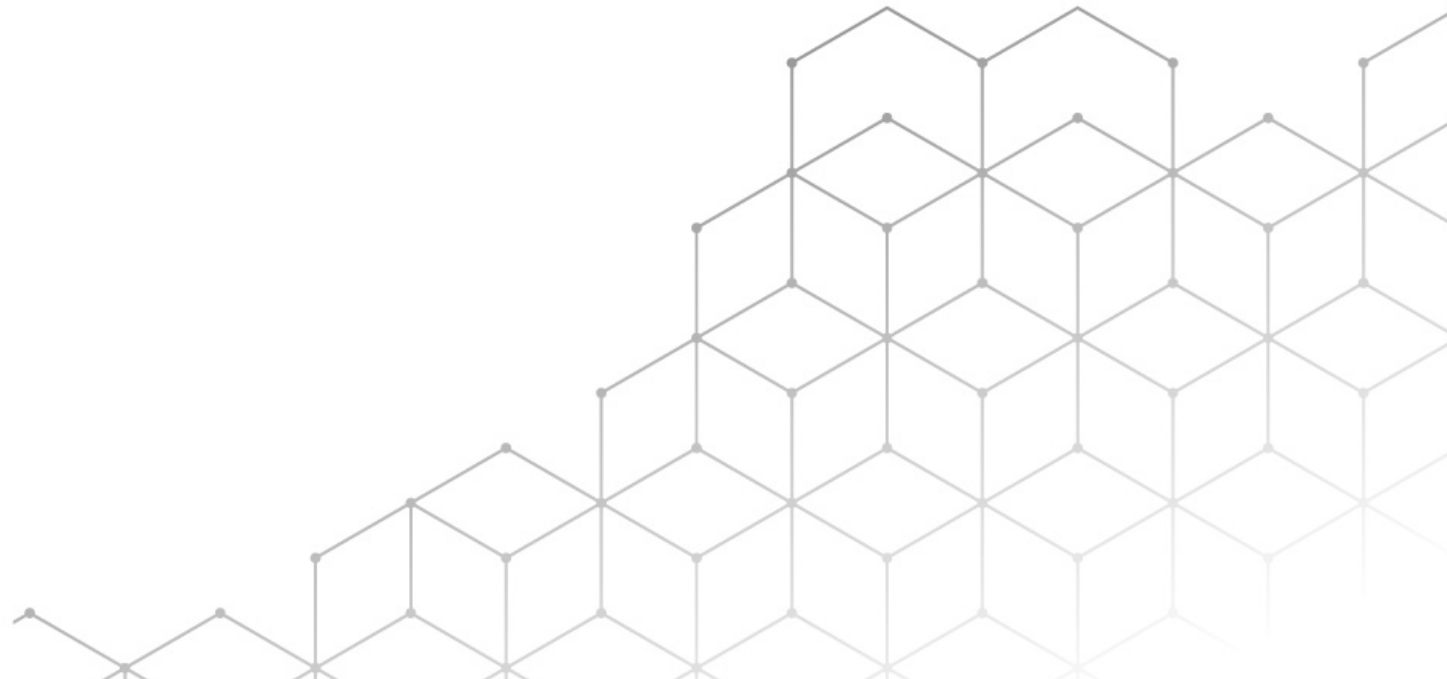
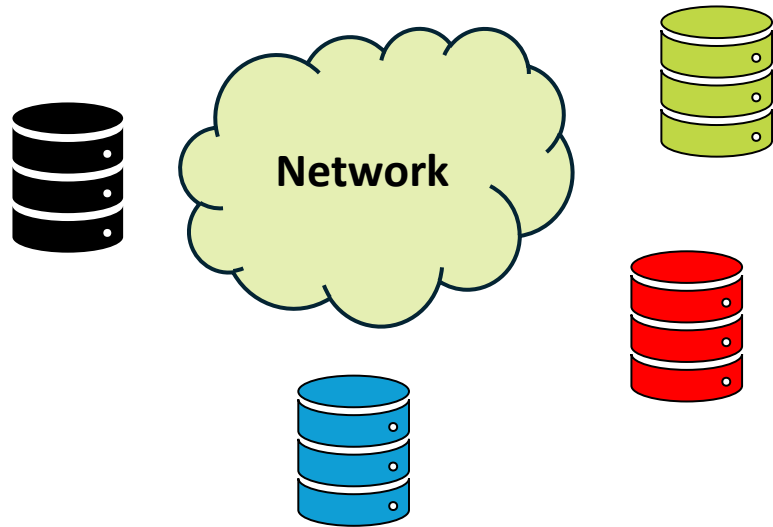
Network Independence (Rule 11)

- The DDBMS system is able to run on any network platform.



Database Independence (Rule 12)

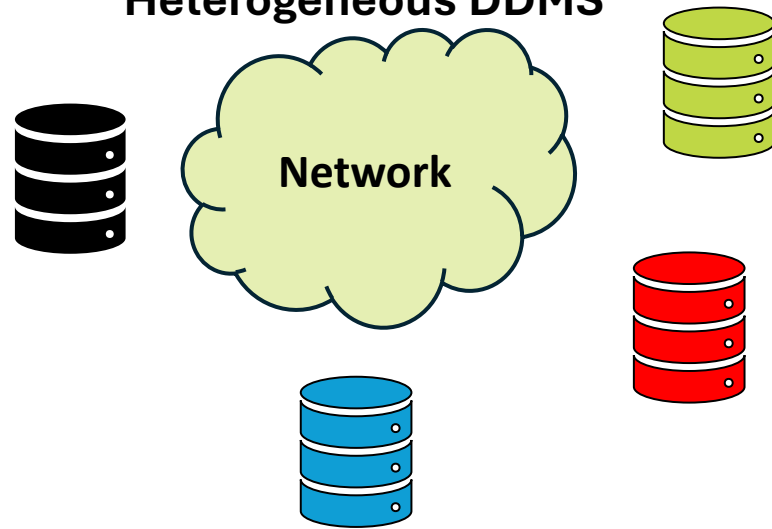
- The system must support any database product.



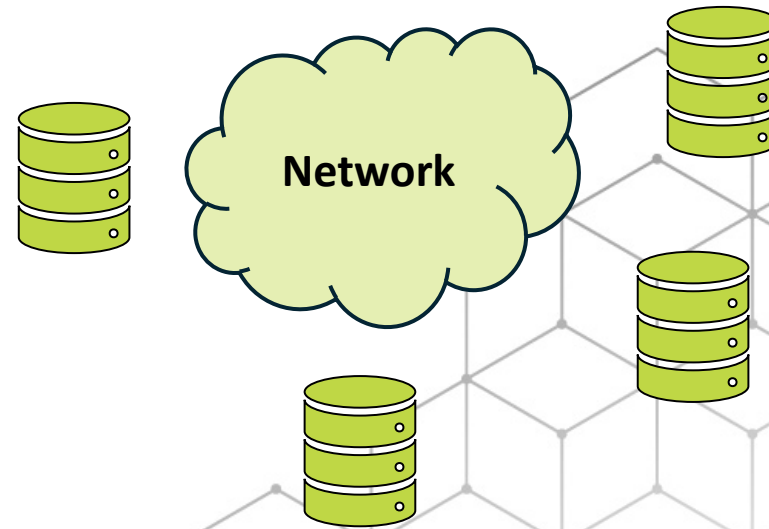
Data Independence (12) in Practice

- **Homogeneous:** – Every site runs same type of DBMS
- **Heterogeneous:** – Different sites run different DBMSs

Heterogeneous DDMS

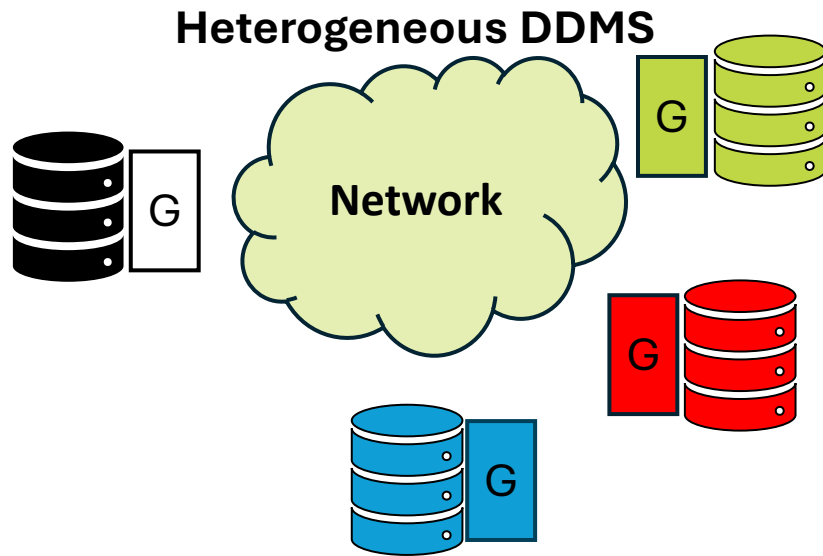


Homogeneous DDMS



Heterogeneous Databases

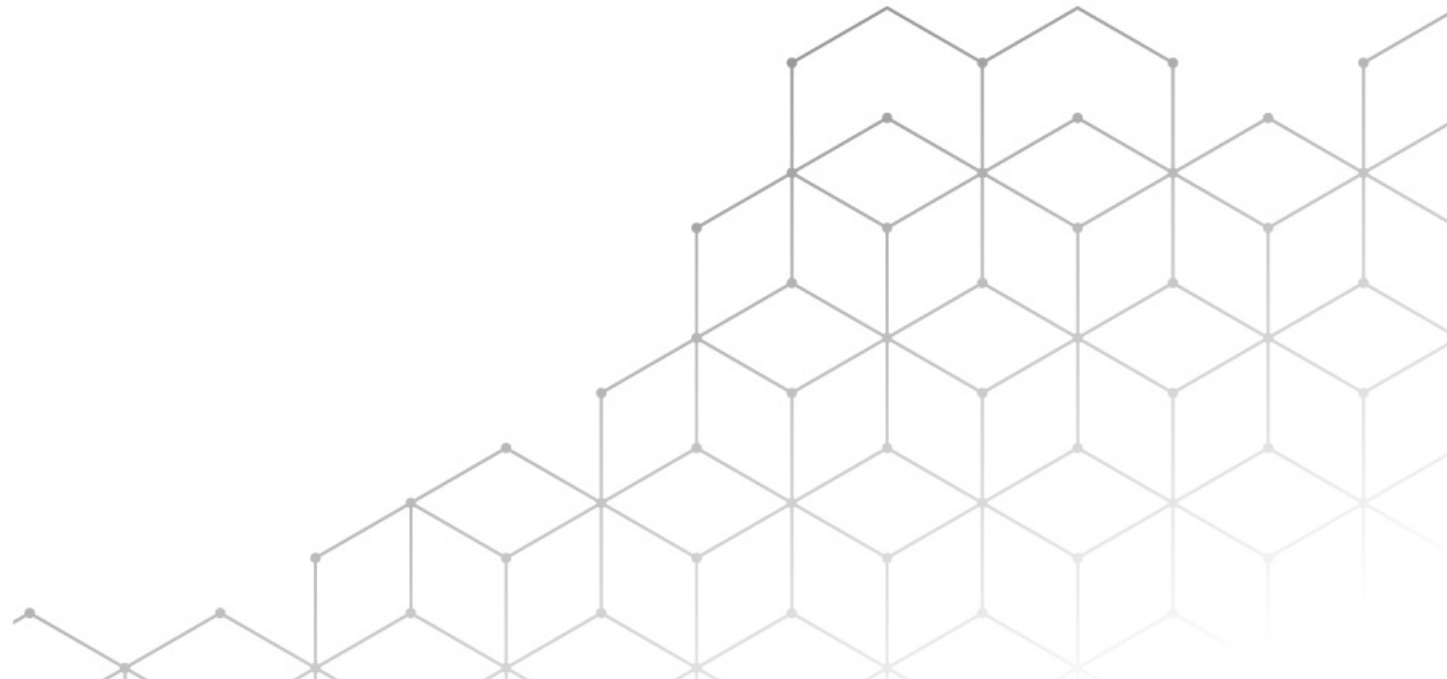
- Database servers are accessed through standard **Gateway** protocols – masks the differences of DBMSs (capability, data format etc.)
- However, can be expensive and may not be hide all differences



Agenda for Today



- Introduction
- History of Databases
- Relational Databases
- Distributed Databases
- NoSQL Databases
 - Motivation for NoSQL Databases
 - Scalability (Go Big or Go Many?)
 - Flexibility
 - Performance
 - Consistency
 - RDBMS vs NoSQL
 - CAP Theorem
 - NoSQL Data Models
 - Column Family
 - Key Value
 - Graph
 - Document
 - Open Challenges

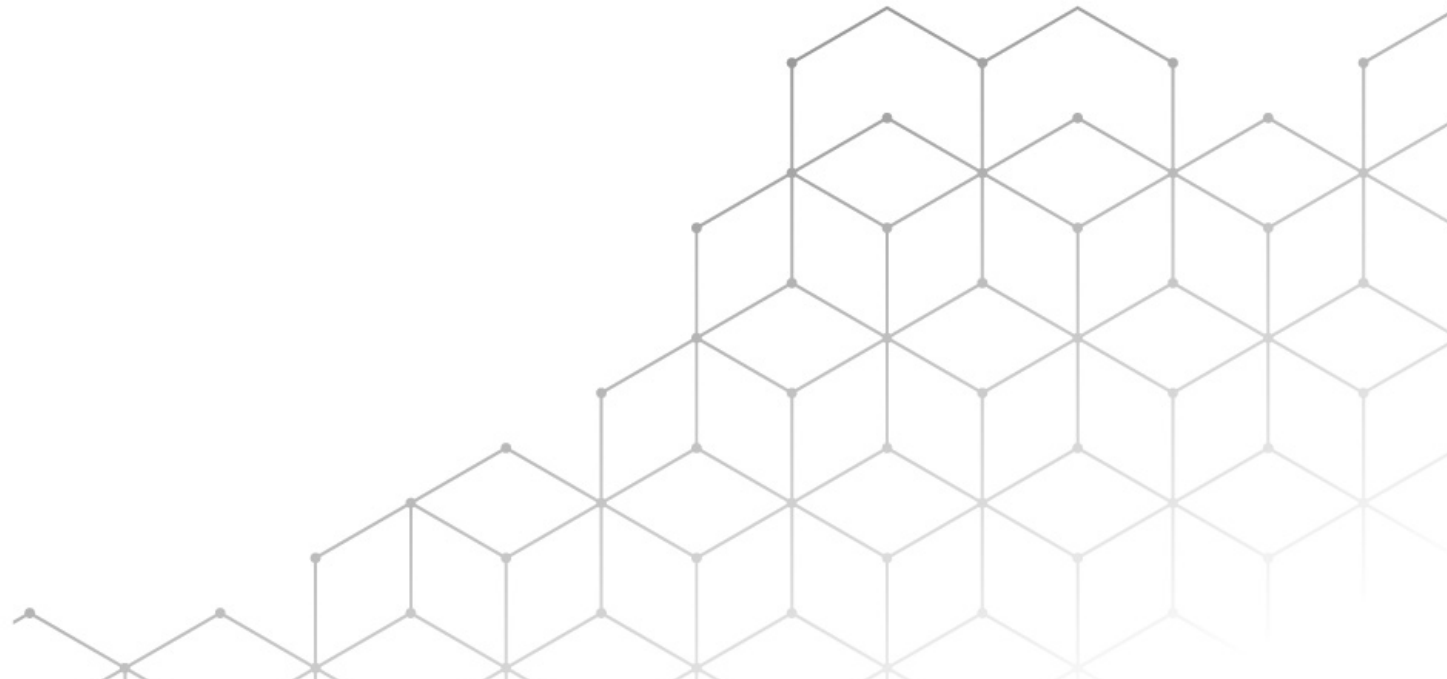


Motivation for NoSQL Databases

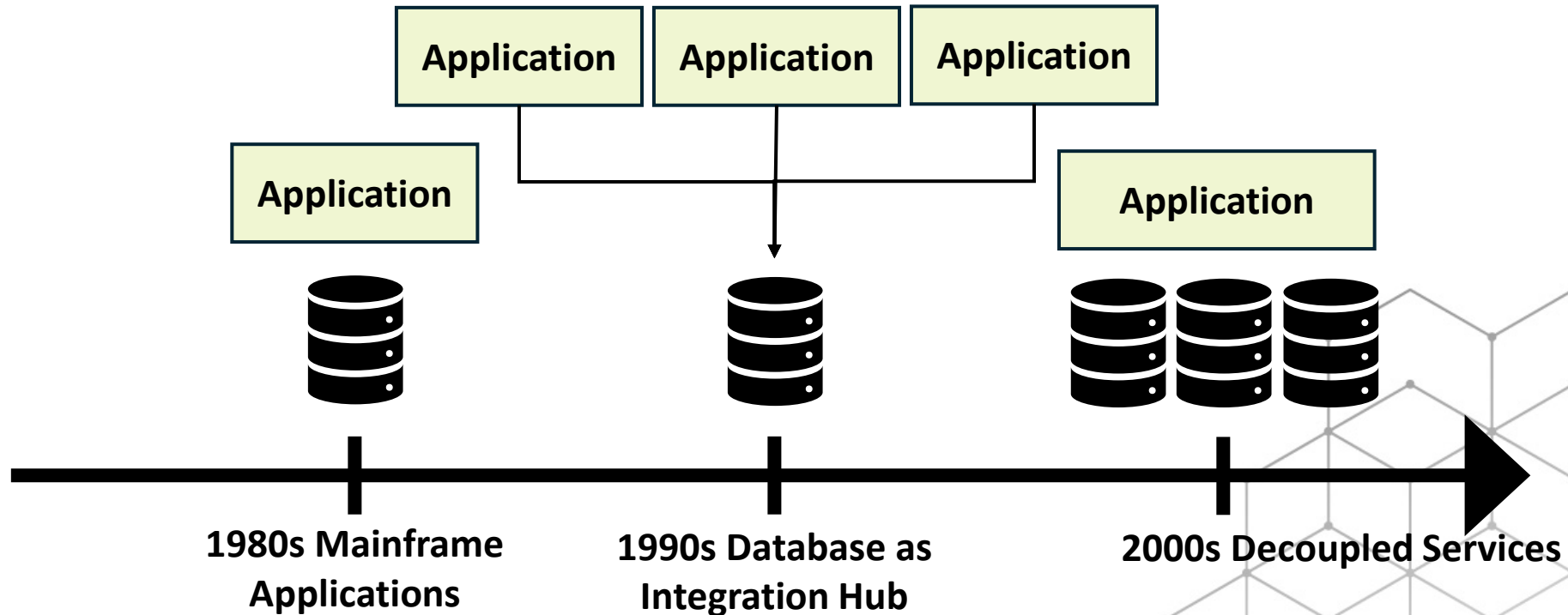
Big data needs storing and querying huge amounts of semi-structured and unstructured data.

So we need databases which can provide:

- Scalability (Sharding)
- Performance
- Flexibility
- Availability



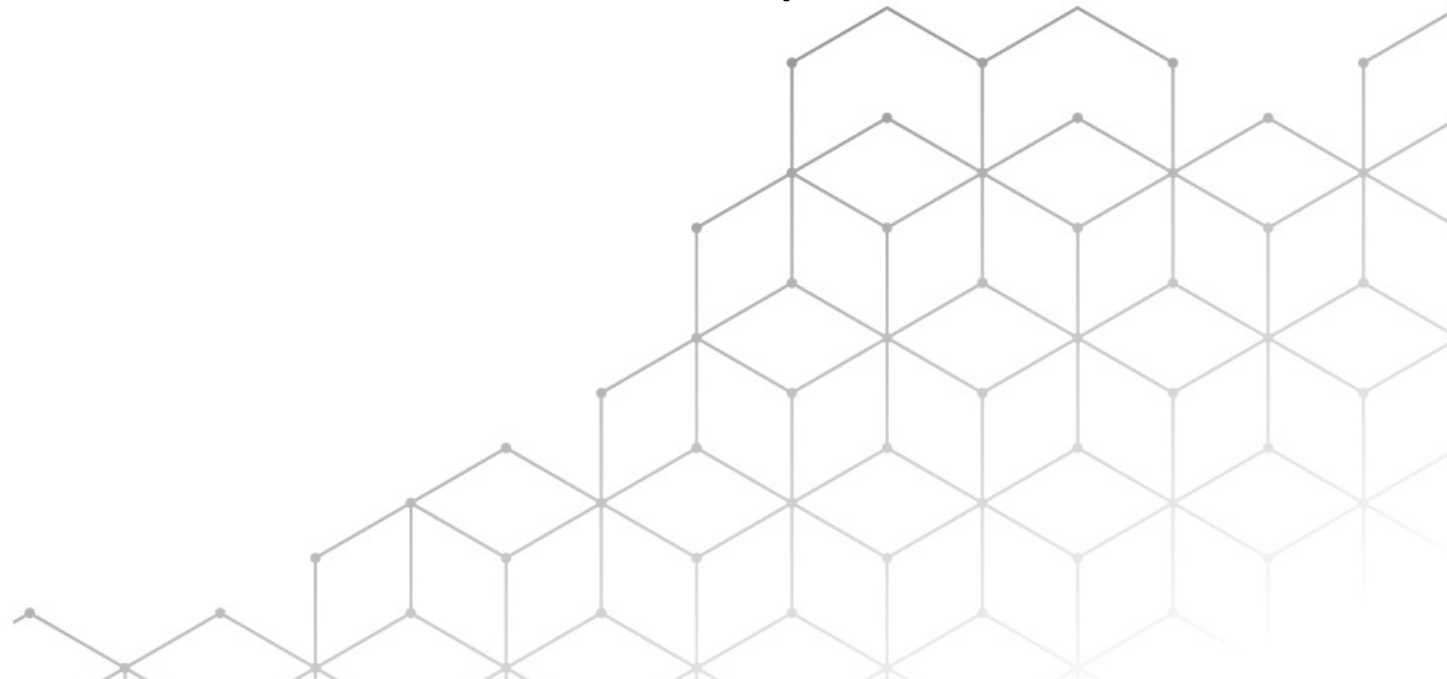
NoSQL: Why DBMS?



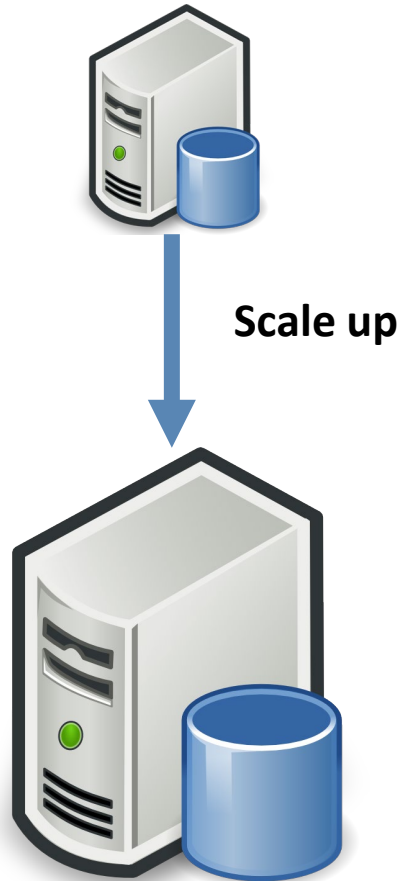
Scalability (Go Big or Go Many?)

Scalability refers to the ability to handle an increasing amount of work or its potential to be expanded to accommodate that growth.

- Example: A scalable database can handle more users, queries, or larger datasets (big data!).

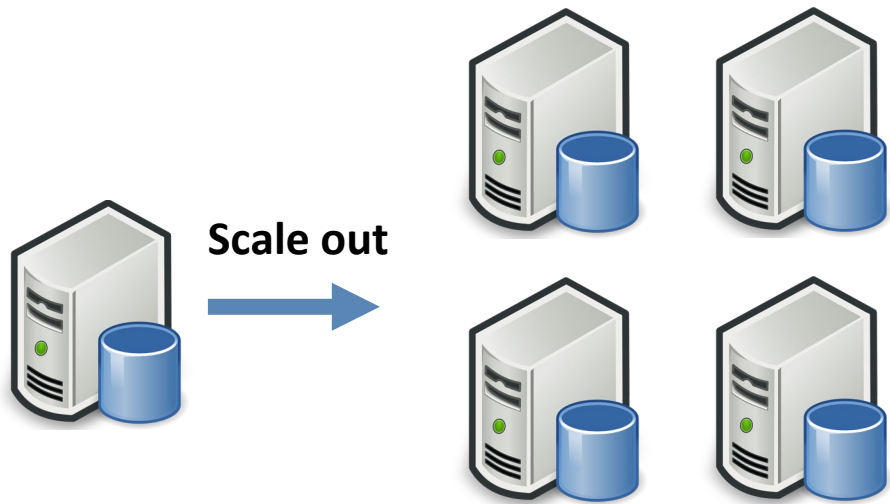


Scale up (Vertical Scaling)



- Move the database to a bigger box:
 - faster CPU
 - More cores
 - More memory
 - Faster disk
- Better performance,
- Extremely reliable.
- But extremely expensive and
- Inflexible to scale

Scale out (Horizontal Scaling)



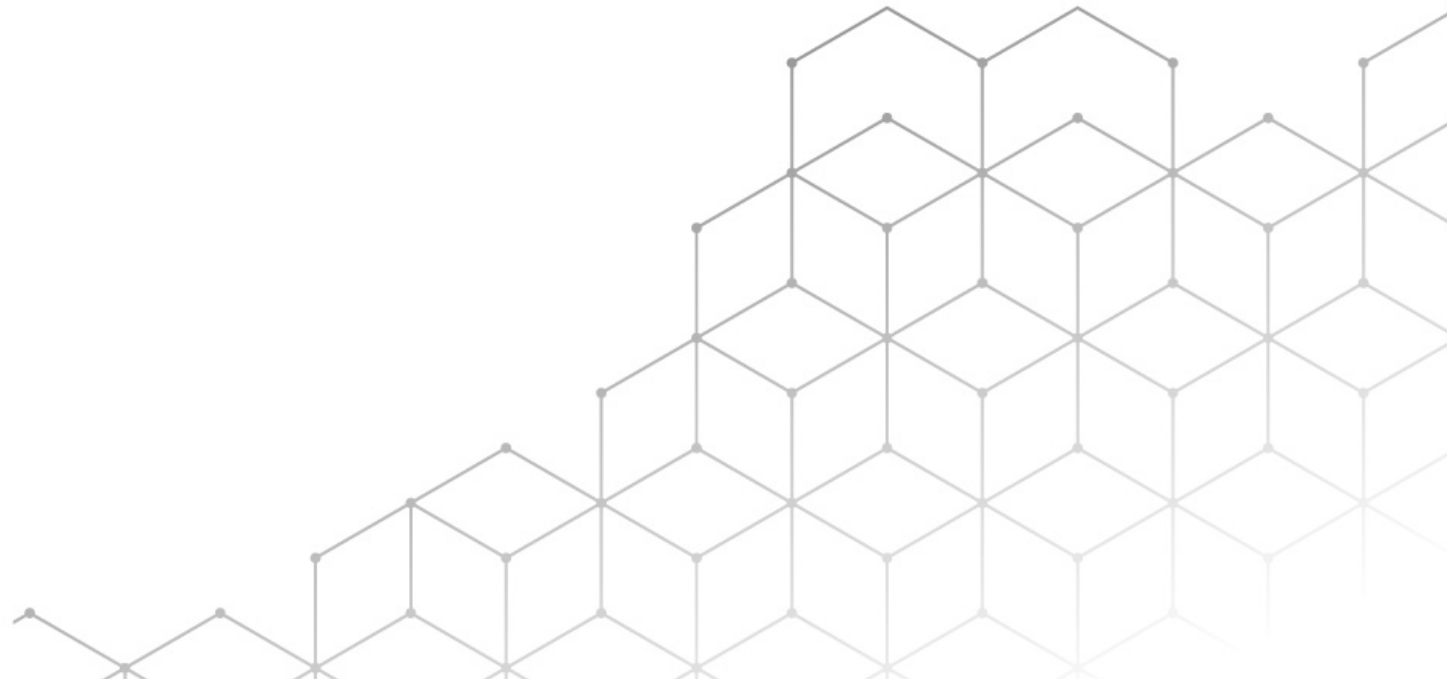
- Distribute the database across multiple nodes.
- Slower due to operational overheads
- More complex to engineer.
- But much cheaper, more flexible, and more fault tolerant

Scalability

- NoSQL DB does horizontal scaling.
- Sharding: Partitioning of data and placing it on multiple machines
- Examples of horizontal scaling databases are MongoDB, Cassandra
- **NoSQL can handle huge amounts of data because of scalability.** As the data grows, NoSQL scales itself to handle that data.
- **High availability:** The replication feature in NoSQL databases makes it easy in case of any failure, to recover

Flexibility of NoSQL

- In relational databases, you need to create the table, schema, set the data types of fields, etc., before you can actually insert the data.
- In NoSQL, you don't have to worry about that; you can insert and update data on the fly.



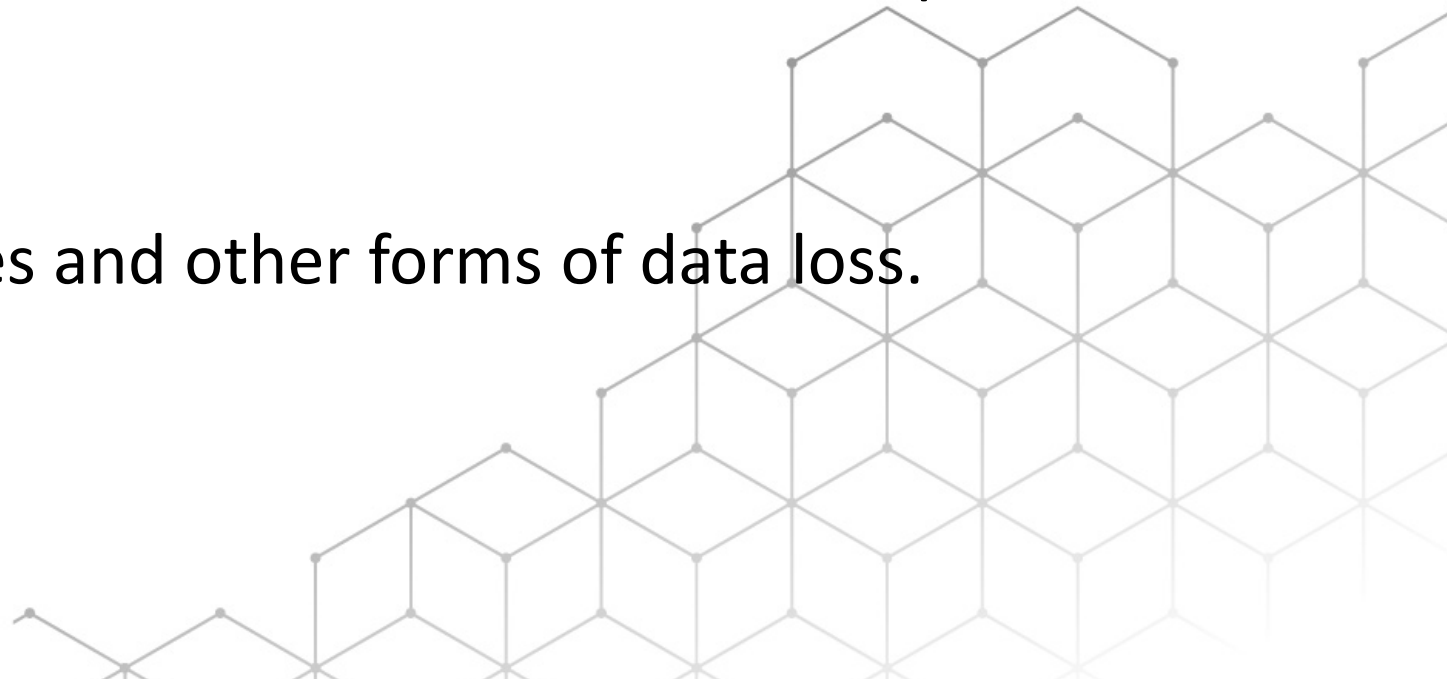
Performance of NoSQL

- Performance refers to how efficiently a database or system completes its tasks under specific conditions.
- It typically measures:
 - Response Time: How quickly a query or request is executed.
 - Throughput: The number of operations (e.g., transactions, queries) completed in a given time frame.
 - Latency: The delay in completing a task.
- In databases, performance ensures fast data retrieval, minimal lag, and efficient handling of concurrent requests.

Data Consistency: Big Challenge

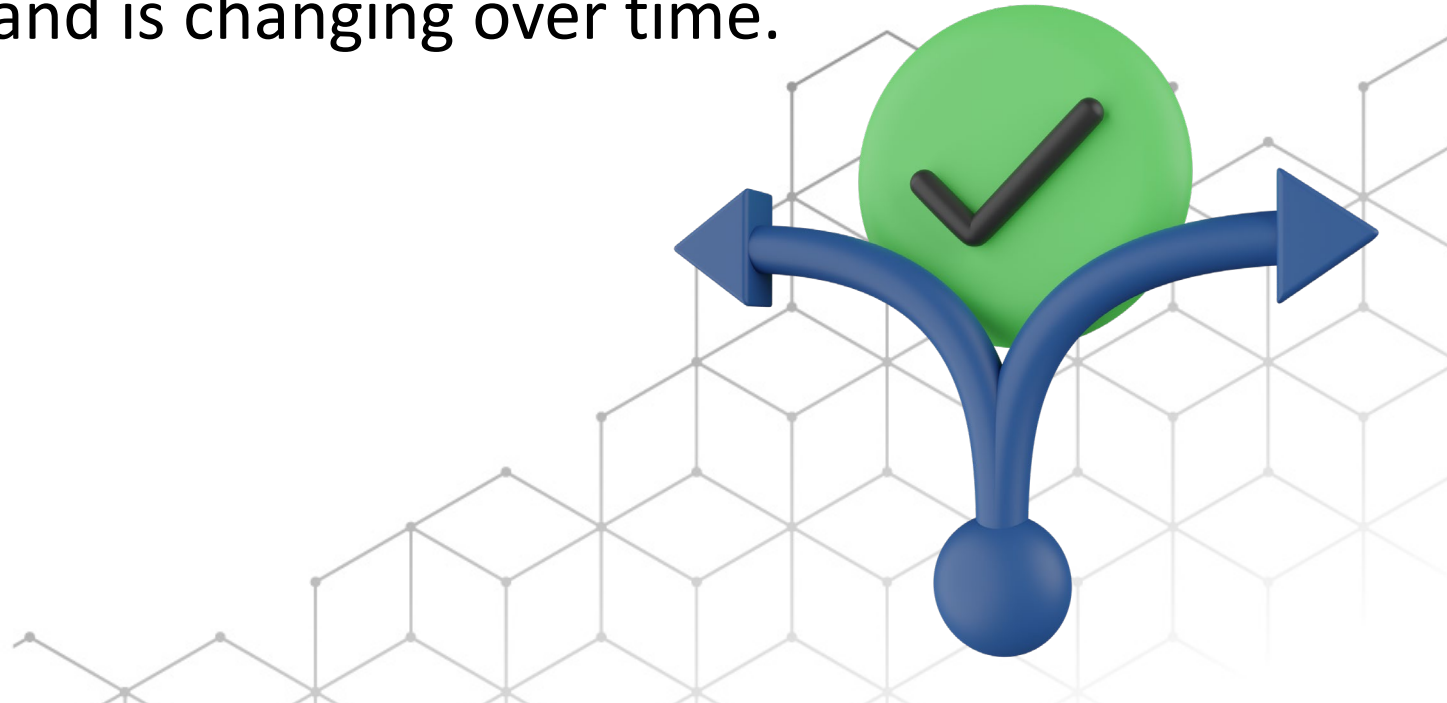
Stale reads problem:

- Most NoSQL databases offer a concept of **eventual consistency**.
- So queries for data might not return updated data immediately or might result in reading data that is not accurate, which is a problem known as stale reads.
- **NoSQL** may exhibit lost writes and other forms of data loss.



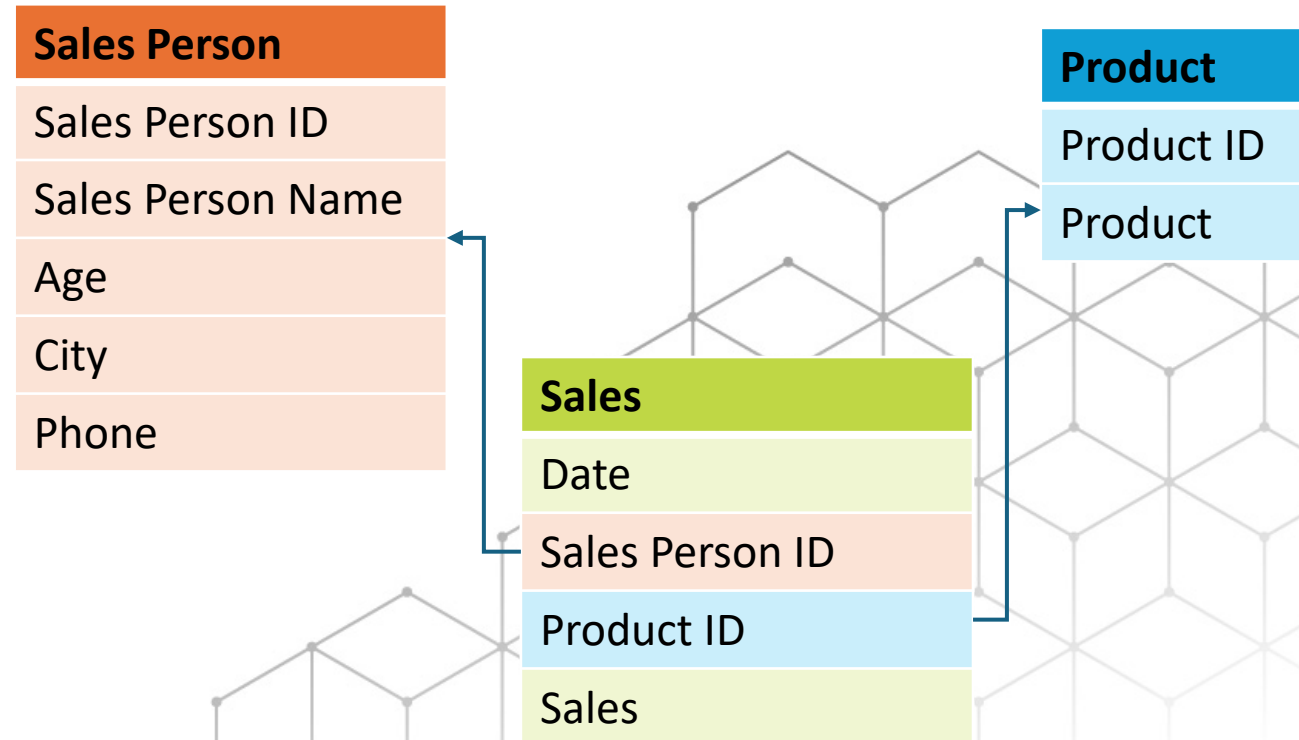
When to go for NoSQL

- A. When you want to store and retrieve a huge amount of data.
- B. The relationships in the data you store are not that important.
- C. The data is not structured and is changing over time.



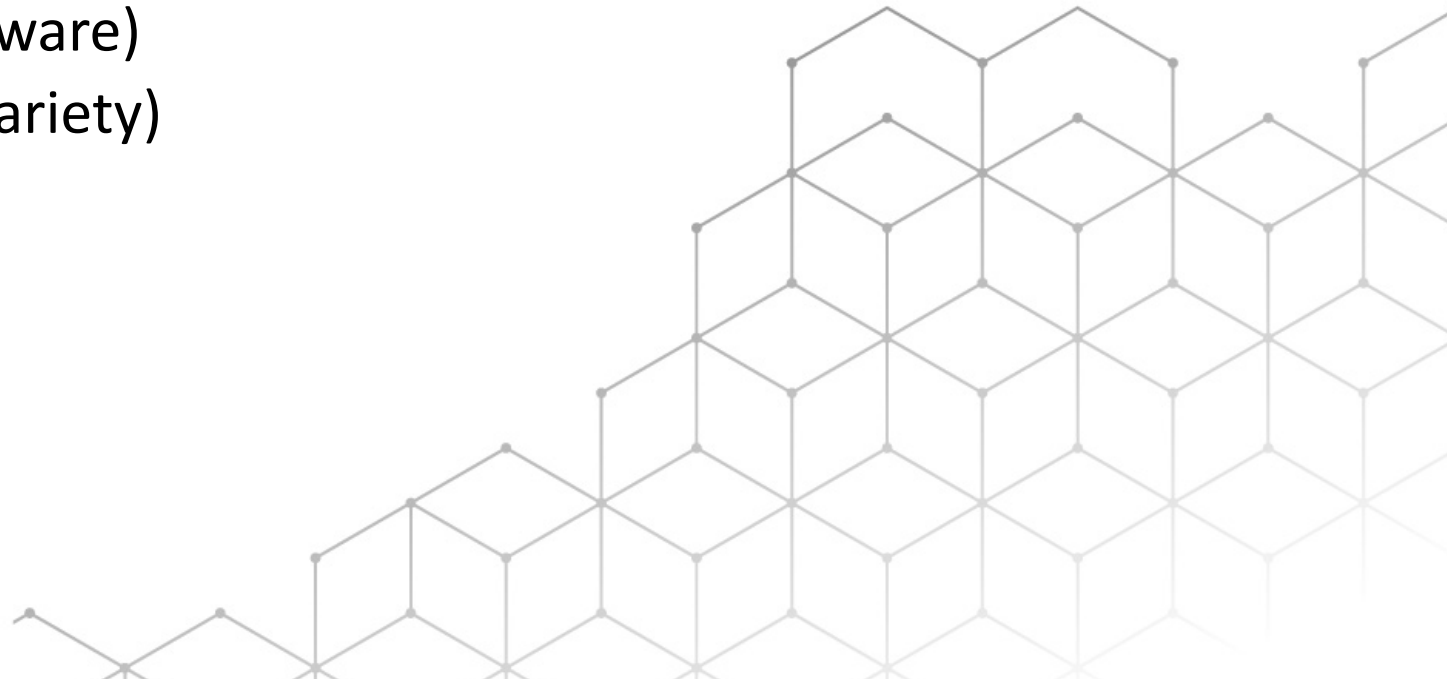
Benefits of Relational Databases

- Based on ACID properties
- Strong consistency, concurrency, recovery
- Normalization
- Standard Query language (SQL)
- Vertical scaling (up scaling)



Limitations of RDBMS

- Relational databases were not built for distributed applications.
- Because:
 - Joins are expensive
 - Hard to scale horizontally
 - Expensive (product cost, hardware)
 - The rise of big data (volume,Variety)



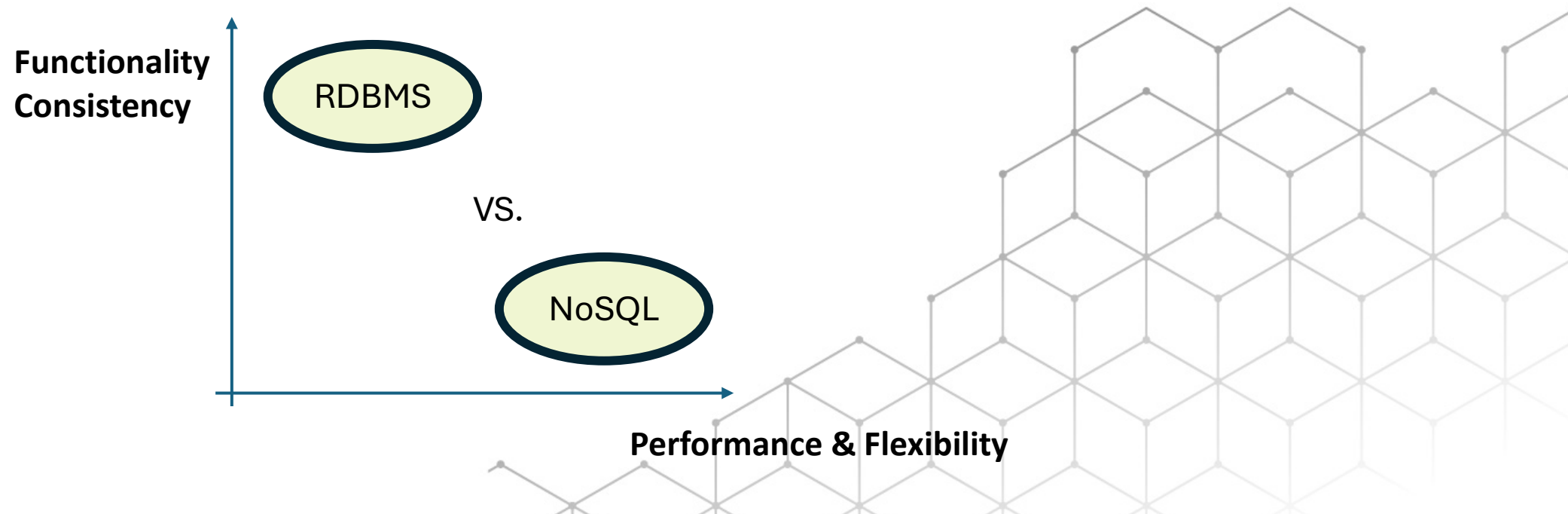
What is No SQL?

- It stands for 'NOT ONLY SQL'
- #NoSQL was a twitter hashtag for a conference in 2009 used by ERIC EVANS.
- There is no strict definition for NoSQL databases.
- It is a nonrelational database.
- Mainly designed to use for Big Data and Real time web applications.



RBDMS vs NoSQL

- RDBMS: structured data, more functionality but less performance.
- NoSQL: structured or semi-structured data, less functionality but high performance.



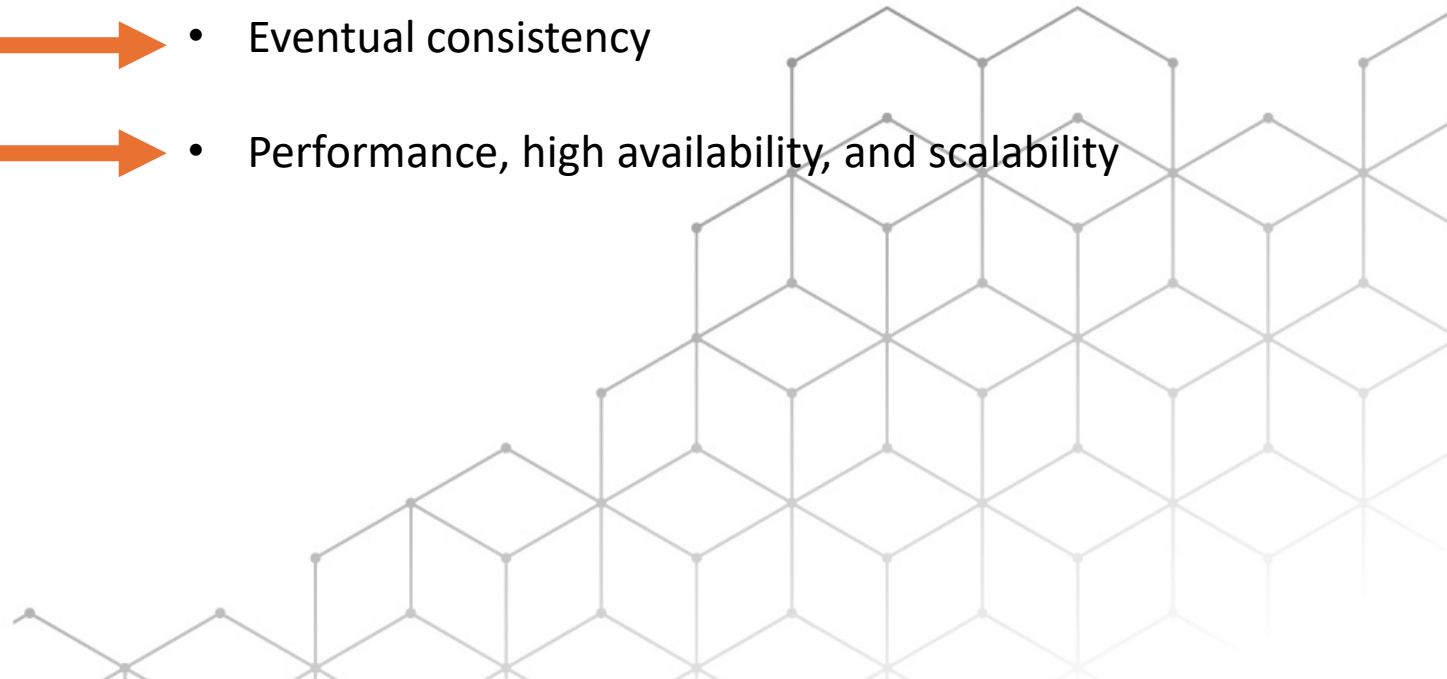
RDBMS vs NoSQL

RDBMS:

- Structured and organized data
- Structured Query Language (SQL)
- Data and its relationships stored in separate tables
- ACID Consistency
- Functionality

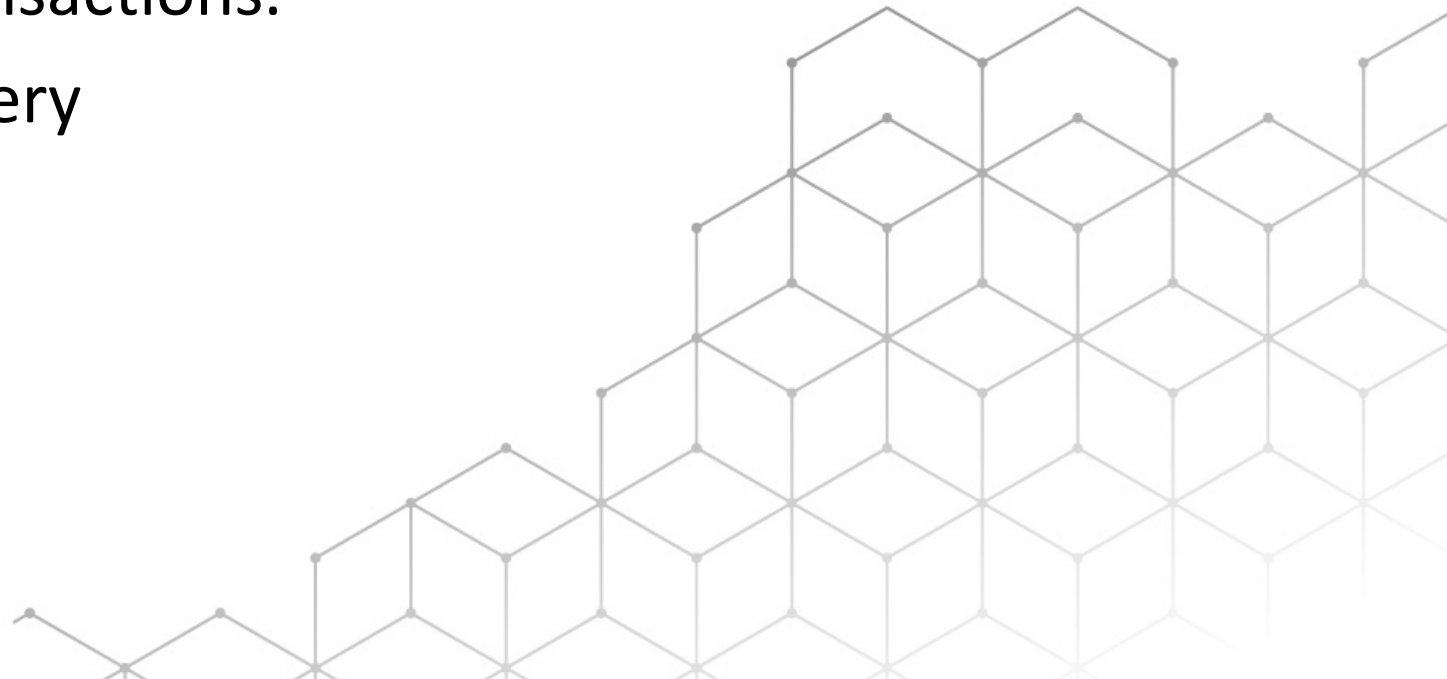
NoSQL:

- Unstructured and semi-structured data
- No declarative query language
- No predefined schema
- Eventual consistency
- Performance, high availability, and scalability



Advantages of NoSQL over RDBMS

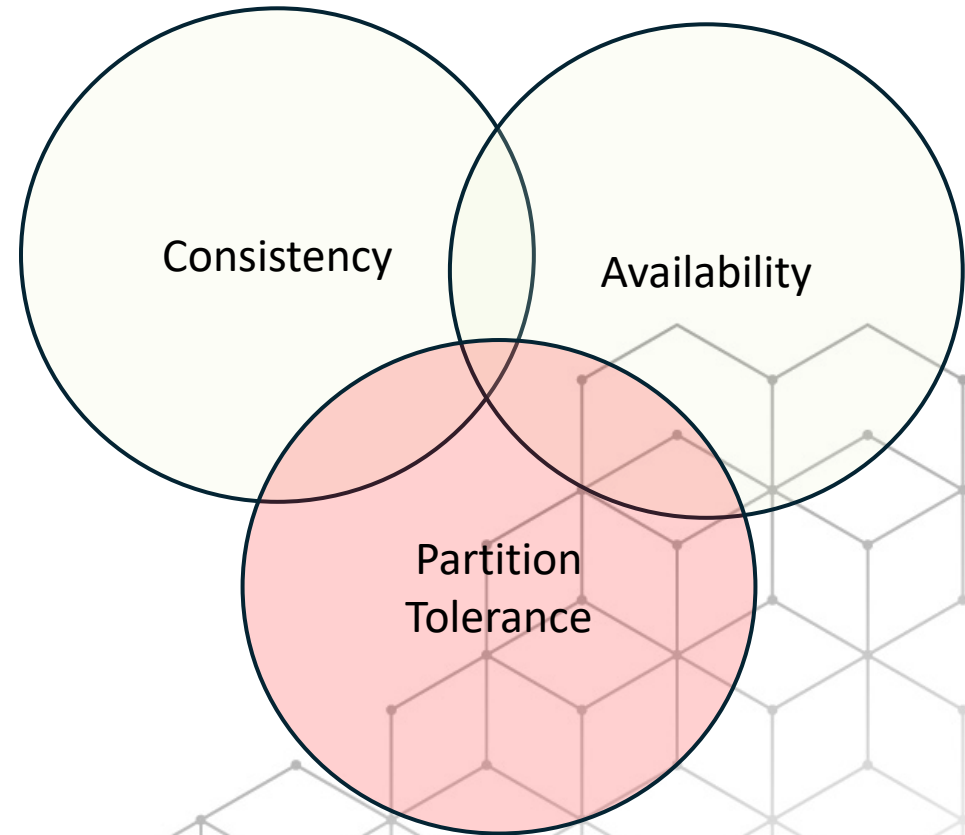
- Can handle Semi-structured and unstructured data.
- Data Models: No Predefined Schema.
- Scaling: Scaling out/ Horizontal Scaling.
- Avoids overhead of ACID transactions.
- Avoids complexity of SQL query



CAP Theorem

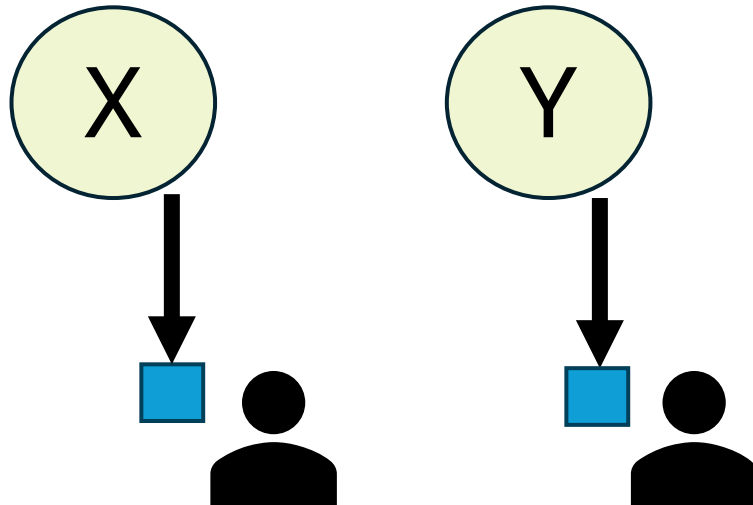
A distributed system can only guarantee two out of the following three properties at the same time:

- Consistency (C): Every read receives the most recent write or an error. In other words, all nodes in the system see the same data at the same time.
- Availability (A): Every request (read or write) receives a response, even if some nodes are down.
- Partition Tolerance (P): The system continues to operate, even if network partitions (communication failures between nodes) occur.

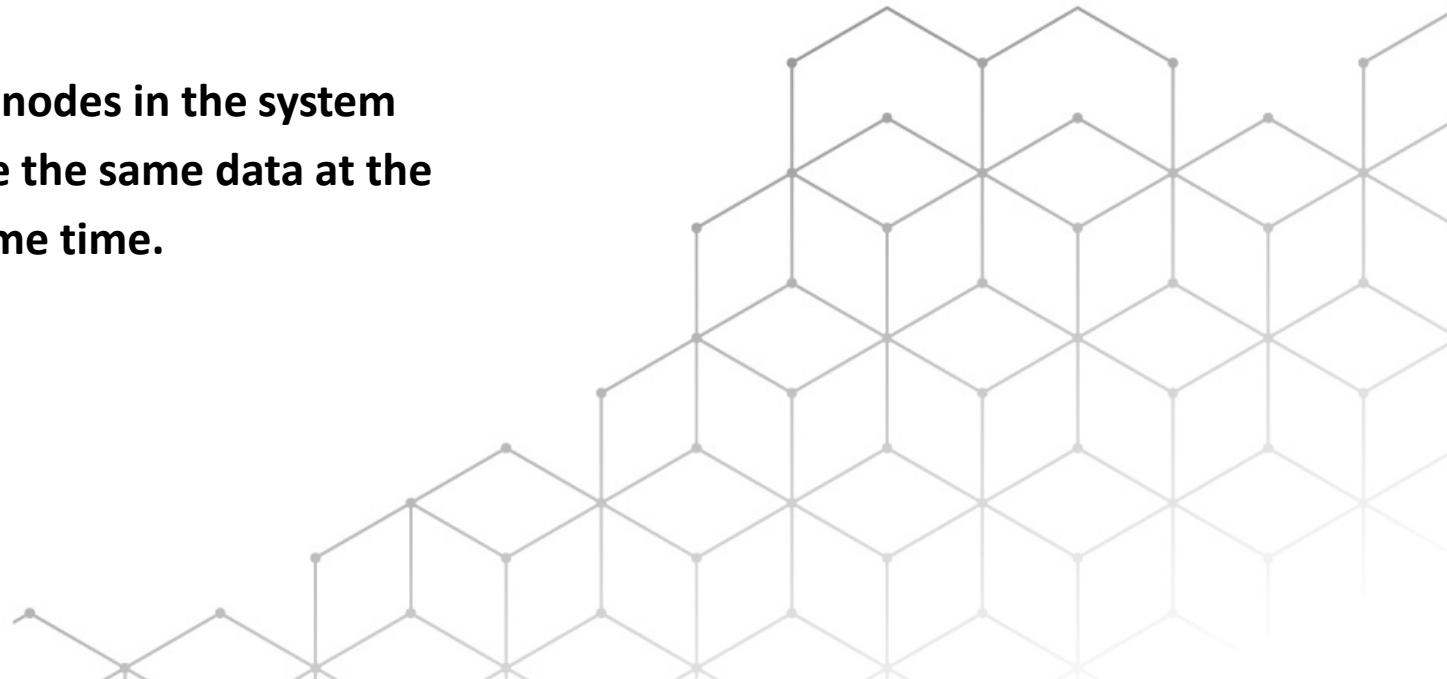


Consistency of CAP

- Consistency in CAP actually means linearizability, which is a very specific (and very strong) notion of consistency.
- It has got nothing to do with the C in ACID, even though that C also stands for “consistency”.

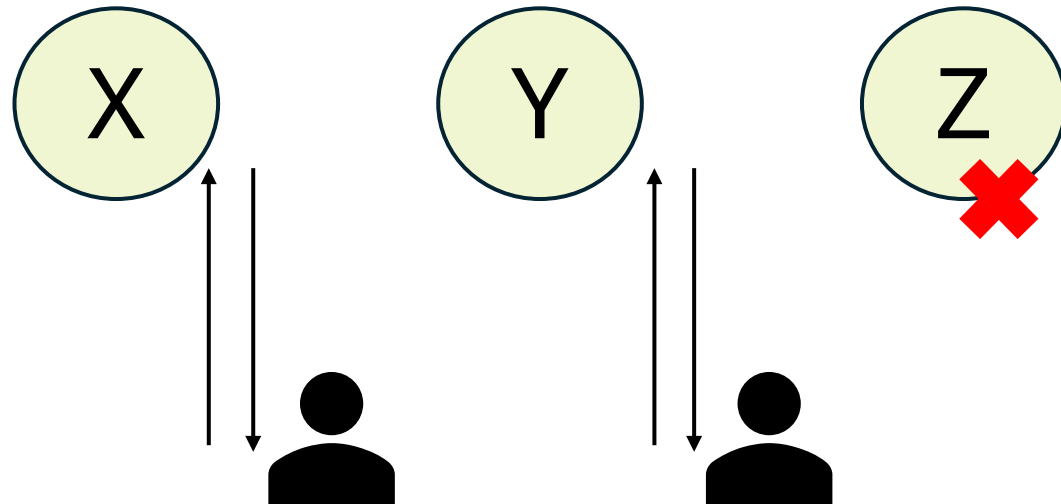


all nodes in the system
see the same data at the
same time.



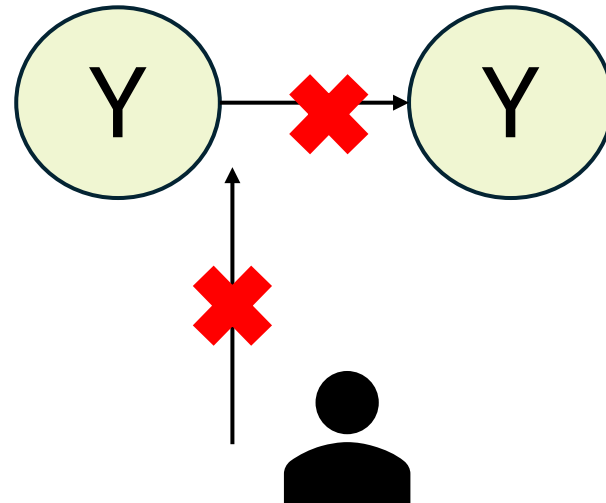
Availability of CAP

- CAP is defined as “every request received by a non-failing [database] node in the system must result in a [non-error] response”. It’s not sufficient for some node to be able to handle the request: any non-failing node needs to be able to handle it.



Partition Tolerance

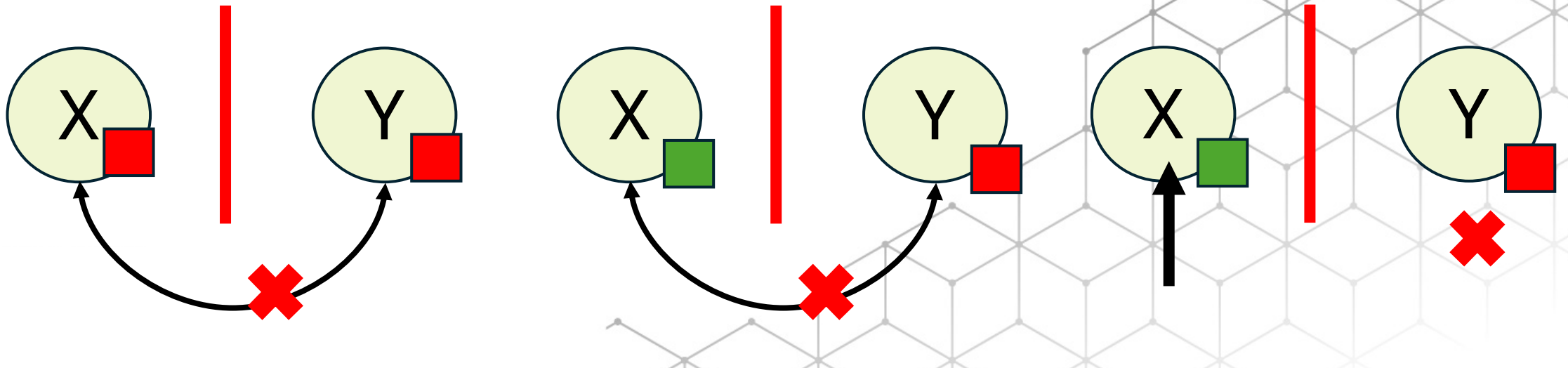
- (terribly mis-named) basically means that you're communicating over an asynchronous network that may delay or drop messages.
- That kind of fault does happen, but it's not the only kind of thing that can go wrong: nodes can crash or be rebooted, you can run out of disk space etc.



Explaining CAP Theorem

Consider two nodes, X and Y. There is a break in communication between X and Y, so they can't sync updates. You can either:

- A) Allow the nodes to get out of sync (giving up consistency), or
- B) Consider the cluster to be "down" (giving up availability)

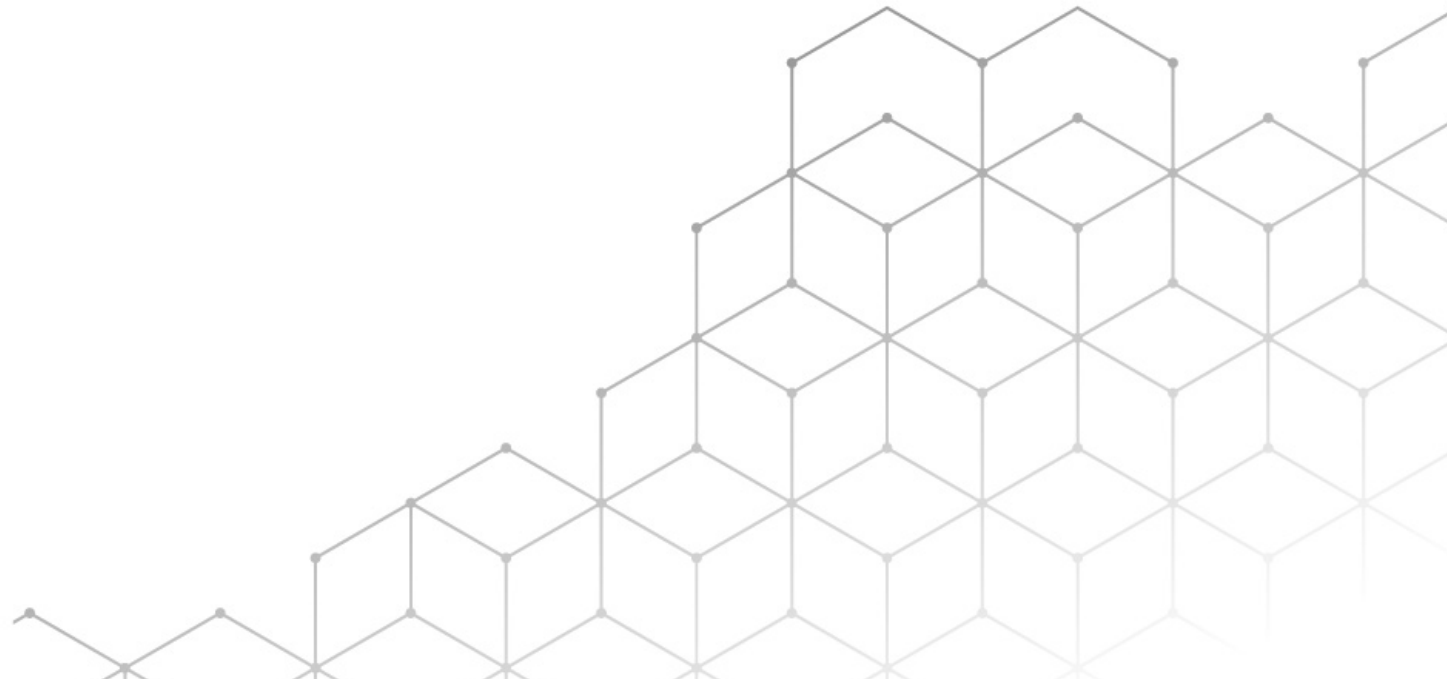


Why is it Important?

- Distributed Databases: The future of databases is distributed, particularly with trends like Big Data.
- CAP Theorem: This theorem explains the trade-offs inherent in distributed systems.
- Importance of Understanding: A proper understanding of the CAP theorem is critical for making informed decisions about distributed database design.
- Consequences of Misunderstanding: Misinterpreting the CAP theorem can lead to erroneous or inappropriate design choices.

CA (Consistent and Available)

- CA (Consistency and Availability): These systems ensure that all nodes have the same data and are always available. However, they cannot tolerate network partitions.
- Examples include traditional relational databases like MySQL and PostgreSQL

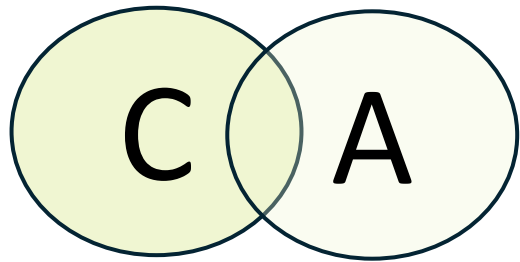


Problem of RDBMS to Scale

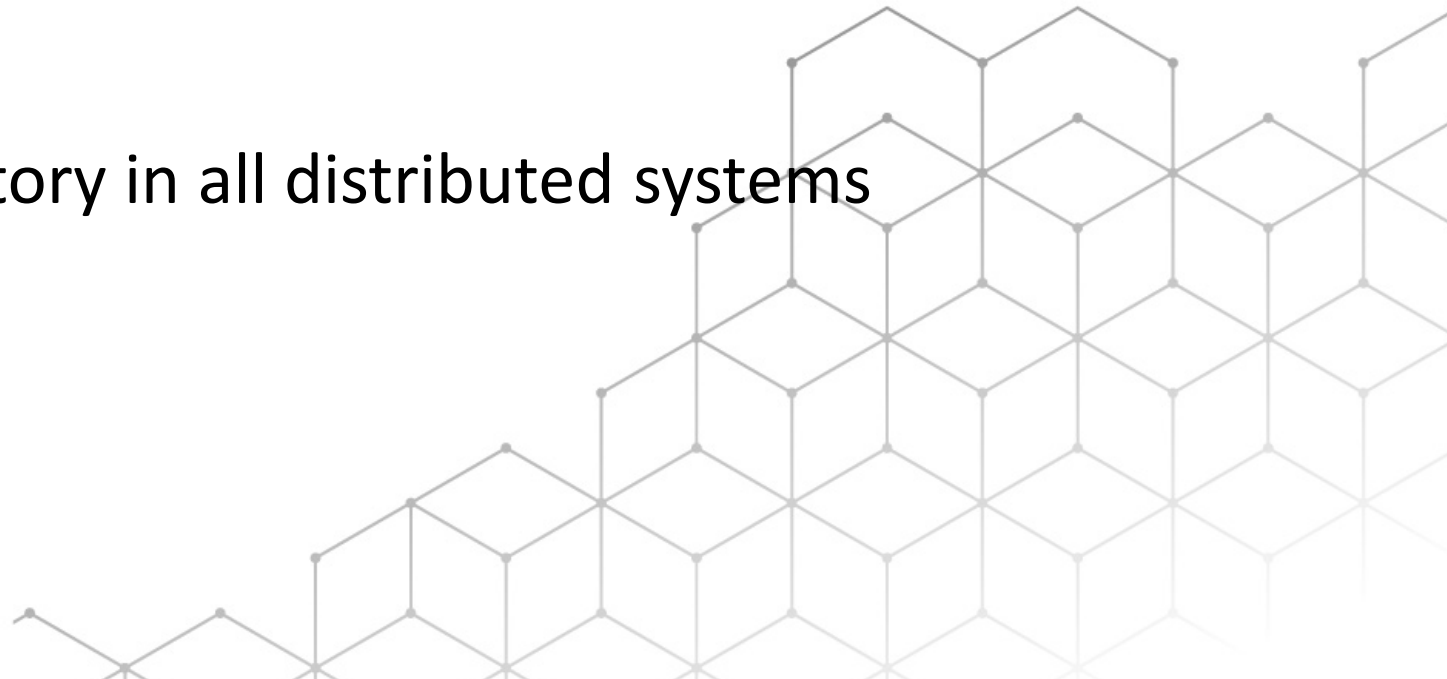
- ACID Principles: Relational databases follow the ACID model, ensuring:
 - Atomicity: Transactions are all-or-nothing.
 - Consistency: Data remains in a valid state after a transaction.
 - Isolation: Concurrent transactions do not interfere with each other.
 - Durability: Completed transactions are permanent.
- Distributed Systems Challenge: In a distributed relational database, achieving availability, consistency, and partition tolerance simultaneously is desired but **inherently impossible** due to the CAP theorem.

Partition Tolerance

- Can a distributed system (with an unreliable network) really be not tolerant of partitions?

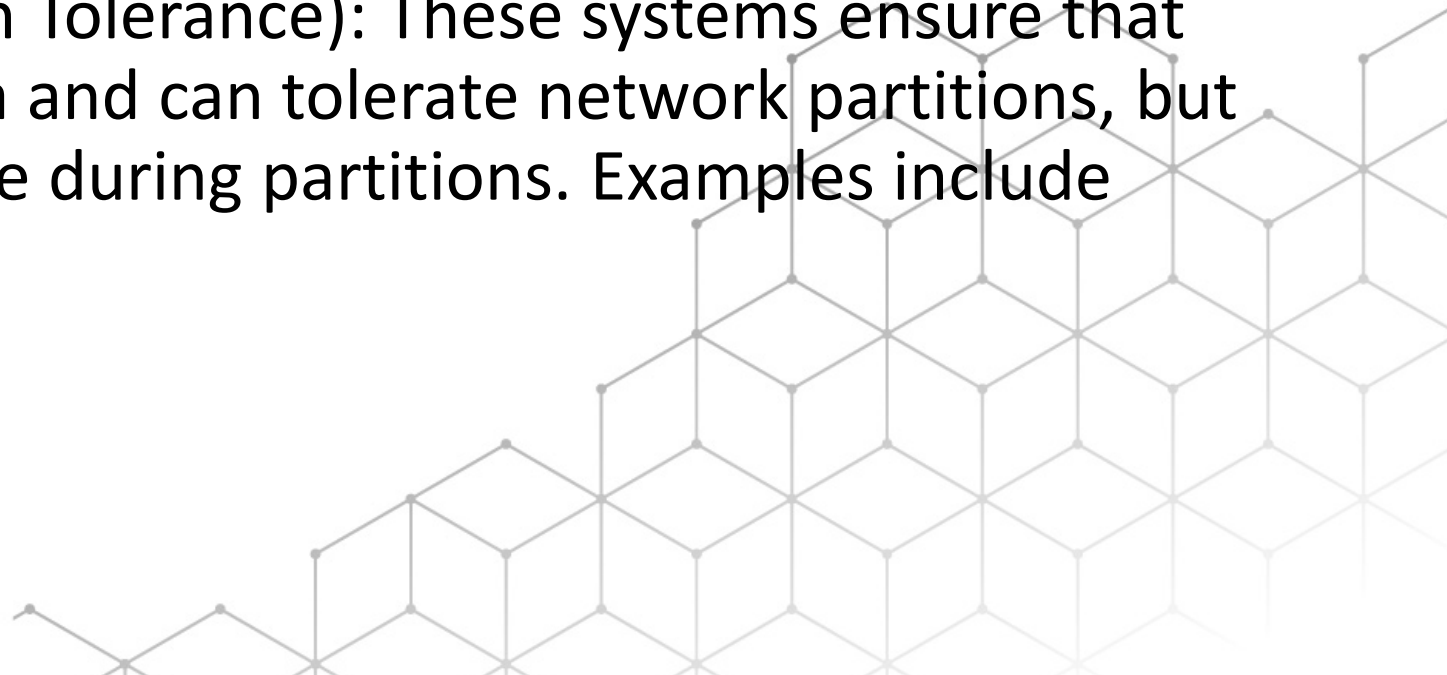


- Partition tolerance is mandatory in all distributed systems

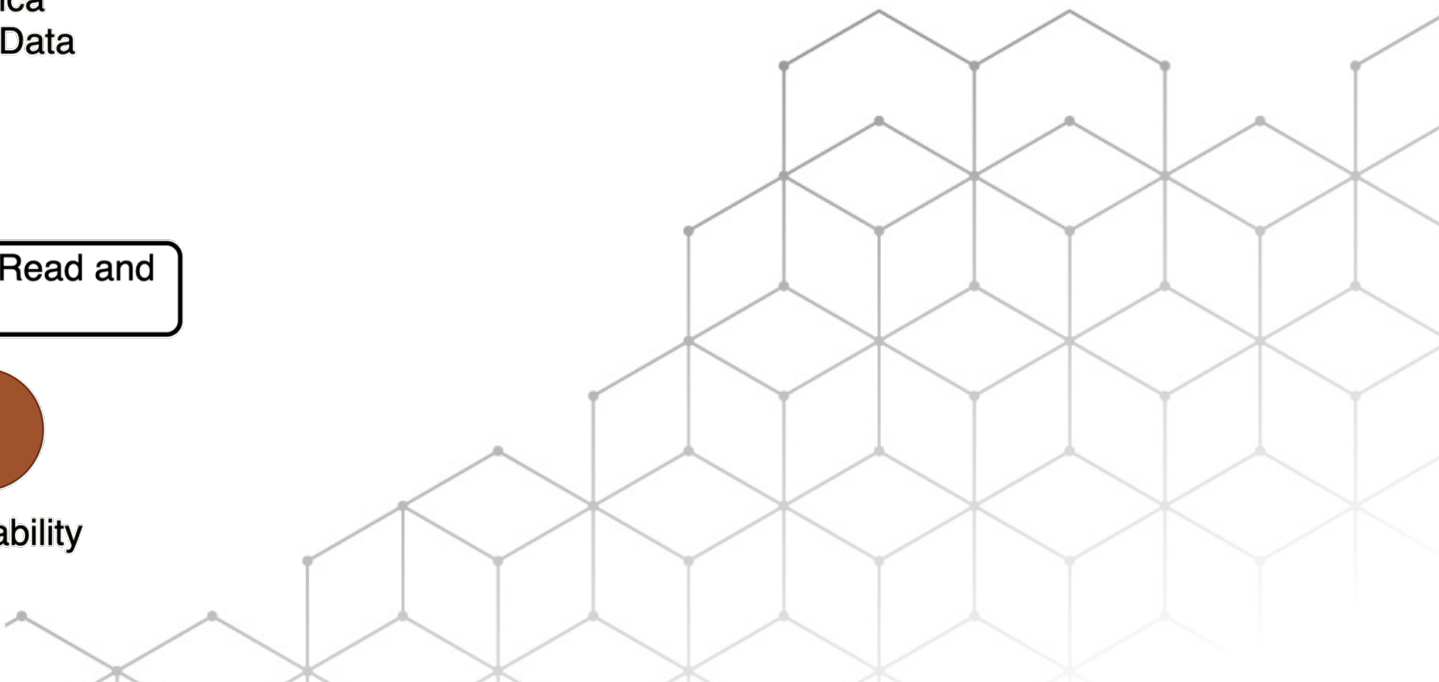
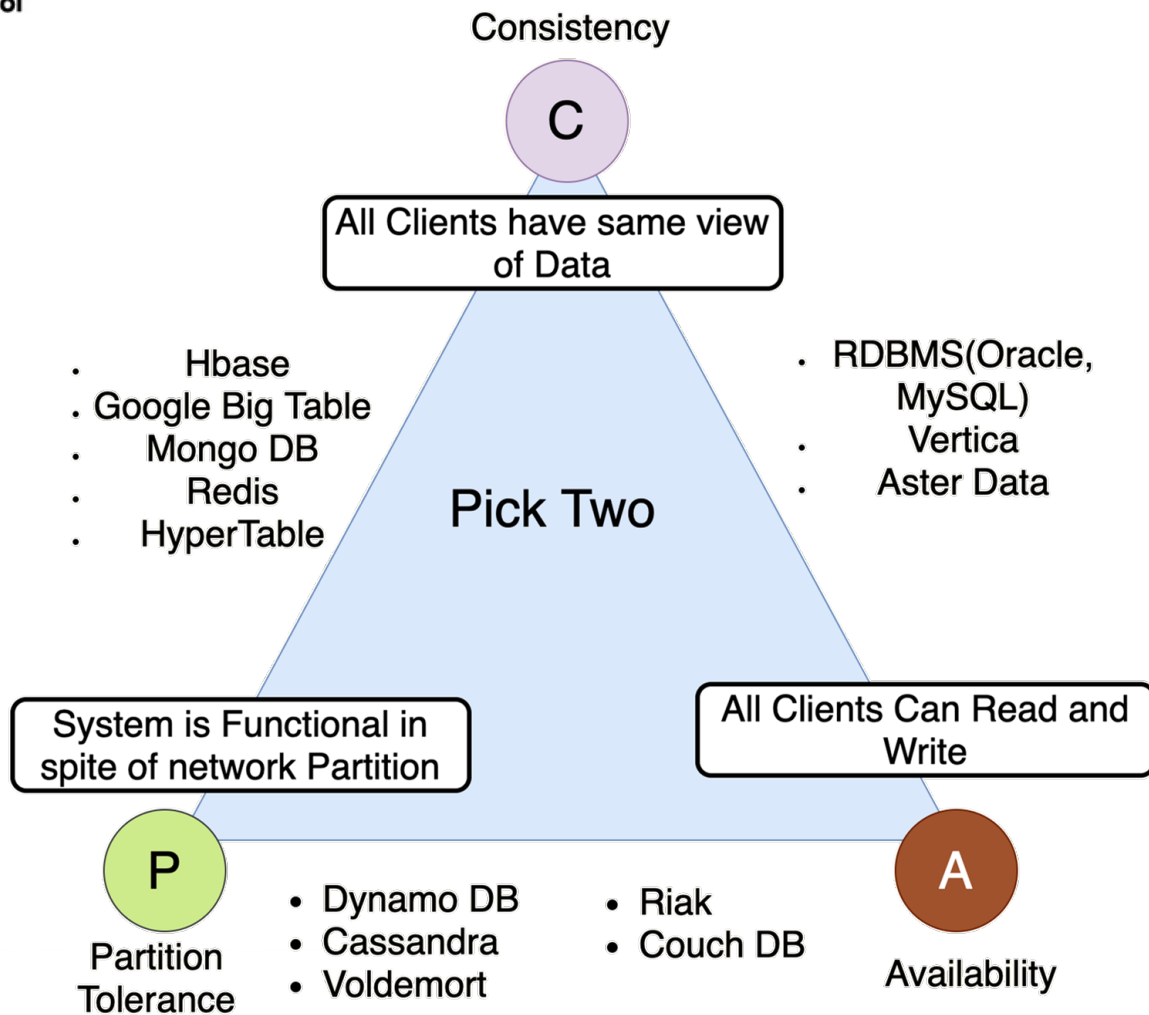


Other Systems

- AP (Availability and Partition Tolerance): These systems remain operational and responsive even during network partitions but may return outdated data. Examples include NoSQL databases like Cassandra and CouchDB
- CP (Consistency and Partition Tolerance): These systems ensure that all nodes have the same data and can tolerate network partitions, but they may become unavailable during partitions. Examples include MongoDB and HBase



CAP Theorem



Disadvantages (1)

- There is no reliable standard for NoSQL yet. In other words, two database systems are likely to be unequal.
- Management Challenge - The purpose of big data tools is to make the management of a large amount of data as simple as possible. But it is not so easy. Data management in NoSQL is much more complex than in a relational database.
- NoSQL, in particular, has a reputation for being challenging to install and even more hectic to manage on a daily basis.

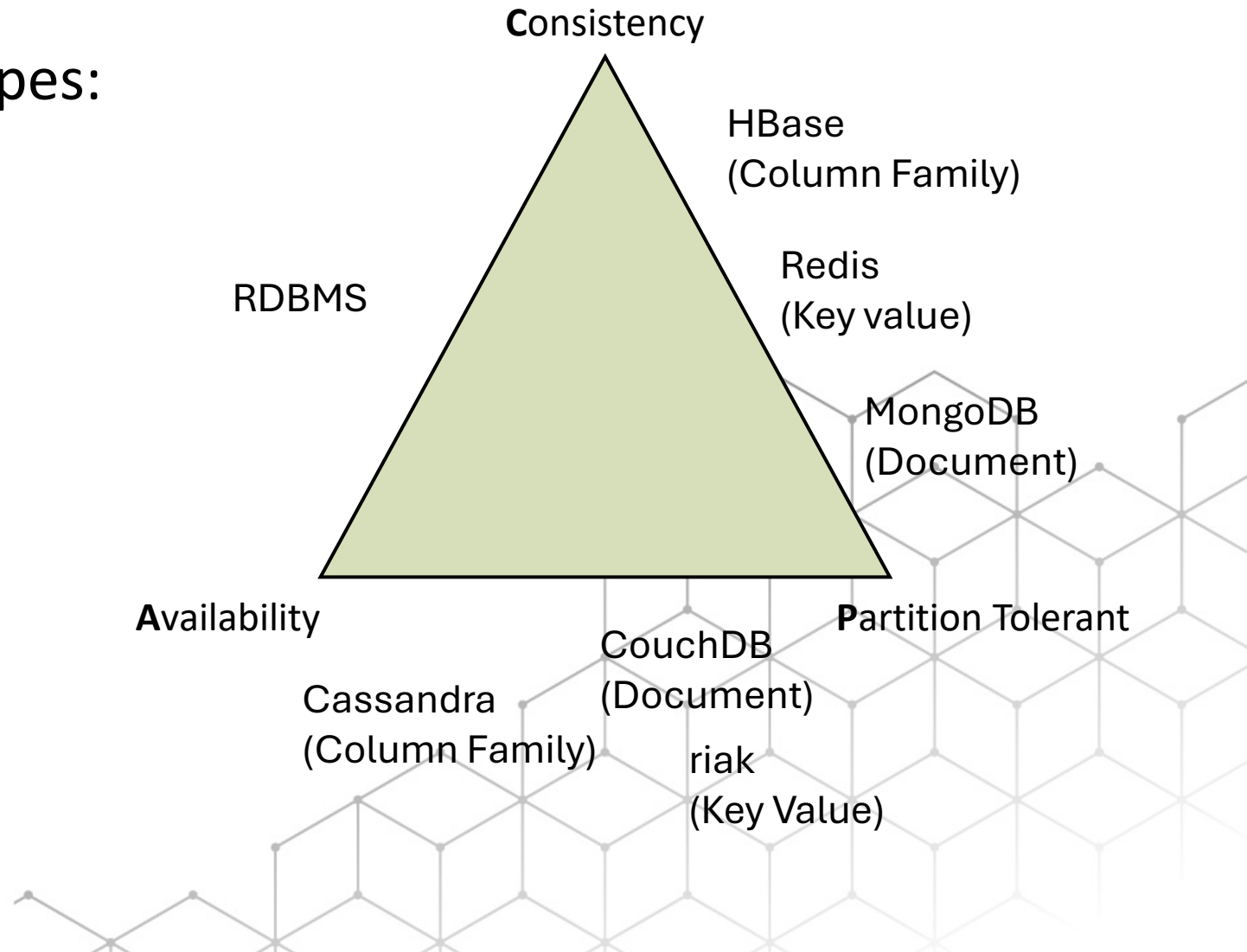
Disadvantages (2)

- GUI tools to access database are not flexibly available in the market.
- Backup - Backup is a great weak point for some NoSQL databases like MongoDB. MongoDB has no approach for the backup of data in a consistent manner.
- Large document size - Some database systems like MongoDB and CouchDB store data in JSON format. This means that documents are quite large (Big Data, network bandwidth, speed)

NoSQL Data Models

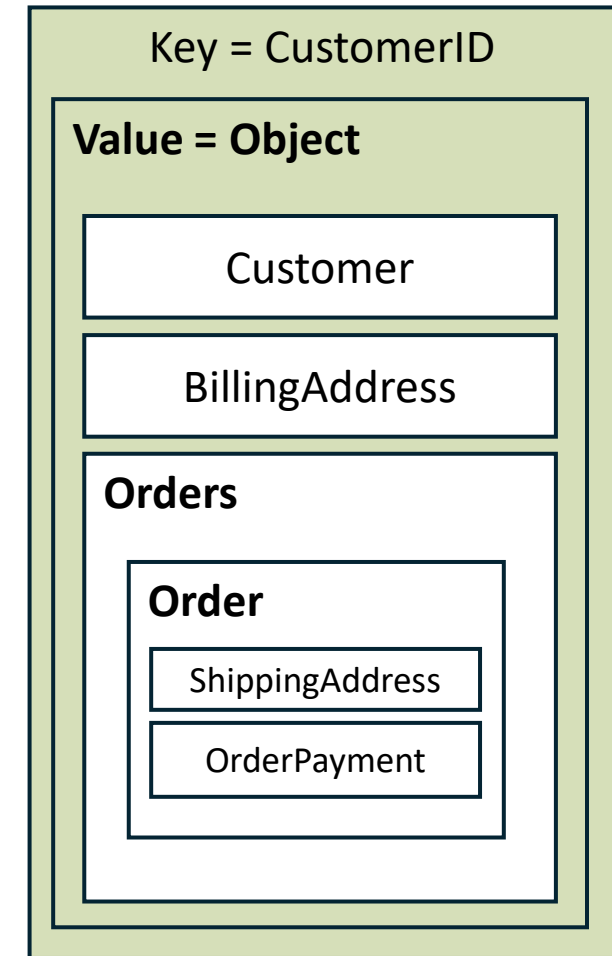
It can be classified into four types:

- Key Value pair based
- Column based
- Document based
- Graph based



Key Value Pair Based

- Data model: (key, value) pairs.
- Designed for processing dictionary.
- Records are stored and retrieved using a key that uniquely identifies the record, and is used to quickly find the data within the database.
- Example: Oracle NoSQL Database, Riak etc.



001	Manel
002	Alberto
003	Maria

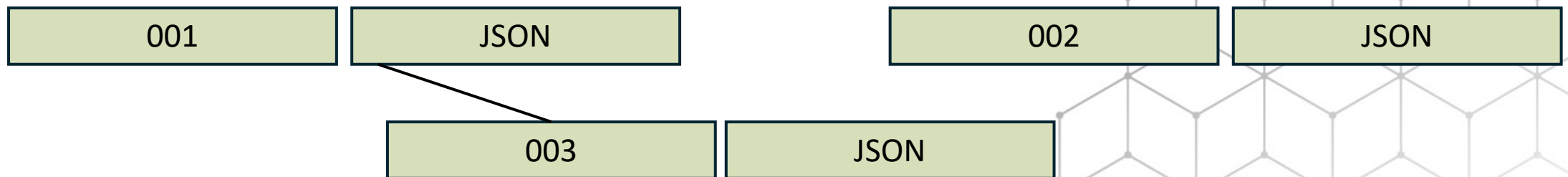
Column Based

- Data are Column families containing rows that have many columns associated with a row key. Each row can have different columns.
- Column families are groups of related data that is accessed together.
- Example: Cassandra, HBase, Hypertable, and Amazon DynamoDB.

001	Name1: value1	Name2: value2	Name3: value3
002	NameA: Value1	NameB: Value2	NameC: Value3

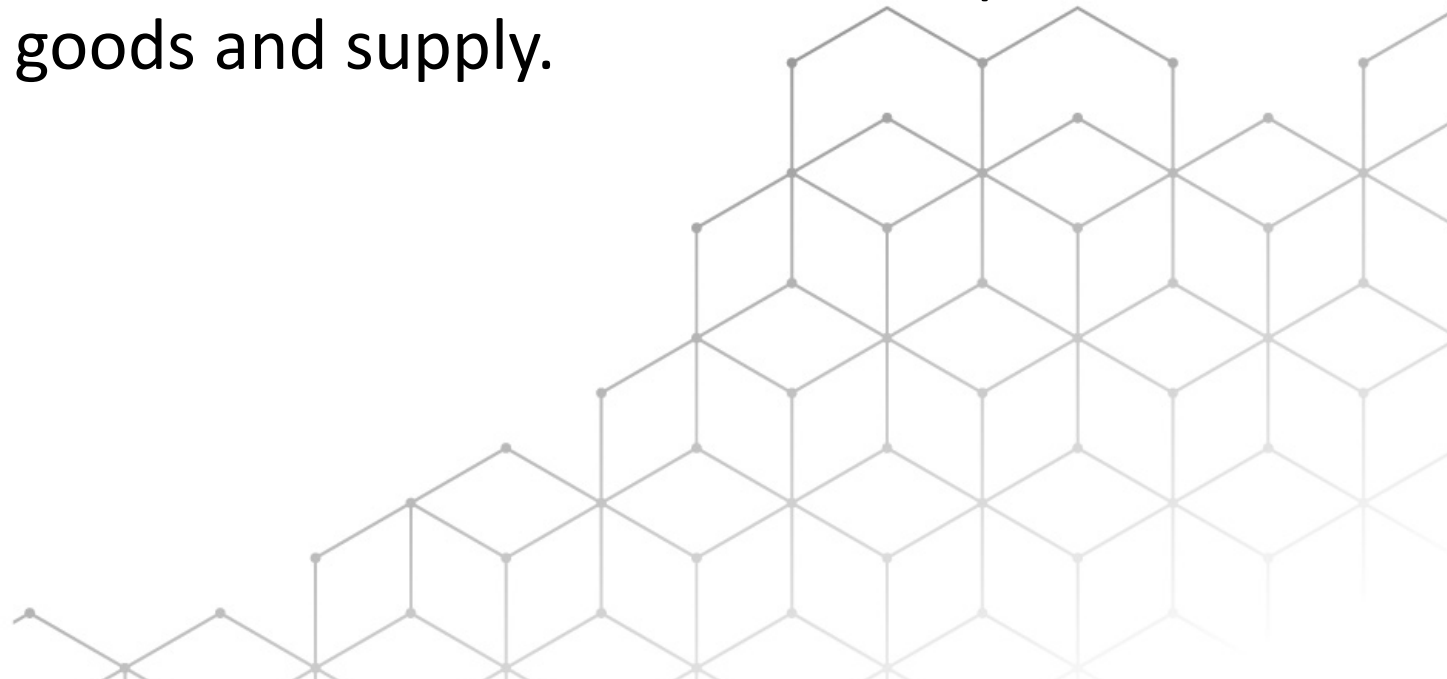
Document Based

- The database store send retrieves documents. It stores documents in the value part of the key-value store.
- Self describing, hierarchical tree data structures consisting of maps, collections, and scalar values.
- Example: Lotus Notes ,Mongo DB, Couch DB, Orient DB, Raven DB.



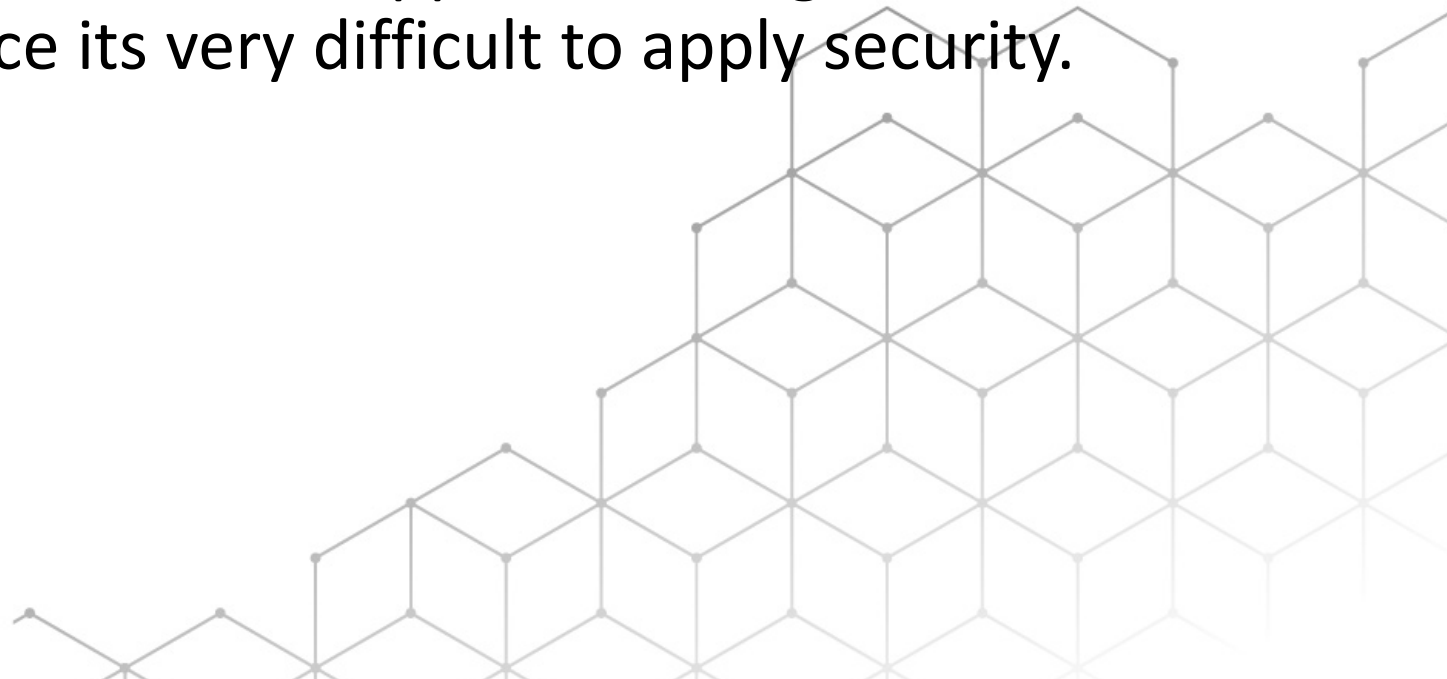
Graph Based

- Store entities and relationships between these entities as nodes and edges of a graph respectively. Entities have properties.
- Example: Neo4J, InfiniteGraph, OrientDB, FlockDB.
- It is well suited for connected data such as social networks ,spatial data ,routing information for goods and supply.



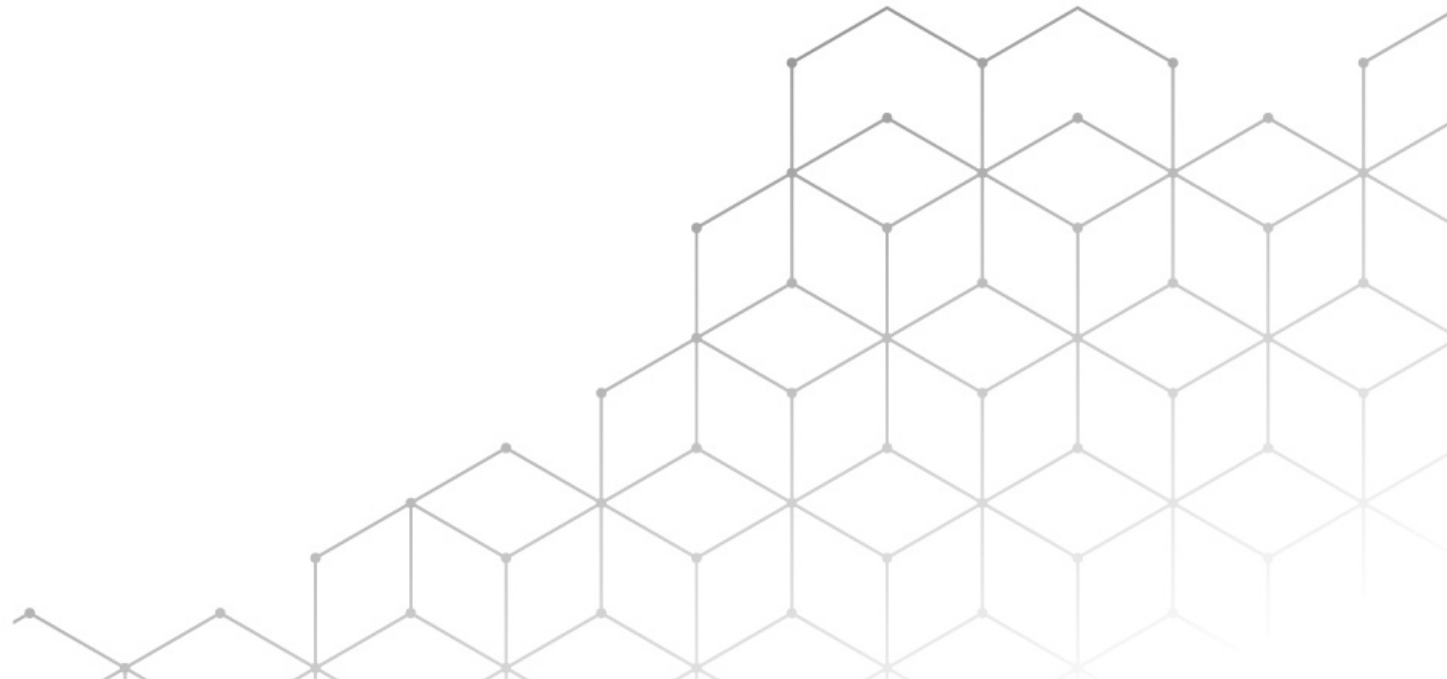
Open Challenges

- Transaction Processing : NoSQL do not strictly follow ACID properties.
- Query Processing: There is no user friendly unified query language for NoSQL.
- Security: Since follow unstructured data approach along with geographic distribution. Hence its very difficult to apply security.



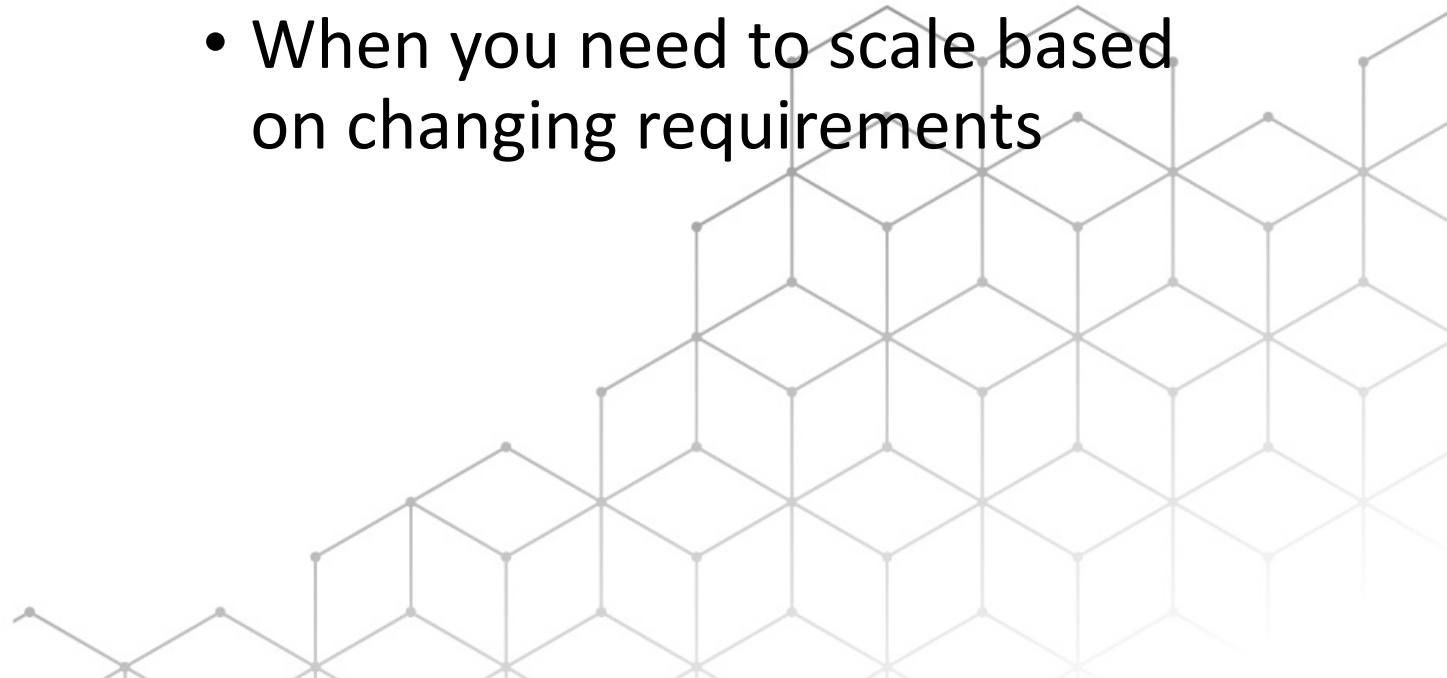
Why NoSQL **Now** Anyway?

- Internet Size
- Distributed database systems
- Rapid Development
- Polyglot persistence



Conclusion

- RDBMS is a great tool for solving ACID problems
- When data validity is super important
- When you need to support dynamic queries
- NoSQL is a great tool for solving data availability problems
- When it's more important to have fast data than right data
- When you need to scale based on changing requirements



Questions and Thank you

